Airbnb Senior ML Engineer

Onsite interview question bank

01. Loop Format and the Senior Bar: Airbnb Senior MLE (Experimentation / A/B Testing)
- Format notes
- Questions
- References

02. Coding Rounds: Airbnb-Tagged LeetCode and Reported Practical Coding Questions
- Format notes
- Questions
- References

03. ML System Design: Search Ranking, Embeddings, Pricing, Trust, Recommendations and More
- Format notes
- Questions
- References

04. A/B Testing and Experimentation Deep Dive: ERF, Marketplace Interference, Variance Reduction, Interleaving and Sequential Testing
- Format notes
- Questions
- References

05. ML Fundamentals and Modeling Depth: Ranking, GBDT vs. NN, Embeddings, Calibration, Features, Offline/Online Gap
- Format notes
- Questions
- References

06. General Backend System Design: Booking, Availability, Search, Messaging, Payments and Feature Store
- Format notes
- Questions
- References

07. Past Project / Technical Experience Deep Dive: Reported Prompts and Probing Follow-ups for Senior MLEs
- Format notes
- Questions
- References

08. Core Values Interviews: Champion the Mission, Be a Host, Embrace the Adventure, Be a Cereal Entrepreneur, plus Cross-Functional and Leadership Behavioral Questions
- Format notes
- Questions
- References

09. Airbnb ML Context: Papers, Platforms and Recent AI Products as Interview Questions
- Format notes (how this material surfaces in the loop)
- 1. Search ranking at Airbnb (KDD 2019 / KDD 2020 deep learning line)
- 2. Retrieval, location and semantic search
- 3. Embeddings, content understanding and recommendations
- 4. Listing quality, pricing, value and trust models
- 5. ML platform: Bighead, Zipline, Chronon
- 6. Metrics and experimentation context: Minerva, ERF, interleaving

7. 2024-2026 Airbnb AI: customer-service LLMs
8. 2025-2026 Airbnb AI: natural-language and AI search
9. Cross-cutting senior-level probes using Airbnb context
References

10. Whiteboard-Ready ML System Design Diagrams: Search Ranking, Embeddings, Pricing, Trust, LLM Support Routing and AI Search

How the tags work
Format notes
Diagram 0. The skeleton every diagram follows
Diagram 1. Stays search ranking (end to end)
Diagram 2. Listing-embedding personalization (similar listings and real-time session signals)
Diagram 3. Smart Pricing and demand forecasting
Diagram 4. Trust: payment fraud, account takeover (ATO) and off-platform payment
Diagram 5. LLM customer-support routing and assistance
Diagram 6. Natural-language AI search ("a quiet cabin near Lake Tahoe for 6 with a hot tub")
Diagram 7. Experimentation loop for ML launches
Diagram 8. The Chronon feature path (zoom-in for any diagram)
Cross-cutting probes (any diagram)
References

11. Timed ML Design Walkthroughs: Diagram Templates, Timing Checklist and Stage-by-Stage Questions for Search Ranking, Host Pricing, Guest LTV, Listing Recommendation, Relisting Detection and Support Retrieval

How the tags work
Format notes for this round
Reusable diagram template (draw this in the first 8-10 minutes)
Timing checklist (45-60 minute round)
Walkthrough A: Search ranking (home search, with and without a text query)
Walkthrough B: Host dynamic pricing (helping hosts set prices)
Walkthrough C: Guest (customer) and listing lifetime value
Walkthrough D: Listing recommendation (similar listings, homepage, email, cover photo)
Walkthrough E: Relisting detection (newly reported prompt)
Walkthrough F: Retrieval and ranking for generative-AI support (newly reported prompt)
Cross-cutting experimentation probes (asked in any walkthrough)
Senior-level trade-off probes (final 5-7 minutes)
References

12. ML Platform and Experimentation Pipeline Diagrams: Bighead, Zipline/Chronon, ERF/Minerva and Search Interleaving

How the tags work
Format notes
Diagram A. Bighead model lifecycle (Airbnb's end-to-end ML platform)
Diagram B. Zipline/Chronon: point-in-time-correct backfill and online serving
Diagram C. ERF/Minerva: assignment, exposure logging, metric computation and analysis
Diagram D. Interleaving pipeline for search ranking
Cross-diagram probes (connecting A, B, C and D)
Section E. Reported prompts mapped onto the four diagrams (added in review)
References

01. Loop Format and the Senior Bar: Airbnb Senior MLE (Experimentation / A/B Testing)

> Research note (read first). This chapter went through an adversarial review on 2026-10-14. During that review the egress proxy blocked igotanoffer.com, glassdoor.com, leetcode.com, medium.com, arxiv.org, dl.acm.org and careers.airbnb.com, and the shared web-search budget was already used up. Only one source could be fetched and checked line by line: the community-collected Airbnb question list on GitHub [11]. Every question tagged **[Reported]** below is supported by that verified source. Questions tagged **[Reported, unverified]** are attributed to Airbnb by sources [1]-[5], which could not be re-fetched in this review. Treat them as plausible but unconfirmed. **[Likely]** means an inference from the role, the level and Airbnb's products, not a candidate report. Source [11] was compiled around 2017-2020 and says Airbnb may have rotated its question pool since, so the 2024-2026 loop could not be confirmed here. A recruiter's prep packet always overrides this chapter.

Format notes

Stage	Typical length	Interviewer	What is assessed (Senior / L5 bar)
Recruiter call	20-30 min	Recruiter	Motivation ("Why Airbnb?"), level calibration (L5 = Senior), team fit (Search/Ranking, Pricing, Trust, Payments, Relevance, Experimentation platform), comp, timeline [1][2]
Technical phone screen	45-60 min	Engineer	One or two practical coding problems on a shared editor (CoderPad-style). The code is expected to **compile and run against test cases**. Pseudocode does not pass [1][2][3]
Onsite: Coding x2	45-60 min each	Engineers	Runnable, clean, tested code. One round is often algorithmic (LeetCode-medium/hard, many Airbnb-tagged problems). The other is a longer "practical" problem with evolving requirements (parsing, pagination, scheduling, simulation) [1][3][4][11]
Onsite: ML design	45-60 min	Senior/Staff MLE	End-to-end ML system for an Airbnb product (search ranking, pricing, fraud, recommendations), including offline/online evaluation and **experiment design** [Likely; 6-10]
Onsite: System design	45-60 min	Senior/Staff engineer	Scalable services and data pipelines (for example a booking system, an experimentation/assignment service, a feature store) [1][2]
Onsite: Past project deep dive	45-60 min	Hiring manager or senior IC	Ownership, technical depth, trade-offs and measured impact of a project you led. Sometimes presented with slides [1][2]

Stage	Typical length	Interviewer	What is assessed (Senior / L5 bar)
Onsite: Core values x2	30-45 min each	Employees **outside** the hiring team (often non-engineers)	Airbnb's four core values: *Champion the Mission, Be a Host, Embrace the Adventure, Be a "Cereal" Entrepreneur*. A strong fail here can veto an otherwise strong loop [1][2][5]
Debrief / hiring committee	n/a	Panel + leadership review	Calibrated on level. Senior candidates are expected to drive ambiguity, lead cross-functional work and show measured impact [1][2]

Key expectations that candidates consistently report:

- **Runnable code is mandatory.** Interviewers run your code. Syntax errors, unhandled edge cases or no tests count against you even if the logic is right [1][3].
- **Practical over puzzle.** Problems are often framed as real product scenarios: listings pagination, host search results, price rounding, travel itineraries, calendar availability [3][4][11].
- **Core values are scored as seriously as the technical rounds** and are run by people with no stake in the team [1][5].
- **Senior bar (L5):** you independently own an ambiguous problem end to end. You make clear trade-offs (latency vs. quality, bias vs. variance in experiment estimates). You influence partners such as Product, Data Science and Design, and you mentor others. Coding and design are expected to be done without heavy hints [Likely; 1][2].
- **Experimentation-flavored MLE roles:** Airbnb runs a mature in-house experimentation platform (ERF) and has written publicly about A/B testing pitfalls, interleaving for search ranking and two-sided-marketplace effects. Expect experimentation probes inside the ML design and past-project rounds [6][7][8][9].

Questions

A. Recruiter call and process logistics

1. Why Airbnb, and why now? [Reported, unverified] [1][2]
    - Which Airbnb product surface (Search, Pricing, Trust, Experiences, Payments) would you most want to work on, and why?
    - Have you used Airbnb as a guest or as a host? What would you change?
2. Walk me through your background in under three minutes, focusing on ML in production. [Reported, unverified] [1]
3. What level are you targeting, and what scope of work did you own at your current level? [Likely]
    - How many engineers or scientists did you lead or influence on your largest project?
4. Which programming language will you use in coding rounds, and are you fluent enough to run code without IDE help? [Likely] [3]
5. How much of your recent work was modeling, and how much was infrastructure or experimentation? [Likely]

6. Have you designed or analyzed online A/B tests yourself, or did a data scientist own that? [Likely]
7. What are your compensation expectations and timeline, and do you have competing processes? [Reported, unverified] [2]
8. Are you open to team matching across Search/Relevance, Pricing, Trust and the Experimentation platform? [Likely]

B. Technical phone screen (practical coding, must run)

9. Display page: given CSV strings of search results (host_id, listing_id, score, city) sorted by score and a page size of 12, reorder them so a host appears at most once per page if possible, otherwise preserving order, and print the pages separated by a blank line. [Reported] [11]
    - What happens on the last page when only one host's listings remain?
    - What is the time complexity, and can you do it in a single pass?
10. Round prices: round each price to its floor or ceiling so the rounded sum equals Round(sum of prices), minimizing the total absolute rounding error. [Reported] [11]
    - What if the target is unreachable? How do you signal that?
    - How would you handle currency with two decimal places?
11. Pour water: given terrain heights, a water quantity and a pour position, simulate the droplets and print the final terrain with water. [Reported] [11]
    - How do you test the boundary indices?
12. Text justification: fully justify lines of words to a fixed width. [Reported] [11]
    - How do you handle a single word on a line, and the last line?
13. List-of-lists iterator with `hasNext`, `next` and `remove` (remove the last returned element; calling it twice is an error). [Reported] [11]
14. Implement a CSV parser that handles quoted fields, escaped quotes and commas inside quotes. [Reported] [11]
    - How do you handle multi-line quoted fields?
15. Given an IPv4 address and a count, return the minimal list of CIDR blocks covering exactly that range. [Reported] [11]
16. Write tests for your solution right now. Which cases did you choose, and why? [Likely] [3]
17. Collatz conjecture: for every start value in [1, n], find the maximum number of steps to reach 1. [Reported] [11]
    - How would memoization change the runtime, and how do you bound memory for large n?
18. Implement a queue backed by a set of fixed-size arrays (no dynamic array resizing). [Reported] [11]
19. Travel buddy: a friend whose city wishlist overlaps yours by more than 50% (of their list) is a buddy. Return buddies sorted by similarity. [Reported] [11]
    - Extend it to recommend up to N cities from buddies' wishlists.
20. Find case combinations of a string: generate all upper/lower case variants and test each against a hidden-string checker API. [Reported] [11]
    - How do you limit API calls when the checker is expensive?
21. Finding ocean: given a map of L (land) and W (water) and a starting ocean cell, mark all connected ocean cells as O and print the map. [Reported] [11]
    - When does recursive DFS fail on large maps, and what do you replace it with?

22. Guess the number: a secret 4-digit code with digits 1-6 sits behind an API that returns the count of correctly placed digits. Find the code with as few calls as possible. [Reported] [11]

C. Onsite coding round 1 (algorithmic, Airbnb-tagged style)

23. Alien dictionary: derive a character ordering from a sorted list of words. [Reported, unverified] [4]
    - How do you detect an invalid ordering, such as a prefix appearing after a longer word?
24. Palindrome pairs: find all index pairs whose concatenation forms a palindrome. [Reported] [11]
    - Can you do better than O(n^2 * k)?
25. Minimum cost with at most K stops: given flights (from, to, price), find the cheapest itinerary from source to destination. [Reported] [11]
    - What changes if edges carry both price and duration and you must optimize a weighted objective?
26. Sliding game: find a sequence of moves that solves a 3x3 eight-tile puzzle. [Reported] [11]
    - How does your search change for an unsolvable board, and how do you detect it up front?
27. String pyramid transition: given the bottom row and a map of allowed parents for each (left, right) pair, can the pyramid reach a root? [Reported] [11]
28. Meeting time: given everyone's meeting intervals, return the common free intervals. [Reported] [11]
    - How would you adapt this to host calendars with blocked dates and minimum-night rules?
29. Smallest common region: find the lowest common region in a region hierarchy. [Likely] [4]
30. Menu combination sum: find all combinations of menu item prices that sum to a budget K. [Reported] [11]
    - How do you handle float prices without precision bugs?
31. Simple regex matching supporting `.`, `*` and `+`. [Reported] [11]
32. Boggle game: on a letter grid with a dictionary, find the path that contains the largest number of dictionary words. [Reported] [11]
33. Basic calculator with `+ - * /` and parentheses. [Likely] [4]
34. Contains duplicate within distance k and within value difference t. [Likely] [4]
35. Mini parser for nested lists (nested-list iterator). [Likely] [4]
36. Maximum nights you can accommodate: given consecutive-night booking requests, pick bookings that maximize occupied nights, leaving at least one day gap between bookings for cleaning. [Reported] [11]
    - What if bookings are intervals with arbitrary overlaps (weighted interval scheduling)?
37. K edit distance: find all dictionary words within edit distance k of a target word. [Reported] [11]
    - How would a trie cut the work compared with checking each word?
38. Preference list: merge everyone's ordered city preferences into one list consistent with all of them. [Reported] [11]

    - What do you return when the preferences contain a cycle?
39. Ten wizards: each wizard knows a list of others, and a hop from i to j costs (i - j)^2. Find the minimum-cost path from wizard 0 to wizard 9. [Reported] [11]
40. Minimum vertices to traverse a directed graph: return the smallest set of start vertices from which every vertex is reachable. [Reported] [11]
41. Find the median of a file of integers too large for memory, with only sequential access. [Reported] [11]
42. Hilbert curve: given (x, y) and an iteration count, return the step index of that point along the curve. [Reported] [11]
43. Number of intersecting rectangles: given many axis-aligned rectangles, count how many groups of them intersect. [Reported] [11]

D. Onsite coding round 2 (practical, evolving requirements)

44. Design and implement a booking calendar that supports `book(start, end)`, rejects overlaps and answers `is_available(start, end)`. [Likely] [3]
    - Now add per-night pricing and return the total price for a stay.
    - Now support split stays across two listings when no single listing is available.
45. Implement a simple key-value store with nested transactions (`begin`, `commit`, `rollback`). [Likely] [3]
46. Parse a log of experiment exposures (user_id, experiment, variant, timestamp) and output per-variant unique user counts, de-duplicating users exposed to multiple variants. [Likely]
    - How would you flag and count users with conflicting assignments?
47. Implement deterministic hash-based bucketing: given user_id, experiment salt and traffic split, return a variant. [Likely] [6]
    - How do you guarantee independence between experiments run on the same users?
    - How would you ramp traffic from 1% to 50% without reshuffling already-exposed users?
48. Given streaming events, compute a rolling 7-day conversion rate per listing with bounded memory. [Likely]
49. Implement a file system with `create(path, value)` and `get(path)`, returning an error when a parent path does not exist. [Reported] [11]
    - Add a `watch(path, callback)` that fires on changes to the path or its children.
50. Given guest reviews with ratings and timestamps, return the top-K listings by a time-decayed average rating. [Likely]
    - How do you handle listings with very few reviews (shrinkage)?
51. Implement an LRU or LFU cache for search results, then extend it with a TTL. [Likely]
52. Refactor your solution so that a new requirement (for example multi-currency support) fits in with minimal change. How do you structure the code? [Likely] [3]
53. Simulate Diplomacy: given orders per army (move, hold, support), resolve the board, where stronger armies win a contested location and ties eliminate both. [Reported] [11]
    - A new rule arrives mid-interview: a support is cut if the supporting army is attacked. How do you add it without rewriting the resolver?
54. Given per-user rows (variant, converted, pre-period bookings), write runnable code that returns the lift and a 95% confidence interval for each treatment arm. [Likely] [6]

    - Add a CUPED adjustment using the pre-period covariate and show the variance reduction on the test data.
55. Given daily assignment counts per variant, write a function that flags sample ratio mismatch against the intended split. [Likely] [6]

E. ML design round (Airbnb product + experimentation)

56. Design the search ranking system for Airbnb stays. [Likely] [7][8][9]
    - What are your labels (bookings, clicks, long clicks, host acceptances), and how do you weight them?
    - How do you handle position bias in training data?
    - How do you evaluate offline (NDCG on booked listings) versus online?
57. Design real-time personalization using listing embeddings from in-session clicks. [Likely] [8]
    - How would you inject booking or market signals into the embedding objective?
    - How do you handle cold-start listings with no sessions?
58. Design a dynamic pricing suggestion model for hosts (Smart Pricing). [Likely]
    - How do you estimate demand elasticity when price is confounded with seasonality and quality?
    - How do you A/B test a pricing model when hosts can override it?
59. Design a fraud or risk model for bookings (Trust and Safety). [Likely]
    - How do you evaluate a model whose positive labels arrive weeks late and are censored by your own interventions?
60. Design "similar listings" or "recommended experiences" recommendations. [Likely]
61. Design a model that predicts host acceptance probability and use it in ranking. [Likely] [7]
62. Design an ML system to detect low-quality or misleading listing photos and text. [Likely]
63. How would you design the online experiment that launches your new ranking model? [Likely] [6][9]
    - What is the primary metric, and what are your guardrail metrics?
    - What unit of randomization would you use (user, session, listing, market), and why?
    - How do you estimate the required sample size and duration?
64. In a two-sided marketplace, why can a user-randomized test of a ranking change be biased? [Likely] [6]
    - How would listing-side or market-level (cluster) randomization change your estimates?
    - What is interference, or a SUTVA violation, in this setting?
65. When would you use interleaving instead of A/B testing for search ranking? [Likely] [9]
    - How does interleaving sensitivity compare with A/B, and what can interleaving not measure?
66. Offline NDCG improved but the online A/B showed no booking lift. How do you debug this? [Likely] [7]
67. Bookings happen weeks after search. How do you choose an experiment metric that is sensitive enough but still aligned with long-term value? [Likely] [6]
    - How would you build and validate a proxy or surrogate metric?

68. How would you apply variance reduction (CUPED or regression adjustment) to booking-conversion experiments? [Likely] [6]
69. How do you handle novelty effects and seasonality (holidays, events) when reading test results? [Likely]
70. How do you control false discoveries when a team runs dozens of concurrent experiments and checks results daily? [Likely] [6]
    - What is the peeking problem, and how do sequential tests or dynamic p-value thresholds address it?
71. How would you measure the long-term effect of a model change after the A/B test has ended (holdouts)? [Likely]
72. Your model improves guest conversion but reduces host earnings in some markets. How do you decide whether to ship? [Likely]
73. The ranking team wants to ship a model every two weeks, but each booking-metric A/B test needs four weeks. How do you redesign the evaluation pipeline? [Likely] [6] [9]
74. Design an ML model that predicts which experiments are likely to produce a false positive (for example, from metric variance, traffic and peeking history) so the platform can warn owners. [Likely] [6]

F. System design round

75. Design an experimentation assignment and logging service used by every Airbnb client (web, iOS, Android, backend). [Likely] [6]
    - How do you keep assignment consistent across logged-out and logged-in states?
    - How do you guarantee exposure logging is not lost or double-counted?
76. Design the metrics computation pipeline for an experimentation platform (daily scorecards for thousands of metrics). [Likely] [6]
    - How do you backfill when a metric definition changes?
77. Design a booking and reservation system that prevents double-booking under high concurrency. [Likely] [1]
78. Design a feature store that serves both offline training and low-latency online ranking. [Likely]
    - How do you prevent training-serving skew?
79. Design a search service that retrieves and ranks listings within 200 ms p99 across millions of listings. [Likely] [7]
    - Where do you place candidate retrieval, filtering (availability, price, map bounds) and ML re-ranking?
80. Design a model serving and rollout system with shadow mode, canary and automatic rollback. [Likely]
81. Design a notification system for hosts and guests that respects time zones and rate limits. [Likely]
82. Design a calendar availability service for millions of listings with frequent updates and iCal sync. [Likely]
83. Design a feature-flag and kill-switch service that lets an on-call engineer turn off a misbehaving model in every region within one minute. [Likely]

G. Past project deep dive

84. Walk me through the ML project you are proudest of, from problem framing to measured impact. [Reported, unverified] [1][2]
    - What was your specific contribution compared with the team's?
    - What alternatives did you reject, and why?
85. How did you measure the impact of that project? Was there an A/B test, and what did it show? [Likely]
    - What were the confidence intervals, and were there any surprising segment-level results?
86. What was the hardest technical trade-off you made on that project? [Likely]
87. If you had to redo the project with half the time, what would you cut? [Likely]
88. Tell me about a model that worked offline but failed in production. What did you learn? [Likely]
89. How did you get alignment from Product and Data Science on the success metric? [Likely]
90. What would the next iteration of that system look like? [Likely]
91. How did you handle data quality or labeling problems in that project? [Likely]
92. Which part of the system would break first at 10x scale? [Likely]
93. Your project's A/B test showed a lift that later disappeared in the holdout. Walk me through how you found the cause. [Likely]

H. Core values interviews (two rounds, cross-functional)

94. Which of Airbnb's core values resonates with you most, and why? [Reported, unverified] [1][5]
95. Tell me about a time you went above and beyond for a customer, user or teammate ("Be a Host"). [Reported, unverified] [1][5]
96. Tell me about a time you took a big risk or tried something no one had done before ("Embrace the Adventure"). [Reported, unverified] [1][5]
97. Tell me about a time you had to be scrappy and resourceful with limited resources ("Be a Cereal Entrepreneur"). [Reported, unverified] [1][5]
98. What does "belonging" mean to you, and when have you created it for someone else ("Champion the Mission")? [Reported, unverified] [1][5]
99. Describe a memorable travel or hosting experience and what it taught you. [Reported, unverified] [1]
100. Tell me about a time you failed. What did you do next? [Reported, unverified] [1][2]
101. Tell me about a time you disagreed with a teammate or manager. How did you resolve it? [Reported, unverified] [2]
102. Tell me about a time you received difficult feedback. [Reported, unverified] [2]
103. Describe a time you helped someone grow or mentored a junior colleague. [Likely]
104. Tell me about a time you made a decision that was right for users but unpopular internally. [Likely]
105. How do you build inclusive teams or include people with different perspectives? [Likely] [5]
106. Tell me about a time you had to adapt to a big change you did not choose (a reorg, layoffs, a pivot). [Likely]

I. Senior-level calibration probes (appear across rounds)

107. How do you decide when a simpler heuristic should ship instead of an ML model? [Likely]
108. Tell me about a time you set the technical direction for a team without formal authority. [Likely]
109. How do you prioritize between model quality, latency and engineering cost? [Likely]
110. Describe a time you stopped or reversed a launch because the experiment results were ambiguous. [Likely]
111. How do you communicate a negative or inconclusive A/B result to leadership? [Likely]
112. How do you make sure your experiment results are reproducible and trusted by other teams? [Likely]
113. What metrics would you put on a dashboard to know your production model is healthy? [Likely]
114. How do you balance short-term booking metrics against long-term guest and host trust? [Likely]
115. A product manager wants to ship a feature whose test was stopped early at p = 0.04 after daily peeking. What do you tell them, and what do you change in the process? [Likely] [6]
116. What questions do you have for us about the team, the experimentation culture or the roadmap? [Reported, unverified] [1]

References

Only [11] was fetched and checked during the 2026-10-14 review. Sources [1]-[10] could not be reached because the proxy blocked their domains. They are kept because they are well-known and their bibliographic details match prior knowledge, but each one is marked "not re-verified". Open each before relying on it.

1. "Airbnb Interview Process & Questions (+ how to prepare)", IGotAnOffer (not re-verified). https://www.igotanoffer.com/blogs/tech/airbnb-interview-process
2. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor candidate reports (not re-verified). https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm
3. "Airbnb" interview discussions, LeetCode Discuss search (not re-verified). https://leetcode.com/discuss/interview-question?currentPage=1&orderBy=hot&query=airbnb
4. Airbnb company-tagged problems, LeetCode (Premium; not re-verified). https://leetcode.com/company/airbnb/
5. Airbnb Careers, core values (not re-verified). https://careers.airbnb.com/
6. Overgoor, "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014 (not re-verified). https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
7. Haldar et al., "Applying Deep Learning to Airbnb Search", KDD 2019 (not re-verified). https://arxiv.org/abs/1810.09591
8. Grbovic & Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018 (not re-verified). https://dl.acm.org/doi/10.1145/3219819.3219885
9. Haldar et al., "Improving Deep Learning for Airbnb Search", KDD 2020 (not re-verified). https://arxiv.org/abs/2002.05515

10. The Airbnb Tech Blog, Airbnb Engineering (Medium) (not re-verified). https://medium.com/airbnb-engineering
11. allaboutjst, "AirBnB Interview Questions" (community-collected from Glassdoor and GeeksForGeeks; 31 questions; v1.0 2017, last updated 2020-08-26). Verified 2026-10-14. https://github.com/allaboutjst/airbnb (raw: https://raw.githubusercontent.com/allaboutjst/airbnb/master/README.md)

02. Coding Rounds: Airbnb-Tagged LeetCode and Reported Practical Coding Questions

> Research note (read first). First researched 2026-10-13; adversarially reviewed 2026-10-14. On review, web search was unavailable (the shared search budget was exhausted), and direct HTTP to GitHub returned proxy 403s, but pages could still be fetched through a fetch tool. About 50 cited pages were refetched and read on review; a few returned transient 503s and were left as cited. Glassdoor, Blind, LeetCode, Reddit, 1point3acres, Prepfully and Airbnb's own sites were not reachable. **All evidence below comes from GitHub-hosted pages:**
>
> - two open mirrors of LeetCode's Airbnb company tag [5][6];
> - two mirrored TrueInterview candidate-report guides, one for the MLE track and one for the SWE track [1][2];
> - a dated question tracker (Sep 2025 to Sep 2026) and a 2025-2026 company guide [3][4];
> - classic Airbnb question collections [7][8][9][10];
> - a GitHub mirror of about 100 1point3acres (一亩三分地) Airbnb interview posts [13]. Posts that carry a specific question are cited individually [14]-[58] and [59]-[82].
>
> All of this is third-party, user-submitted material, so treat every [Reported] tag as "a candidate or aggregator says so", not as Airbnb confirmation. Several mirror files carry no role, level or date, and some do not name the company; they are attributed to Airbnb only because the mirror files them under an Airbnb folder. Where that is the only link to Airbnb, the question says so. [Likely] marks an inference from the MLE/experimentation role. It does not mean anyone reported it.

Format notes

- **How many coding rounds an MLE gets.** The one public MLE-track summary (3 candidate reports, mainly 2026 Q2) gives MLEs one technical phone screen and **typically one onsite coding round** (45-60 min). It says the track is "filtered by design depth, not by code": the decisive rounds are 1-2 ML system design rounds [1]. One 1point3acres MLE-type loop had one coding round (LC 1298) plus two ML design rounds (LTV and host pricing) [14]. Another had two ML design rounds plus a "ConnectN" OOD coding round [15]. SWE-track loops usually have 1-2 coding rounds plus a separate **code review** round (about 3 PRs) [2][4]. One aggregator says code review replaced the second coding round around 2024, while some 2026 India loops still had two 45-minute coding rounds [4]. In several 2025-2026 loops, a weak code review round led to an added coding round or to rejection [59][72][77]. Ask your recruiter whether the MLE loop includes code review.

- **Runnable code with tests is the bar.** Candidates report that Airbnb requires "bug free" code on the spot with all cases passing [20]. Other reports: code must run and be tested within 45 minutes [2]; there were "about 10 test cases" for Connect 4 [32]; problems arrived with stdin-style test strings [17]; one interviewer gave no interface, so the candidate had to agree the API and then write tests that pass [36]. Airbnb has long stressed "compilable, runnable code" [11]. One MLE screen expected the candidate to explain pruning or memoization *before* coding, then write test cases [1].
- **Shape of a round.** A round is one practical problem in 45 min, often in two parts. Interviewers add constraints once part 1 runs, so leave 10-15 minutes for follow-ups [4][33]. One candidate instead got three short LeetCode problems in 60 minutes [47]. Several posts say the interviewer said nothing and just pasted the problem [18][20]. Communication still counts: a promotional prep guide claims that silent candidates score lower than ones who think aloud [12]. Switching algorithms mid-round and polishing helpers too early are cited as reasons for rejection [2].
- **Flavor.** Problems are framed as product stories: split stays, group lodging, menus and combos, refunds, layovers, ski runs, host pagination, review highlighting. They test DP (often bitmask), graph search, simulation, parsing and small-object design rather than exotic tricks [1][3][4]. LeetCode's Airbnb tag over the last 6 months is led by Design Excel Sum Formula, Maximum Profit in Job Scheduling, Shortest Uncommon Substring, Maximum Candies from Boxes, Shopping Offers, Accounts Merge, Text Justification and Smallest Sufficient Team [5].
- **Senior bar.** Seniors are expected to clarify ambiguous specs themselves, for example tie-breaking rules, whether the graph has cycles, and whether to return the path or only the cost [16]. They must also reach working code fast enough to discuss complexity and extensions. One candidate passed with a suboptimal solution that ran and passed every test, and only *described* the optimization [12][19]. Another failed because they could not beat O(N^2) when asked [19]. Several senior rejections were attributed to design or deep-dive rounds even when coding was strong [16][28].

Questions

A. Airbnb "classic" practical problems (long-running question bank)

1. Display Page / pagination: given CSV search results (host_id, listing_id, score, city) sorted by score and a page size, reorder them so that a host appears at most once per page when possible, while otherwise preserving order. Print the pages as blocks. [Reported] [7][8][52]
    - What should happen when the remaining results all belong to one host?
    - Can you do it in one pass without repeatedly rescanning the list?
    - How would you test that the relative order inside a page is preserved?
2. Text Justification: pack words greedily into lines of exactly width L, spreading extra spaces left-first, with the last line left-justified. [Reported] [5][7][8]
    - How do you handle a line that holds a single word?
    - What changes if words can be longer than L?
3. Fixed-width text formatting (simpler variant: left-align and pad with trailing spaces), then lay the lines out into k side-by-side columns separated by one space. [Reported] [1][40][50]

    - When one column runs out of lines, what do you print in its place?
    - What is the complexity for n up to 1e5 words?
4. Menu combination sum: given menu item prices, return every combination of items whose total is exactly the budget. Items may repeat, and duplicate combinations are not allowed. [Reported] [7][49]
    - Prices are floats like 2.40. How do you avoid floating-point equality bugs? [Reported] [17][78]
    - Now return only the count of combinations. Does order matter? [Reported] [10] [48]
5. Palindrome Pairs: find all pairs (i, j) such that words[i] + words[j] is a palindrome. [Reported] [5][7][39]
    - The interviewer wants O(n*k) rather than O(n*k^2). Which requirement do you clarify first? [Reported] [16]
    - How do the empty string and duplicate words affect correctness?
6. IP range to CIDR: given a starting IPv4 address and a count n, return the shortest list of CIDR blocks that covers exactly that range. [Reported] [5][7][8]
    - How do you test the block boundaries, for example a range starting at an odd address?
7. List-of-list (2D) iterator with `hasNext`, `next` and a `remove` that may be called once per `next` (LeetCode: Flatten 2D Vector; reported Oct 2025 as "Flatten 2D Array With Remove"). [Reported] [3][5][7][8]
    - What should `remove` do when it is called twice, or before `next`?
    - How would you make the iterator lazy over a very large nested input?
8. Flatten Nested List Iterator. [Reported] [5][6]
9. Cheapest flights / minimum cost with at most K stops. Return both the cost **and** the route. [Reported] [5][7][27]
    - If the route must be returned, which algorithms still work?
    - How would you handle ties, or a cycle of zero-cost edges?
10. Sliding puzzle: find the minimum moves to solve the 2x3 board (LC 773), or produce a move sequence for the 3x3 (8-puzzle) variant. [Reported] [5][7][8]
    - How do you detect that a board is unsolvable before searching?
    - Would A* help, and with which heuristic?
11. Terrain rendering plus water simulation: Part 1 prints the terrain as ASCII from a heights array. Part 2 drops water at a column and prints the terrain with W for water. The classic form is Pour Water (LC 755): pour V units at index K and return the final heights. [Reported] [3][5][7][35]
    - Which way should water flow when both sides are equally low?
    - How do you test the printed output exactly, including trailing whitespace?
12. Round prices: round each price up or down so that the rounded values sum to the rounded total, minimizing the total rounding error (LC 1058). [Reported] [5][7][10]
    - What do you return when the target is unreachable?
13. CSV parser: parse a CSV line with quoted fields, escaped quotes and commas inside quotes. [Reported] [7][8]
    - How do you handle a quoted field that spans multiple lines?
14. Design File System: `create(path, value)` fails if the parent is missing, and `get(path)` returns the value. [Reported] [5][7]
    - How would you add `delete` with recursive children?

    - Add `watch(path, callback)`, which fires when a `create` succeeds on that path or any path below it. [Reported] [81]
15. Alien Dictionary: derive the letter order from a sorted word list. [Reported] [5][6][8]
    - How do you detect an invalid input, such as a longer word appearing before its own prefix?
16. Regular expression matching with `.`, `*` and also `+`. [Reported] [5][7]
17. Boggle game: find the path on a letter grid that spells the **largest number** of dictionary words (not just any word). Word Search II is the tagged LeetCode version. [Reported] [5][7]
18. Pyramid transition matrix: can the bottom row be built up to a single root using the allowed triples? [Reported] [5][7][9]
19. Ten wizards: the cost of a link is the squared difference of wizard IDs. Find the minimum-cost path from wizard 0 to wizard 9. [Reported] [7][8][9]
20. Travel buddy list: friends whose wishlists overlap more than half with yours are "buddies". Recommend up to `max` cities from buddies' wishlists, ordered by similarity. [Reported] [7][57]
    - How do you break ties between buddies with equal overlap?
21. Collatz conjecture: find the longest step count for any start value in [1, n]. [Reported] [3][7][65]
    - How would you memoize, and how do you bound memory for large n?
22. Find the median of a file of integers too large for memory, with only sequential access. [Reported] [3][7][8]
23. Guess the number: find a hidden 4-digit code (digits 1-6) using an API that returns how many digits are in the correct position. Minimize the number of API calls, and supply your own test cases. [Reported] [3][7][54][69]
    - Variant: talk to a server over a socket with a START/GUESS protocol. How do you structure the I/O? [Reported] [54]
24. Meeting time / Employee Free Time: return the common free intervals across everyone's schedules. [Reported] [5][7]
25. Maximum number of nights you can accommodate: choose non-adjacent booking requests to maximize nights (House Robber framing). [Reported] [5][7][9]
26. Generate all upper- and lower-case combinations of a string and test each against a hidden string. [Reported] [7]
27. K edit distance: find all dictionary words within edit distance k of a target. [Reported] [7][8]
    - How would a trie change the complexity?
28. Preference list: merge everyone's ranked lists into one ordering that respects every person's order. [Reported] [7][8]
29. Minimum vertices from which every node of a directed graph is reachable. [Reported] [3][5][7]
    - Does the graph contain cycles, and if so, how does that change the answer? [Reported] [16]
30. Finding ocean (flood fill from a water cell), number of intersecting rectangles, and Hilbert curve step count. [Reported] [7][8][10]
31. Implement a queue backed by a chain of fixed-size arrays. [Reported] [7][8]

B. Practical problems reported in 2025-2026 (current question bank)

32. Split stay: given listings with lists of available day numbers and a date range, return every pair of listings [X, Y] where X covers a prefix of the range and Y covers the rest (one split only). [Reported] [3][18][19][20]
    - Your O(N^2 * D) solution works. The interviewer says something better exists. Where is the redundant work?
    - What if a split stay may use up to three listings?
    - How do you deduplicate symmetric pairs, and must the order be preserved?
    - Must the first listing cover a contiguous prefix and the second a contiguous suffix, or is plain set coverage enough? (The two readings give different answers on the reported example.) [Reported] [59]
    - One candidate wrote brute force with `combinations` and earned extra credit for a bitmask encoding of availability. How would you use one? [Reported] [59]
33. Group lodging / select properties: for a target neighborhood and group size, choose properties with total capacity >= group size, minimizing the number of properties and then the total capacity. Return the IDs, or -1 or [] if impossible. [Reported] [4][21][41]
    - Which tie-break do you confirm first: fewer properties or less wasted capacity? [Reported] [21]
    - Is this subset-sum-like DP or backtracking, and what is the worst-case cost?
34. Minimum-cost menu order: given single items and combos with prices plus a list of wanted items (at most 3 unique), return the lowest cost **and** the items to order. [Reported] [1][16][22][23]
    - You wrote DFS. Now convert it to DP. How do you add bitmask state? [Reported] [22]
    - Which menu entries can you prune before searching? [Reported] [16][56]
    - Return every optimal combination, not just one. [Reported] [16][62][63]
    - Variant II: order *at least* the required dish types using combos or singles. Why discard combos containing no required item before the DP? [Reported] [68]
    - Variant: rental "services" such as wifi, parking and TV instead of food. [Reported] [23]
35. Menu bundles closest to budget: choose up to 3 bundles whose total is closest to, but not above, the budget. Write at least 5 test cases. [Reported] [35]
36. Layover: the experience durations can each be booked repeatedly. Can you exactly fill the layover hours? Follow-up: what is the minimum number of bookings? [Reported] [1][3][51]
37. Connect Four simulation: take a list of column moves, stop on a win or an invalid move, ignore the remaining moves, and print the grid. [Reported] [3][17][32][37][76]
    - How do you avoid rescanning the column for gravity on every move? [Reported] [32]
    - Check the diagonals without off-by-one errors in the limited time. [Reported] [16]
    - Determine win, draw or ongoing from a static board alone. [Reported] [3][73] [74]
    - One interviewer asked for logically clear code over a clever algorithm, yet the candidate expected to fail for not running it. How do you budget 45 minutes so the diagonal check and a test run both fit? [Reported] [74]
38. ConnectN object model: configurable width, height and N in a row to win, alternating players, and draw detection. [Reported] [15]

    - Which classes and interfaces would you expose so that a new rule, such as an irregular board, is easy to add?
39. Retryer: wrap a function with maximum attempts, an interval, and exponential backoff (for example capped at 16 s). Rethrow the last error. Make it extensible. [Reported] [2][10][32][75][78]
    - How would you plug in jitter, retry-on-specific-exceptions or a time budget?
    - How do you unit-test timing without real sleeps?
40. Key store with sum dependencies: `set_value(k, v)`, `set_sum(k, [keys])` (keys may repeat) and `get_value(k)`. Updates must cascade to dependents. [Reported] [3][24] [77][82]
    - What happens on a cycle?
    - Choose between eager propagation and lazy recompute for a workload with frequent writes and rare reads. [Reported] [2][16]
41. Design Excel Sum Formula (LC 631). [Reported] [4][5]
42. Banking class: deposit, withdraw, transfer, balance, and balance change over (start, end], where the boundary convention matters. [Reported] [2][3][34][62][63][64]
    - Your tests fail on a boundary. Which spec detail did you assume?
    - Online-assessment version: extend in levels to top spenders, scheduled payments and account merging, where later levels reuse earlier data structures. [Reported] [4]
43. Refund allocation: given payments (method, date, amount) and existing refunds, allocate a new refund in full to one payment before the next. Prioritize by method (CREDIT, then CREDIT_CARD, then PAYPAL), and within a method by most recent payment. [Reported] [2][3][42]
    - How do earlier partial refunds change the remaining refundable amount per payment?
    - How do you design the data structures when the interviewer gives only examples? [Reported] [42]
44. URL query parser: parse `?k=v&...` into a map. Duplicate keys become lists, a bare key means true, and percent-encoded separators (%26, %3D) are decoded *after* splitting. [Reported] [2][3][26]
    - In a typed language, how do you return values that may be a string or a list?
    - Variant: a valueless key maps to an empty string rather than true. Which spec detail do you confirm first? [Reported] [2]
45. Ski tournament: in a DAG of checkpoints with point values and edge travel times, find the path from start to any end node that maximizes points minus time, and print the path. [Reported] [3][17][42]
    - Is the input guaranteed acyclic? What if it is not?
    - Is Dijkstra correct here, and why or why not?
    - Extend to multiple skiers. How should the first version be structured so this follow-up is cheap? [Reported] [2][61]
46. Task scheduling with deadlines: unit-time tasks with a deadline and a reward. Return the order that maximizes total reward (for example, (profit, deadline) = (100,2), (19,1), (27,2), (25,1), (15,3) gives 142). [Reported] [4][28][67][82]
    - Why does the naive greedy fail, and what is your optimal-runtime approach? [Reported] [28]
    - How do you find the latest free slot before a deadline quickly? [Reported] [2]

47. Maximum Profit in Job Scheduling (LC 1235). Follow-up: output the chosen jobs. [Reported] [5][38]
48. Account registration deduplication: accounts share email, phone or government ID. Return all duplicates of a given account. [Reported] [2][3][4]
    - Generalize from two attributes to an arbitrary attribute set. [Reported] [2]
    - What changes if new accounts stream in continuously?
49. Build the smallest number from a list of digits (leading zeros dropped). Part 2: the smallest permutation that is >= a lower bound. [Reported] [2][30][55][64]
50. Intersection of two linked lists where either list may contain a cycle (four structural cases). [Reported] [2][3][5]
51. Multi-stream round-robin iterator: read bytes from several character streams in round-robin order with `has_next` and `next`. [Reported] [2][37]
52. Calculate board score: for every cell, add the length of the same-character run going right and going down. R and C go up to 2000. [Reported] [3][53]
53. Run-length encoding of an int array. Follow-ups: arithmetic-sequence RLE, and an iterator-based decoder for each encoding. [Reported] [25]
54. Sum tree: leaves hold numbers and parents hold the sum of their children. Return any node's value, then support mutations (leaf to subtree, subtree to leaf) while keeping reads correct. [Reported] [29]
55. Payment data store: `insert(ts, value, id)`, `get(id)` and `getValueInRange(start, end)`. [Reported] [33]
56. Keyword tagging in reviews: wrap keywords as `[tag]{keyword}` case-insensitively. Overlapping matches go to the longer keyword. [Reported] [3][10]
57. Replace text in a review using a replacement dictionary. [Reported] [3][10]
58. Shortest uncommon substring: for each string, find the shortest substring that appears in no other string (LC 3076). [Reported] [4][5][31]
59. Maximum Candies You Can Get from Boxes (LC 1298). It was given with no interface, so you define the API and write passing tests. [Reported] [5][14][36]
    - One solution passed every test but was rejected because its class model did not match the interviewer's intent. Which entities and methods would you model? [Reported] [2]
60. Office layout grid (-1 wall, 0 open): find the maximum reachable open area and count the unreachable rooms. [Reported] [36]
61. Two water jugs: return the minimum operations to measure a target volume **and** the sequence of states. [Reported] [10][27][64]
62. Given three numbers, list every way the first two combine with + - * / to equal the third. Follow-up: allow parentheses. [Reported] [45][64]
63. Card pile: by moving the top card to the bottom, can you reach N identical consecutive cards? [Reported] [44]
64. Parse strings of letter-score tokens and compute per-letter totals. [Reported] [43]
65. Find Median from Data Stream (LC 295). [Reported] [5][46]
66. Chain booking: find the maximum number of cascading reservations. [Reported] [2][3]
67. Compute browsing metrics in Python from event logs. [Reported] [3]
    - Which metrics would you compute per session versus per user, and how do you define a session?
68. Durable key-value store serialization, an in-memory database with SQL operations, and an in-memory object manager with TTL. [Reported] [2][3]

69. Car-speed controller: through a socket API (`getStatus`, `setStatus(acceleration)`), hold a simulated car at 29 mph. [Reported] [58] (The mirror files it under Airbnb; the page itself names no company.)

C. LeetCode Airbnb-tag problems (company tag; frequency per the mirrors)

70. Smallest Sufficient Team (bitmask DP). It ranks near the top of the Airbnb tag frequency lists. [Reported] [4][5]
71. Shopping Offers (DP and backtracking with bundles, the LeetCode cousin of the menu problem). [Reported] [5][16]
72. Accounts Merge. [Reported] [4][5]
73. Smallest Common Region. [Reported] [5][6]
74. Detonate the Maximum Bombs. [Reported] [4][5]
75. Simple Bank System (LC 2043). [Reported] [5][6]
76. Trapping Rain Water. [Reported] [5][6]
77. Shortest Path to Get All Keys (LC 864). It was asked in the same round as Palindrome Pairs. [Reported] [5][39]
78. Minimum Number of Flips to Convert Binary Matrix to Zero Matrix. [Reported] [5][6]
79. Mini Parser and Basic Calculator II. [Reported] [5][6]
80. Strings Differ by One Character. [Reported] [5][6]
81. Number of Ways to Build House of Cards. [Reported] [5][6]
82. Minimum Window Substring and Minimum Window Subsequence. [Reported] [5][6]
83. Making a Large Island. [Reported] [5][6]
84. All O`one Data Structure. [Reported] [5][6]
85. Minimum Cost to Reach Destination in Time. [Reported] [5][6]
86. Time Based Key-Value Store. [Reported] [6][12]
87. Coin Change and Combination Sum. [Reported] [5][17]

C2. Additional practical problems reported 2025-2026 (added on review)

88. Task build order: given tasks and prerequisite pairs (Course Schedule / Airflow-style DAG), decide whether every task can finish, then return a valid order. [Reported] [3] [4][59][60]
    - One candidate mixed up in-degree and out-degree. Which test would have caught it? [Reported] [59]
    - How do you report which tasks are stuck in a cycle rather than just returning false?
89. Two-hotel coverage with interval input: hotels are given as (start, end) availability ranges. Can at most two hotels cover a whole stay? The reported example marks (Apr 1-10), (Apr 5-15), (Apr 20-30) as covering Apr 5-25 (True), even though Apr 16-19 is uncovered. What would you ask the interviewer before coding? [Reported] [79]
    - Are dates nights or days? How does that change the boundary test?
90. Maximum ski score on a height grid (n, m up to 100): each move goes to a strictly lower neighbor. Maximize the total height drop along a path. [Reported] [68]
    - Memoized DFS or processing cells in height order: which is easier to test in 45 minutes?

91. Accommodation allocation: requests and segments each have a start and an end, and segments cannot overlap. Return the maximum number of fulfilled requests and the allocation itself. [Reported] [66]
    - The reporter passed with brute force. When does the input size force something better?
92. Wallet class: create a wallet, deposit, withdraw and check the balance, with one wallet per user. A withdrawal that would overdraw must fail and leave the balance unchanged. No `main` or test harness is provided, so you write it. [Reported] [70]
    - Which invariants would you assert after every operation?
93. Single-source distances in a weighted undirected graph (phone-screen problem-solving part). [Reported] [64]
    - The reporter proposed topological sort. Why does that not work here, and what does?
94. Product of Array Except Self, without division and in O(n). [Reported] [72]
    - How do zeros in the input change your edge-case tests?
95. Maximum Product Subarray. [Likely] [71] (The mirror files it under Airbnb, but the page has no interview context.)
96. Search in a rotated sorted array, as a "classic with a twist" onsite problem. [Reported] [2]
    - How do duplicates break the O(log n) guarantee?
97. Remove digits to form the smallest possible number (Remove K Digits). [Reported] [2]
98. Print sentences inside a bordered, fixed-width table, wrapping words across lines. [Reported] [2]
    - What happens when one word is longer than the width?
99. Gift-card spend: find the fewest items whose prices exactly equal the card value, and return the items. [Reported] [2][3]
    - What changes if each item can be bought at most once?
100. Board-game region scoring: score every connected region of same-type cells on a grid. [Reported] [2]
101. Maze score: given a board-game maze state, compute its score via shortest path. The reporter warns to expect an OOD angle, not just an algorithm. [Reported] [4][61]
102. Infection spread time on a grid: return the time until every reachable cell is infected, or -1. [Reported] [2][3] - Follow-up: infection also spreads diagonally. How much of your BFS changes? [Reported] [2]
103. Diplomacy battle resolution: parse `<Army> <Location> <Hold | Support X | Move L>` actions. Each army has strength 1 and each valid supporter adds 1. In a contested location the strongest army wins and ties kill all tied armies. A supporter whose own location is attacked loses its support. Return each army's final location or `[dead]`. [Reported] [7][80] - Can an army support a move into the location it is supporting from? Which ambiguities do you raise before coding?
104. MLE phone screen: implement a key-value store, then state the test cases you would write against it. Onsite, extend it with additional constraints and show how the tests change. [Reported] [1]
105. Document redaction exercise, reported in recent senior MLE loops: given text and a set of sensitive terms or patterns, return the redacted text. [Reported] [1] - How do you handle overlapping matches, case and word boundaries? Which tests prove the redaction is complete?

106. Code review round: review about three pull requests in about 60 minutes and leave written comments. Reported planted defects include wrong results without a crash, swallowed exceptions, unchecked nulls, a race on shared state, missing error handling around an external API, and a hidden quadratic loop. [Reported] [2][4][59] [60] - The third PR is long and time is short. How do you triage which comments to write first?
107. Multi-level online assessment: cloud storage or file store with add, copy, prefix and suffix search, per-user quotas, and compression and decompression. The next level unlocks only when the current tests pass. [Reported] [3][4] - How do you choose level-1 data structures that will not break at level 4?
108. Multi-level online assessment: task management system with add, update, get, search, priority sort, per-user quotas and an overdue report. [Reported] [4]

C3. SQL and data-analysis coding reported for Airbnb (relevant to experimentation roles)

109. Find users with impressions on seven consecutive days. [Reported] [3] - How do you handle several impressions on the same day and time-zone boundaries?
110. Find hosts who meet listing-count, rating and review-coverage rules. [Reported] [3]
111. Estimate page-visit duration as the gap to the user's next event. [Reported] [3] - What do you do with the last event of a session, which has no next event?
112. Calculate the median house price in each city in SQL. [Reported] [3]
113. Compute the average vacant days per listing. [Reported] [3]
114. Match rentals to required amenities (listings that have all of a given amenity set). [Reported] [3]
115. Compute conversion (C/T) metrics from booking and visit tables. [Reported] [3] - Is the denominator visits, unique visitors or searches? How does the choice interact with the randomization unit?
116. Build a user-by-period panel in SQL, then run a causal regression on it. [Reported] [3] - Which fixed effects would you include, and how do you cluster the standard errors?
117. Model cumulative booking metrics with first and last booking times per user. [Reported] [3]

D. ML/experimentation-flavored practical coding (role-specific, not yet reported)

None of these were reported for Airbnb. They follow the same practical, "make it run with tests" format and fit an MLE role on experimentation, search ranking or pricing.

118. Deterministic experiment bucketing: given user_id, an experiment salt and variant weights, return the assigned variant. [Likely] - How do you show that assignment is uniform and independent across two experiments? - How would you ramp from 5% to 50% without reassigning users already exposed?
119. From raw exposure and conversion logs (user, experiment, variant, ts, event), compute per-variant conversion, the absolute and relative lift, and a 95% confidence interval. [Likely] - How do you handle a user exposed to both variants, and events that happen before exposure?

120. Implement a sample-ratio-mismatch check for an expected 50/50 split and decide whether to alert. [Likely]
121. Implement CUPED or regression adjustment on a per-user metric with a pre-period covariate, and report the variance reduction. [Likely]
122. Implement a bootstrap confidence interval for the difference in a ratio metric (for example bookings per search). [Likely] - Why is resampling individual searches wrong when the unit of randomization is the user?
123. Implement NDCG@k for ranked search results with graded relevance (booking > click > impression). [Likely] - How do you handle queries with no relevant results?
124. Implement team-draft interleaving of two rankers' result lists and credit clicks to each ranker. [Likely]
125. Re-rank a scored list of listings to enforce host diversity: at most one listing per host in each window of k, as in the pagination problem. [Likely] [52]
126. Compute a rolling 7-day booking rate per listing from an event stream with bounded memory. [Likely]
127. Given availability calendars and nightly prices, compute the total price of a stay, including weekly-discount and cleaning-fee rules, and test the boundary nights. [Likely]
128. Reservoir-sample k events from a stream of unknown length, then make it weighted. [Likely]
129. Implement streaming AUC, or approximate quantiles, over a large prediction log without loading it all in memory. [Likely]
130. Write a sequential-testing guard: given daily cumulative results, decide whether a pre-registered stopping boundary has been crossed. [Likely]

E. Cross-cutting probes interviewers use in every coding round

131. Before you code: what are the input format, the output format and the tie-breaking rules? Is any of them ambiguous? [Reported] [16][21]
132. Write your own test cases now. Which edge cases did you choose, and why? [Reported] [1][2][35]
133. What are the time and space complexity, and where is the bottleneck for the largest stated input? [Reported] [1][56]
134. Your code fails one hidden test. How do you debug it in the remaining five minutes? [Likely] [17][34]
135. A new requirement arrives, such as returning the path instead of the cost, or k columns instead of one. How much of your code has to change? [Reported] [3][4][27]
136. If this ran in production behind an API endpoint, what would you validate, log or cache? [Likely] [18]

References

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, mirrored on GitHub; based on 3 candidate reports, mainly 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md

2. "Airbnb Software Engineer Interview Process" (TrueInterview guide, CC BY 4.0, mirrored; based on 41 candidate reports, 2025 Q4 to 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
3. "Airbnb" question list with report dates (algorithm, OOP and SQL questions, Sep 2025 to Sep 2026). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
4. "Airbnb" company guide (2025-2026 reports; merges LeetCode tag data, LeetCode Discuss, PracHub and Hello Interview). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. LeetCode company-tag mirror: Airbnb (All, Six Months and Three Months CSVs; updated June 2025). liquidslr/leetcode-company-wise-problems, GitHub, 2025. https://github.com/liquidslr/leetcode-company-wise-problems/tree/main/Airbnb
6. LeetCode company-tag mirror: airbnb/all.csv. snehasishroy/leetcode-companywise-interview-questions, GitHub. https://github.com/snehasishroy/leetcode-companywise-interview-questions/blob/master/airbnb/all.csv
7. "AirBnB Interview Questions" (31 classic problems with Java solutions). allaboutjst/airbnb, GitHub. https://github.com/allaboutjst/airbnb
8. "Airbnb_Interview" (Airbnb Beijing prep, 36 problems). hbsun2113, GitHub, 2019-2020. https://github.com/hbsun2113/Airbnb_Interview
9. "airbnb-interview-problems". shua2018ti, GitHub, 2018. https://github.com/shua2018ti/airbnb-interview-problems
10. "airbnb_interview_questions.md". warrenzhu25/system-design, GitHub. https://github.com/warrenzhu25/system-design/blob/master/airbnb_interview_questions.md
11. "Interview formats of top companies" (Airbnb section). Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
12. "Airbnb coding interview questions: the problems that actually show up...". yhzl350/airbnb-dp-interview, GitHub, 2026. Promotional page with unverified claims. https://github.com/yhzl350/airbnb-dp-interview
13. Airbnb 1point3acres interview-post mirror (folder index). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a
14. 1p3a post #40: project deep dive, coding LC 1298, ML design (LTV, host pricing). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
15. 1p3a post #79: two ML design rounds (listing recommendation, price prediction), coding ConnectN OOD, ML experience round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
16. 1p3a post #37: compilation of 16 algorithm questions with clarification tips (menu, Connect 4, select property; LC 336/864/284/1298/1557/1235). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/37.md.md
17. 1p3a post #72: phone screen Connect 4 with print, Combination Sum, onsite ski DAG problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/72.md
18. 1p3a post #34: split stay problem statement. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/34.md
19. 1p3a post #44: split stay; O(N^2) judged not good enough. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/44.md

20. 1p3a post #15: hotel split stay; "bug free, all cases must run". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/15.md
21. 1p3a post #38: group-size property selection DP. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/38.md
22. 1p3a post #10: menu min cost with bitmask; DFS then DP follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
23. 1p3a post #53: rental services min-cost variant. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/53.md
24. 1p3a post #65: set_value / set_sum / get_value key store. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/65.md
25. 1p3a post #87: RLE with arithmetic-sequence and iterator follow-ups. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/87.md
26. 1p3a post #100: URL query parser with percent decoding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/100.md
27. 1p3a post #101: cheapest itinerary with max layovers returning the route; two-jug problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/101.md
28. 1p3a post #104: task deadlines and rewards; greedy rejected. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/104.md
29. 1p3a post #55: G10 full-stack; sum-tree with mutations; retry higher-order function. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/55.md
30. 1p3a post #33: smallest number from digits; lower-bound follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/33.md
31. 1p3a post #27: shortest uniquely-mapping substring. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/27.md
32. 1p3a post #69: Connect 4 phone screen (about 10 tests, column-height optimization); onsite retryer with increasing interval. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/69.md
33. 1p3a post #64: payment data insert/get/getValueInRange. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/64.md
34. 1p3a post #98: banking system with timestamped balance changes. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/98.md
35. 1p3a post #12: menu bundles up to 3 with 5+ tests; terrain print and water drop. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/12.md
36. 1p3a post #24: LC 1298 with no interface given, tests required; office layout grid. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/24.md
37. 1p3a post #18: multi-stream iterator. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/18.md
38. 1p3a post #97: LC 1235 variant; running code on the spot. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/97.md
39. 1p3a post #50: coding round with LC 336 and LC 864. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/50.md
40. 1p3a post #85: simplified text formatting with column follow-up; task-score scheduling. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/85.md
41. 1p3a post #75: room allocation (set cover). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/75.md

42. 1p3a post #4: refund rules with examples; ski problem input format. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/4.md
43. 1p3a post #82: per-letter score aggregation. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/82.md
44. 1p3a post #90: card pile N-consecutive problem. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/90.md
45. 1p3a post #17: three-number arithmetic combinations with parentheses follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/17.md
46. 1p3a post #81: LC 295 in a 45-minute coding round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/81.md
47. 1p3a post #36: three LeetCode problems in 60 minutes (incl. Happy Number). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/36.md.md
48. 1p3a post #95: count of meal orders summing to budget. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/95.md
49. 1p3a post #68: menu exact-target combinations. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/68.md
50. "Fixed-Width Text Formatting (Multi-Column Follow-up)" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Fixed-Width%20Text%20Formatting%20(Multi-Column%20Follow-up)/qs.md
51. "Fill Layover Hours by Booking Experiences" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Fill%20Layover%20Hours%20by%20Booking%20Experiences/qs.md
52. "pagination" problem statement (host_id, listing_id, score, city). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/pagination/qs.md
53. "Calculate Board Score" problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Calculate%20Board%20Score/qs.md
54. "guess word" (guess-number variants). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/guess%20word/qs.md
55. "perm_qs 1" (smallest permutation >= lower bound). FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/perm_qs%201/qs.md
56. 1p3a post #56: menu DP, with the follow-up to optimize for the at-most-3-items constraint. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/56.md
57. "buddy list" follow-up statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/buddy%20list/qs.md
58. "car speed" (socket-based car speed controller) problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/car%20speed/qs.md
59. 1p3a post #13: Beijing loop, two coding rounds (topological sort; split stay with bitmask bonus), three-PR code review, extra design round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
60. 1p3a post #21: Course Schedule-style topological sort; three-PR code review in Python. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/21.md

61. 1p3a post #3: board-game / maze-score phone screen; ski problem with multiple-skier follow-up; code review (updated 2026-03). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md
62. 1p3a post #6: bank-account phone screen; menu min cost with DP plus bitmask and listing the combination. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/6.md
63. 1p3a post #16: SWE loop Oct-Dec 2025; bank account with time queries; menu combo with combination-listing follow-up. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/16.md
64. 1p3a post #7: three-number arithmetic; weighted-graph distances; bank account; water pouring; smallest number. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/7.md
65. 1p3a post #25: Collatz longest steps. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/25.md
66. 1p3a post #28: 2025 loop; accommodation segment allocation; code review. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/28.md
67. 1p3a post #29: job sequencing with deadlines (phone screen, worked example). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md
68. 1p3a post #31: Minimum Menu Order Cost II; max ski score on a height grid (updated 2025-09). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/31.md
69. 1p3a post #39: guess the four-digit number with test cases. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/39.md
70. 1p3a post #42: wallet class with a self-written test harness. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/42.md
71. 1p3a mirror file #49: Maximum Product Subarray (no interview context). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/49.md
72. 1p3a post #58: Product of Array Except Self; weak code review led to an extra coding round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/58.md
73. 1p3a mirror file #60: Connect 4 static-board result (win, draw, ongoing). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/60.md
74. 1p3a post #103: Connect 4 static-board check; candidate did not run code. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/103.md
75. 1p3a post #63: `retry(func, max_retries)` signature. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/63.md
76. 1p3a post #67: column-drop game with a memoization follow-up; three-PR code review. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/67.md
77. 1p3a post #71: setSum/getValue/setValue coding; code review feedback. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/71.md
78. 1p3a post #73: senior booking-team loop; float Combination Sum phone screen; Retryer onsite. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/73.md
79. 1p3a post #76: two-hotel interval coverage; ski scoring. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/76.md
80. "Resolve Battles" (Diplomacy) problem statement. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/Resolve%20Battles/qs.

md

81. "file_creation" problem statement with the `watch` follow-up. FlowSight/coding. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/file_creation/qs.md
82. 1p3a post #11: senior IC loop; task scheduling and get/set/getSum coding rounds. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md

03. ML System Design: Search Ranking, Embeddings, Pricing, Trust, Recommendations and More

> Research note (read first). This chapter was reviewed again on 2026-10-14. The shared web-search budget was used up, and the network proxy could not resolve arxiv.org, dl.acm.org, medium.com, airbnb.tech, news.airbnb.com, kdd.org, glassdoor.com, teamblind.com, igotanoffer.com, interviewquery.com or trueinterview.io. Reddit refused the fetch. **GitHub was the only reachable source.** Four candidate-report pages were fetched and read there:
>
> - a TrueInterview-derived summary of the Airbnb MLE loop, built from 3 candidate reports (mostly 2026 Q1-Q2) [22];
> - a TrueInterview-derived Airbnb question index with report dates [23];
> - two translated 1point3acres (一亩三分地) Airbnb onsite posts that list the ML design rounds [24][25].
>
> A general ML-system-design question list was also read [36].
>
> Tagging policy:
>
> - **[Reported]** means one of those fetched pages names the prompt, or a close variant of it, for Airbnb. The TrueInterview question pages [26]-[35] are cited only as the links those pages give. They could not be opened directly.
> - **[Reported, role unspecified]** means the prompt is in the Airbnb index [23], but the page does not say whether it came from an MLE or a data-science loop.
> - **[Likely]** means the question is inferred from the role, the senior level and Airbnb's published systems. A paper or blog citation next to [Likely] points to the system the question probes. It does **not** mean anyone was asked the question.
>
> The Airbnb papers and blog posts [1]-[12] and [16]-[21], [37] and [38] could not be re-fetched. They are listed from prior knowledge. Before the onsite, cross-check this list against Glassdoor, IGotAnOffer and LeetCode Discuss [13][14][15]. Those sites were unreachable for this review.

Format notes

- **Slot and length.** The MLE-track summary says the loop is "filtered by design depth, not by code". It describes **1-2 ML system design rounds** as the decisive rounds; the level guidance applies from Senior (G9) up [22].

    - One 1point3acres loop had two ML design rounds: guest/customer LTV and host pricing [24].
    - Another loop also had two: listing recommendation and price prediction. In the same loop, the "ML experience" round unexpectedly turned into a further ML design question [25].
    - Rounds run about 45-60 minutes over video, with a shared doc or whiteboard [22].

- **Interviewer.** Usually a senior or staff MLE from an ML-heavy area such as Search/Relevance, Pricing, Trust, Experiences/Recommendations, Customer Support/GenAI or a platform team. Expect the interviewer to steer toward their own domain [Likely]. Recent senior loops reportedly added retrieval and ranking prompts around **generative-AI customer-support tools** [22].

- **Shape of the round.** Typically one open-ended Airbnb product prompt, such as "design search ranking", "estimate a listing's lifetime value" or "detect relisted properties". You are expected to drive it:

    - Clarify the business goal and **agree a definition** of the target. In one LTV round the interviewer first made the candidate define "lifetime value" [22].
    - Pick the labels, design data and features, and choose a model.
    - Plan offline evaluation and online experimentation.
    - Cover serving and latency, then monitoring and iteration [22][1][2].

  Interviewers reportedly steer mid-round, for example from regression to bucketized classification in the LTV round, or into geohashing in the relisting round [22].

- **Senior bar.** The round looks for these signals:

    - You frame the problem as a marketplace with two sides: guest and host, supply and demand.
    - You choose labels and objectives with care, including delayed or censored outcomes.
    - You propose a pragmatic baseline before reaching for deep models.
    - You reason about position bias, feedback loops and interference.
    - You design the **launch experiment** yourself.
    - You discuss trade-offs without heavy prompting.

  Airbnb's own search papers stress simple baselines, failed ideas and lessons learned [Likely; 1][2].

- **Experimentation weight.** For this role, assume every design ends with "how would you launch and measure it?" Expect probes on interleaving, guardrails, two-sided interference and long-horizon metrics [Likely; 8][9][17]. Experiment-design prompts also appear in the Airbnb question index [23].

Questions

A. Search ranking for stays (core prompt)

1. Design Airbnb search ranking, from candidate generation through offline and online evaluation and deployment: a guest enters a location, dates and guest count, and you return an ordered list of listings. [Reported] [22][26]
    - What is the business objective: bookings, booking value, guest satisfaction or long-term host and guest retention? How do you encode it?
    - What are the stages (candidate retrieval, filtering, first-pass ranker, second-pass re-ranker), and what is each stage's latency budget?
2. What label would you train on, given that bookings are rare and many searches end with no booking? [Likely] [1]
    - How would you use clicks, listing-page views, wishlist saves, contacts to host and bookings as weaker labels or auxiliary tasks?
    - How do you treat a booking request the host later rejected or cancelled?
3. How would you frame ranking: pointwise, pairwise or listwise? Why? [Likely] [1][2]
    - How would you build pairs from one search session with a single booked listing?
4. How do you correct for position bias in logged search data? [Likely] [2]
    - How would you model position as a feature during training, and what do you set it to at inference time?
    - How would you validate that your debiasing helped, rather than just shifting the distribution?
5. Which feature families would you build for listings, queries, guests and query-listing pairs? [Likely] [1]
    - How do you normalize price so that the model generalizes across markets and currencies?
    - How do you represent location relevance (distance to map center, the neighborhood the guest wanted)?
6. Your first model was a GBDT. When would you move to a neural network, and what would you expect to break? [Likely] [1]
    - How do you handle feature scaling, sparse categorical features and very wide value ranges in a neural ranker?
7. The ranker strongly prefers cheaper listings. Is that a bug or correct behavior? How would you investigate? [Likely] [1][2]
    - How would you make the model price-aware without hard-coding a monotonic price penalty?
8. How do you handle cold-start listings that have no booking or review history? [Likely] [2]
    - How do you estimate engagement for a new listing, and how do you give it controlled exposure without hurting conversion?
9. How would you add diversity to the results so the top of the page is not ten near-identical listings? [Likely] [5]
    - How would you measure diversity, and how would you trade it off against relevance in the loss?
10. How would you rank listings shown as map pins, where there is no clear top-to-bottom position? [Likely] [2][38]
    - How does user attention on a map differ from attention on a list, and how does that change your discounting?
11. How would you model the whole guest search journey (many searches across days before one booking) rather than a single search? [Likely] [6]

    - What would a multi-task model with heads for clicks, long views, wishlist and booking look like? How do you weight the heads?
12. How do you handle flexible-date or flexible-location searches ("I'm flexible", category browsing)? [Likely] [11]
13. How would you incorporate host-side outcomes (acceptance, cancellations, host earnings) into guest-facing ranking? [Likely] [1]
14. Offline NDCG went up but the online A/B test showed flat bookings. Walk me through your debugging plan. [Likely] [1][2]
    - How would you check for training-serving skew, feature leakage and a mismatch between offline and online candidate sets?
15. What offline evaluation set would you build, and how do you avoid leakage from future data or from repeat sessions by the same guest? [Likely] [1]
16. Rank home-page search results when the guest has typed no text query, for example on app open or a logged-in home feed. [Reported] [23][29]
    - What signals replace the query: past searches, wishlists, location, season, booking history?
    - How do you handle a brand-new guest with no history?
    - How does this ranking differ from query-driven ranking in objective and evaluation?
17. A guest types a natural-language query ("cabin near a lake with a hot tub for six next weekend"). How would you parse it into hard filters, soft preferences and a semantic retrieval query? [Likely] [12]
    - How do you evaluate parsing errors that silently drop a hard constraint?
    - Where would you use an LLM, and how would you bound its latency and cost on the search path?

B. Candidate retrieval and serving infrastructure for search

18. Design candidate retrieval that cuts millions of listings to a few thousand within strict latency, respecting date availability, guest count and map bounds. [Likely] [12]
    - Where would embedding-based retrieval fit next to hard filters, and how do you combine them?
19. How would you train a two-tower model for embedding-based retrieval of listings? [Likely] [12]
    - What is your negative-sampling strategy (in-batch, same-market, hard negatives)?
    - How do you handle availability, which changes daily and cannot easily live inside the embedding?
20. Design semantic search driven by user embeddings: represent each guest by an embedding and retrieve listings or destinations that match their intent. [Reported] [23][31]
    - How do you build and refresh the user embedding from sparse, long-gap travel histories?
    - How do you serve approximate nearest-neighbor search under geography and availability filters?
    - How do you evaluate retrieval recall separately from end-to-end ranking quality?
21. How would you serve a neural ranker at p99 latency under 200 ms for thousands of candidates per query? [Likely]

    - Which features are precomputed offline, which are computed in real time, and how do you keep them consistent?
22. How do you prevent training-serving skew in features such as "listing price for these dates" or "number of recent bookings"? [Likely] [1]
23. How often would you retrain the ranking model, and how would you detect that it has gone stale? [Likely]
24. What would you log at serving time so you can retrain, debias and debug later? [Likely] [2]
25. Listings with stale calendars cause booking requests that hosts then decline. How would you predict that a listing's availability is unreliable, and use that in retrieval or ranking? [Likely] [1]

C. Listing embeddings and real-time personalization

26. Design listing embeddings trained from guest click sessions, and use them for similar-listing recommendations and in-session personalization. [Likely] [3][10]
    - How do you define a session and its context window?
    - What negatives would you use? Why might random negatives make embeddings that only learn geography?
27. How would you change the embedding objective so that the booked listing influences every listing in the session? [Likely] [3]
28. How would you build user-type and listing-type embeddings to capture long-term preferences from sparse booking histories? [Likely] [3]
29. How would you create an embedding for a brand-new listing with no sessions? [Likely] [3]
30. How would you evaluate embeddings offline before using them in production? [Likely] [3]
    - What would a "rank of the eventually booked listing among the clicked listings" evaluation look like?
31. How would you turn in-session clicks and skips into real-time ranking features? [Likely] [3]
    - What streaming infrastructure is needed, and what is the freshness requirement?
32. When would you replace skip-gram embeddings with a sequence model or transformer over user actions? What would you expect to gain? [Likely]

D. Pricing, price prediction and lifetime value

33. Design a system that helps hosts set prices for each future date, taking market demand into account. [Reported] [24][25]
    - What is the objective: booking probability, expected host revenue, or marketplace occupancy?
    - Why can't you train directly on "the optimal price" as a label?
34. Predict the nightly price a listing should command (price prediction), including for a brand-new listing with no booking history. [Reported] [25]
    - Which comparable-listing, location, amenity and seasonality features would you use?
    - How do you avoid learning hosts' mispricing as the target?

35. How would you predict booking probability as a function of price, lead time and listing quality? [Likely] [4]
    - How do you handle price being confounded with demand, because hosts raise prices when they expect demand?
36. How would you estimate price elasticity per listing or per market from observational data? [Likely] [4]
    - When would you need randomized price experiments, and what ethical and host-trust constraints apply?
37. Hosts can accept, modify or ignore your suggestion. How do you evaluate the model offline and online? [Likely] [4]
    - What is your unit of randomization, and how do you handle spillover between treated and control hosts in the same market?
38. How would you forecast market demand for a city on a given date (events, holidays, seasonality)? [Likely] [16]
39. How would you prevent the pricing model from producing extreme or volatile suggestions that destroy host trust? [Likely]
40. How would you design total-price display or fee-related ranking changes, and how would you measure whether guests trust the price more? [Likely]
41. A pricing-model launch raised bookings but cut average host revenue. How do you decide? [Likely] [17]
42. Estimate the lifetime value of a property (listing). First agree with the interviewer on what lifetime value means here. [Reported] [22][27]
    - What horizon, which revenue (gross booking value or Airbnb take), and how do you treat churned or delisted hosts?
    - The interviewer pushes you from regression to bucketized classification. Why might value buckets work better for heavy-tailed, censored LTV, and what do you lose? [Reported] [22]
    - How would the LTV estimate be used, for example in host acquisition, support prioritization or ranking?
43. Design a system that computes a guest's (customer's) lifetime value on the platform. [Reported] [24]
    - How do you handle guests who book once every two years, so that the data is censored and very sparse?
    - How would you validate an LTV model whose labels take years to mature?
44. Which hosts should be prompted to turn on Smart Pricing or adopt a suggested price? Design an uplift (heterogeneous treatment-effect) model and the experiment that validates it. [Likely] [4][17]

E. Fraud, risk and trust and safety

45. Design a model that scores every booking for payment-fraud risk in real time. [Likely] [18]
    - Chargeback labels arrive weeks later. How do you train and evaluate with delayed labels?
    - How do you handle label censoring, where blocked transactions never reveal whether they were fraud?
46. Rather than block or allow, how would you design graduated friction (ID verification, extra payment checks) driven by risk scores? [Likely] [18]

    - How do you choose thresholds per action, given the cost of false positives (lost good guests) and false negatives?
47. Design an account-takeover detection system for guest and host logins. [Likely] [18]
48. Design a system that predicts and prevents unauthorized parties at listings. [Likely] [19]
    - What features are legitimate and fair to use, and which risk proxy discrimination (for example by age)?
    - How do you measure the system's impact when the outcome (a party happened) is rarely observed?
49. Design a fake or scam listing detector at listing-creation time. [Likely] [19]
    - How do you combine text, image, host-history and graph signals (shared devices, payment instruments)?
50. Design a relisting-detection system: find a property that was removed or banned and then relisted, possibly under a new account. [Reported] [22][28]
    - How does geohashing help you generate candidate pairs efficiently? [Reported] [22]
    - How would the outputs show up in online metrics, and what would you watch for false-positive harm to legitimate hosts? [Reported] [22]
    - How would you separate relisting from legitimate cases such as a new host buying the property, or a co-host managing it?
51. Design a system that detects duplicate listings (the same property listed several times at the same time). [Likely]
52. Design a model that detects attempts to move payment or communication off the platform in guest-host messages. [Likely] [19][20]
53. How would you use graph-based features or graph neural networks to detect fraud rings? [Likely]
54. Fraudsters adapt to your model within weeks. How do you design for adversarial drift? [Likely]
    - How do you keep a clean holdout to measure true fraud rates when the model blocks most of it?
55. How do you A/B test a fraud model when exposing the control group to fraud has real costs? [Likely] [18]
56. How do you build fairness checks into a trust model that affects who can book? [Likely] [19]

F. Recommendations (Experiences, similar listings, discovery surfaces)

57. Design Airbnb listing recommendation for a guest. [Reported] [25][36]
    - What are the candidate sources (embedding neighbors, popular-in-destination, wishlist-similar), and how do you blend them?
    - How does it differ from search ranking when the guest has no dates or destination yet?
58. Design search ranking for Airbnb Experiences, which has far less data than stays. [Likely] [7]
    - How would you stage the system as data grows (heuristics, then GBDT, then personalization, then online scoring)?
59. Design "similar listings" recommendations on the listing detail page. [Likely] [3]

    - What is the right objective: clicks, bookings, or rescuing guests whose first choice is unavailable?
60. Design personalized marketing or email recommendations (destinations, wishlisted listings with price drops). [Likely]
    - How do you measure incremental lift rather than attribution from people who would have booked anyway?
61. Design recommendations for cross-selling Experiences or Services to guests with an upcoming stay. [Likely]
62. Homes, Experiences and Services now share discovery surfaces. How would you rank a blended feed whose item types have different conversion rates, prices and data volumes? [Likely] [7][37]
    - How do you calibrate scores across verticals so that one vertical does not crowd out the others?
63. How would you recommend wishlist listings or destinations for group trips where several people collaborate? [Likely]
64. Design a system that matches hosts with co-hosts or local service providers. [Likely] [37]
    - What does a successful match mean, and how do you handle two-sided preferences and capacity limits?
65. How do you avoid a popularity feedback loop in which top listings get more exposure, more bookings and more exposure? [Likely] [5]

G. Host acceptance, response time and other time-to-event predictions

66. Design a model that predicts the probability a host will accept a booking request, and use it in ranking. [Likely] [1]
    - How do you avoid unfairly penalizing a host for a single rejection?
    - How do you use host preferences (trip length, lead time, gaps between bookings) as features?
67. Design a model that predicts host response time to a guest message or booking request. [Likely]
    - How do you handle censoring for messages that never get a reply?
    - Would you use survival or hazard models, or classification at fixed horizons? Why?
68. Predict the probability that a confirmed booking will be cancelled by the host or by the guest. [Likely]
    - How would you use this in ranking, in host coaching, or in guest-protection policy?
69. Predict booking lead time and length of stay for a search, and explain how they would change ranking. [Likely] [1]
70. Design a model that estimates customer-support wait time or resolution time, and routes cases to the right agent queue. [Likely] [20]

H. Categories, content understanding and listing quality

71. Design the system behind Airbnb Categories (for example "Beachfront", "Treehouses"), which assigns listings to themed browse categories. [Likely] [11]

    - How do you combine ML candidates with human-in-the-loop review, and how do you budget human review?
    - How do you rank listings within a category, and how do you rank the categories themselves for a user?
72. Design an image-understanding pipeline that detects amenities and room types from listing photos. [Likely] [21]
    - How would you get labels cheaply, and how do you measure precision when amenities drive search filters?
73. Choose a listing's cover photo to raise its booking rate. [Reported] [22]
    - How do you handle cold start for a new listing with no engagement on any photo? [Reported] [22]
    - How do you avoid choosing a photo that raises clicks but misleads guests, and so raises cancellations or bad reviews?
    - Would you treat this as a bandit, a learned photo-quality model, or both?
74. Design a model that organizes a listing's photos into a room-by-room tour and orders them. [Likely] [21][37]
75. Design a system for finding high-quality listings, used in ranking and host coaching. [Reported] [22]
    - How do you define quality without simply rewarding listings that are already popular?
    - Which signals would you use: photos, description, completeness, reviews, cancellations, response rate?
76. Design the selection of a "most-loved" or "Guest Favorite" designation from ratings, reviews and reliability. [Likely] [37]
    - How do you keep the badge stable over time and hard to game, and how do you measure its effect on bookings?
77. Design a system that detects inaccurate or misleading listing descriptions by comparing text, photos and reviews. [Likely]
78. How would you use LLMs or multimodal models for listing understanding at Airbnb's scale while controlling cost? [Likely]
79. How would you translate or localize listing content and decide when machine translation is good enough to show? [Likely]

I. Reviews: quality, ranking and trust

80. Design a system that ranks reviews on a listing page so guests see the most helpful reviews first. [Likely]
    - How do you define "helpful" without explicit helpfulness votes?
81. Design a fake or retaliatory review detector. [Likely] [19]
82. Design a system that extracts aspects (cleanliness, location, accuracy) from review text and feeds them into search or listing quality. [Likely]
83. Design LLM-generated review summaries or highlights for a listing. [Likely]
    - How do you evaluate faithfulness, so the summary claims nothing the reviews do not support?
    - How do you A/B test whether summaries change booking or cancellation behavior?
84. Ratings are highly skewed toward 5 stars. How do you build a reliable quality signal from them? [Likely]

    - How would you apply Bayesian shrinkage for listings with few reviews?
85. How would you predict a guest's post-stay satisfaction before the stay, and how would you use it in ranking? [Likely] [1]

J. Messaging, customer support and LLM systems

86. Generate customer-support instructions (step-by-step agent guidance) from Airbnb's company policies. [Reported] [22][30]
    - How do you keep generated instructions faithful to the source policy when policies change?
    - How do you evaluate correctness before agents or an automated assistant act on them?
87. Design the retrieval and ranking layer for a generative-AI customer-support tool: given a case, retrieve the right policies, past resolutions and booking context. [Reported] [22]
    - What are the candidate sources, and what is the relevance label for training a ranker?
    - How do you measure whether better retrieval actually reduces resolution time or escalations?
88. Design an ML system that classifies the intent of guest-host messages to route or auto-suggest replies. [Likely] [20]
89. Design an LLM-based customer-support assistant that resolves common issues (cancellations, refunds) and escalates the rest. [Likely] [20]
    - How do you evaluate it offline, monitor hallucinations and policy errors, and measure deflection versus satisfaction online?
    - What guardrails are needed for actions involving money or safety?
90. Design suggested replies for hosts. How do you avoid generating replies that commit the host to things they did not intend? [Likely]
91. How would you build an LLM-as-judge pipeline to produce relevance or quality labels for search and support, and how would you validate it against human raters? [Likely] [20]

K. Experimentation and measurement inside the ML design round

92. For your ranking model, what is the primary metric of the launch A/B test, and what are the guardrails? [Likely] [8][17]
    - Why might "bookings" be a better primary metric than "booking requests" or "clicks"?
93. Design robust primary and guardrail metrics for a product or model change, so the metrics resist gaming and noise. [Reported, role unspecified] [23][33]
94. What randomization unit would you use for a search ranking test (guest, session, search, listing, market)? Why? [Likely] [8]
95. Explain why a guest-randomized test of a ranking change can be biased in a two-sided marketplace with limited inventory. [Likely] [8]
    - How could market-level, cluster or switchback designs help? What do they cost in statistical power?
96. Design an experiment for a listing badge (for example a "fast Wi-Fi" badge) when guests and listings interact across a network and treatment can spill over. [Reported,

role unspecified] [23][35]

97. Design an A/B test with causal inference for a case where clean randomization is hard (hosts opt in, or the change is market-wide). [Reported, role unspecified] [23][34]
    - Which quasi-experimental methods would you use (difference-in-differences, synthetic control, regression discontinuity), and how do you check their assumptions?
98. When would you use interleaving instead of a classic A/B test for ranking? [Likely] [9]
    - How do you attribute a booking to one ranker in an interleaved result list?
    - How do you validate that interleaving and A/B testing agree on direction?
99. Bookings happen days after search, and stays happen weeks later. How do you get a sensitive, timely decision metric? [Likely] [8][17]
    - How would you build and validate a surrogate or predicted long-term value metric?
100. How would you reduce variance in a booking-conversion experiment (CUPED, stratification, trimming)? [Likely] [8]
101. How do you detect and handle sample ratio mismatch, bot traffic or logging bugs in a ranking experiment? [Likely] [8]
102. How do you handle novelty and primacy effects, and seasonality around holidays, in experiment readouts? [Likely]
103. Your model wins overall but loses badly in one market or one guest segment. What do you do? [Likely]
104. How would you set up a long-term holdout to measure the cumulative impact of many ranking launches? [Likely]
105. How would you measure the impact of a pricing or fraud model where the treatment changes host or fraudster behavior over time? [Likely] [4][18]
106. How would you use counterfactual or off-policy evaluation from logged ranking data before running an online test? [Likely] [2] - What propensities do you need to log, and how much randomization would you inject?
107. How do you trade off guest-side and host-side metrics in a single ship decision? [Likely] [17]
108. Design the experimentation platform that your ML launches run on: assignment, exposure logging, metric computation and readout. [Reported, role unspecified] [23] [32] - How would it support marketplace designs (market-level or switchback) and interleaving, not only user-level tests?
109. The ranking team has more candidate models than traffic to test them. How would you prioritize, run sequential or adaptive tests, and control false discoveries across many launches? [Likely] [9]

L. Monitoring, reliability and iteration (senior depth)

110. What would you monitor once your ranking or fraud model is live (data drift, feature nulls, score distributions, business KPIs)? [Likely] - Which alerts would page an on-call engineer, and which go to a dashboard?
111. How would you roll out a new model safely (shadow scoring, canary, ramp, automatic rollback)? [Likely]
112. A feature pipeline broke silently and the model has served on stale features for three days. How do you detect it, mitigate it and prevent it from happening again? [Likely]

113. How do you decide what to build next after launch: more features, a bigger model, or better labels? [Likely] [1][2]
114. How would you reduce serving cost by 50% without measurable loss in bookings? [Likely] - What is your approach to distillation, caching and candidate-set pruning?
115. How would you design the system so that other teams (Experiences, Services, Long-term stays) can reuse your ranking stack? [Likely]
116. Which parts of this design would you not use ML for at all, and why? [Likely] [1]
117. What are the main fairness and discrimination risks in Airbnb ranking and trust systems, and how would you audit for them? [Likely] [19]

M. Variations and curveball prompts

118. Design ranking for long-term stays (28+ nights). What changes from nightly stays? [Likely]
119. Design ranking for Airbnb Services or new verticals with almost no historical data. [Likely] [7][37]
120. Design autocomplete and query understanding for the search box (locations, landmarks, categories). [Likely]
121. Design a model that predicts which markets will face supply shortages for an upcoming event, and recommend actions to recruit hosts. [Likely]
122. Design a host-onboarding recommendation system that tells new hosts what to improve first. [Likely]
123. Design a "price drop" or "rare find" badge. How would you verify that badges are accurate and actually help guests? [Likely]
124. The interviewer scheduled a project deep dive but instead gives you an unfamiliar ML design prompt. How do you scope it in the first five minutes? [Reported] [25]

References

Sources [22]-[25] and [36] were fetched and read on GitHub on 2026-10-14. Sources [26]-[35] are TrueInterview question pages linked from [22] and [23]; their host could not be reached directly. The remaining sources could not be reached from this environment, because the proxy could not resolve arxiv.org, dl.acm.org, medium.com, glassdoor.com or igotanoffer.com. They are listed from prior knowledge. Entries marked "URL not verified" give a title and an index page rather than a guessed article URL. Confirm every link before relying on it.

1. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019. https://arxiv.org/abs/1810.09591
2. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv / KDD 2020. https://arxiv.org/abs/2002.05515
3. Grbovic & Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
4. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219830
5. Abdool, Haldar et al., "Learning To Rank Diversely At Airbnb", arXiv, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774

6. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning", arXiv / KDD 2023. https://arxiv.org/abs/2305.18431
7. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences", Airbnb Tech Blog (Medium), 2019. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
8. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
9. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
10. Grbovic, "Listing Embeddings in Search Ranking", Airbnb Tech Blog (Medium), 2018. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
11. "Building Airbnb Categories with ML and Human-in-the-Loop", Airbnb Tech Blog, 2022-2023. URL not verified. Search via https://medium.com/airbnb-engineering
12. "Embedding-Based Retrieval for Airbnb Search", Airbnb Tech Blog, 2025. URL not verified. Search via https://medium.com/airbnb-engineering
13. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor (candidate reports), ongoing. Host unreachable from this environment. https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm
14. "Airbnb Interview Process & Questions", IGotAnOffer, 2023-2024. Host unreachable from this environment. https://www.igotanoffer.com/blogs/tech/airbnb-interview-process
15. "Airbnb Interview Discussions", LeetCode Discuss (search "Airbnb"), ongoing. https://leetcode.com/discuss/interview-question?currentPage=1&orderBy=hot&query=airbnb
16. "Learning Market Dynamics for Optimal Pricing", Airbnb Tech Blog, 2018. URL not verified. Search via https://medium.com/airbnb-engineering
17. "How Airbnb Measures Future Value to Standardize Tradeoffs", Airbnb Tech Blog, 2022. URL not verified. Search via https://medium.com/airbnb-engineering
18. "Fighting Financial Fraud with Targeted Friction", Airbnb Tech Blog, 2018. URL not verified. Search via https://medium.com/airbnb-engineering
19. Airbnb Trust & Safety and anti-discrimination posts (party prevention, Project Lighthouse), Airbnb Tech Blog and Airbnb Newsroom, 2020-2024. URL not verified. Search via https://medium.com/airbnb-engineering
20. Airbnb posts on message-intent classification and customer-support ML / LLM systems, Airbnb Tech Blog, 2019-2025. URL not verified. Search via https://medium.com/airbnb-engineering
21. Airbnb posts on amenity detection and room-type classification from listing photos, Airbnb Tech Blog, 2018-2019. URL not verified. Search via https://medium.com/airbnb-engineering
22. "Airbnb Machine Learning Engineer interview process" (TrueInterview-derived summary of 3 candidate reports, mostly 2026 Q1-Q2: ML design prompts, LTV and relisting steering, cover photo, high-quality listings, GenAI support retrieval and ranking). kevin-2023-code/Tech-Interview-Questions, GitHub. Fetched 2026-10-14. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md

23. "Airbnb" question index with report dates (user-embedding semantic search, Apr 2026; A/B testing platform, Apr 2026; rank home results without a text query, Sep 2026; listing LTV, May 2026; experiment-design prompts). kevin-2023-code/Tech-Interview-Questions, GitHub. Fetched 2026-10-14. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
24. 1point3acres post #40 (English mirror): ML design rounds "design a system to calculate customer lifetime value" and "design a system to help landlords set prices", plus coding LC 1298. FlowSight/coding, GitHub. Fetched 2026-10-14. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
25. 1point3acres post #79 (English mirror): two ML design rounds (Airbnb listing recommendation; price prediction); the ML experience round turned into an unexpected ML design question; ConnectN coding. FlowSight/coding, GitHub. Fetched 2026-10-14. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
26. TrueInterview, "Design A Personalized Search Ranking System" (tagged Airbnb, among others). Linked from [22]. https://trueinterview.io/questions/d6c8316c-c7a9-4074-bec9-f19e4252f21d
27. TrueInterview, "Listing Lifetime Value — Estimation" (reported May 2026). Linked from [22][23]. https://trueinterview.io/questions/listing-lifetime-value-ml-design
28. TrueInterview, "Design Airbnb Relisting Detection". Linked from [22]. https://trueinterview.io/questions/design-airbnb-relisting-detection
29. TrueInterview, "Rank Home Search Results Without a Text Query" (reported Sep 2026). Linked from [22][23]. https://trueinterview.io/questions/rank-home-search-results-without-a-text-query
30. TrueInterview, "Generate Customer-Support Instructions from Company Policies". Linked from [22]. https://trueinterview.io/questions/generate-customer-support-instructions-from-company-policies
31. TrueInterview, "Design User Embedding Semantic Search" (reported Apr 2026). Linked from [23]. https://trueinterview.io/questions/design-user-embedding-semantic-search
32. TrueInterview, "Design an A/B Testing Platform" (reported Apr 2026). Linked from [23]. https://trueinterview.io/questions/design-an-a-b-testing-platform-3
33. TrueInterview, "Design robust primary and guardrail metrics". Linked from [23]. https://trueinterview.io/questions/design-robust-primary-and-guardrail-metrics
34. TrueInterview, "Design an A/B test with causal inference". Linked from [23]. https://trueinterview.io/questions/design-an-a-b-test-with-causal-inference
35. TrueInterview, "Design a network-aware Wi-Fi badge experiment". Linked from [23]. https://trueinterview.io/questions/design-a-network-aware-wifi-badge-experiment
36. A. Dirafzoon, "ML System Design" sample questions (lists "Rental recommendation (Airbnb)" among common MLE design prompts). alirezadir/AIMLInterviews, GitHub. Fetched 2026-10-14. https://github.com/alirezadir/AIMLInterviews/blob/main/src/MLSD/ml-system-design.md
37. Airbnb product release announcements: Guest Favorites (2023), Photo Tour (2023), Co-Host Network (2024), and the Services and Experiences relaunch (2025). Airbnb Newsroom. URL not verified. Search via https://news.airbnb.com
38. "Learning to Rank for Maps at Airbnb", Airbnb search team, KDD 2024. URL not verified. Search the title on arXiv or https://medium.com/airbnb-engineering

04. A/B Testing and Experimentation Deep Dive: ERF, Marketplace Interference, Variance Reduction, Interleaving and Sequential Testing

> Research note (read first). This chapter went through an adversarial review on 2026-10-14. During that review the egress proxy blocked medium.com, arxiv.org, igotanoffer.com, glassdoor.com, dl.acm.org, airbnb.tech, teamblind.com, 1point3acres.com, leetcode.com and reddit.com, and the shared web-search budget was already used up. Only raw.githubusercontent.com was reachable. Five GitHub-hosted sources were fetched and read: two curated indexes of company engineering posts [23][24], which confirm the title, URL and year of the Airbnb posts cited here; the community Airbnb question list [22]; and two general ML/DS interview question banks. None of them contains a candidate report of an Airbnb experimentation or A/B-testing question. **Every question is therefore tagged [Likely].** No question is tagged [Reported], and none could honestly be. The 2024-2026 candidate reports the review set out to find (Glassdoor, Blind, 1point3acres, LeetCode Discuss) could not be reached. Citations such as "[Likely; 3]" point to the publication that makes a topic a plausible probe. They do not mean the question was asked. The References section marks each entry as **confirmed via index** (title, URL and year confirmed from [23] or [24], page itself not opened), **verified** (page fetched and read) or **unverified** (from prior knowledge only). To close the gap, rerun with web access and look for real candidate reports.

Format notes

- **Where this material shows up.** Public descriptions of Airbnb's engineering loop (coding x2, system design, ML design, past-project deep dive, core values x2) do not list a separate "A/B testing" round for MLEs [9][10]. For an MLE role that names experimentation or A/B testing, expect it in three places: (a) the evaluation and launch half of the ML design round; (b) the past-project deep dive, with questions like "how did you know it worked?"; and (c) possibly a dedicated experimentation or analytics conversation with a data scientist or experimentation-platform engineer. Ask your recruiter which of these your loop includes. [Likely]
- **Duration and interviewer.** Usually 45-60 minutes with a senior or staff MLE, a data scientist (Airbnb's DS org has an Inference/Experimentation track) or an engineer from the experimentation platform team (ERF) [1][3]. [Likely]
- **The senior (L5) bar.** It goes beyond knowing the t-test. Expect to show you can:

    - spot when the standard A/B setup is biased (interference, novelty, selection, peeking);
    - choose the randomization unit and the metrics, and defend that choice;
    - quantify trade-offs between bias and variance and between speed and rigor;
    - turn ambiguous or conflicting results into a launch decision that Product and leadership accept.

  Airbnb's own writing stresses that experiments run on a two-sided marketplace with long, rare conversion funnels (search, then contact or book, then stay, then review). Expect probes on exactly that [1][2][4][5]. [Likely]

- **Format of the questions.** Expect open-ended scenarios ("we changed X in search ranking; design the test; results came back like this; what now?"). Some rounds may add light whiteboard math, such as sample size, standard errors or CUPED adjustment. Expect few closed-form trivia questions. [Likely]

Questions

A. Airbnb experimentation platform (ERF) and experiment fundamentals

1. Walk me through how you would design an online A/B test for a new search-ranking model at Airbnb, from hypothesis to launch decision. [Likely; 1][2]
    - What is your primary metric, and why not just click-through rate?
    - What is the randomization unit, and what are you assuming by choosing it?
    - How long would you run it, and what determines that?
2. Airbnb's experimentation platform (ERF, the Experiment Reporting Framework) computes metrics for hundreds of concurrent experiments. If you were designing such a platform, what are its core components? [Likely; 3]
    - How would you implement deterministic, salted hashing for assignment so that concurrent experiments are independent?
    - How do you log exposures versus assignments, and why does the difference matter?
    - How would you make metric definitions consistent across teams (for example, through a central metrics layer)? [Likely; 11]
3. Airbnb's 2017 post is about scaling ERF as the number of experiments grew. If concurrent experiments and metrics grew 10x again, where would the pipeline break first, and how would you restructure it? [Likely; 3]
    - What would you precompute per user-day versus compute on demand?
    - How would you keep results reproducible when metric definitions change mid-experiment?
4. What is the difference between assignment and exposure (trigger) analysis, and when is an intent-to-treat analysis the right choice? [Likely]
    - The treatment only changes the page when a listing has more than 10 photos. Who should be in your analysis population?
    - How does triggering affect your power, and how do you translate a triggered effect back to a site-wide impact?
5. Your exposure log fires on page load for every visitor, including those who never reach the component you changed (overtracking). How does this dilute the measured effect

and the power, and how would you fix both the logging and the analysis? [Likely; 29]

6. For a search-ranking test, treatment and control produce identical results on many queries. How would you define a triggered population of queries where the rankings actually differ, and what does computing the counterfactual control ranking cost in serving? [Likely; 29][8]
    - How do you avoid bias when the trigger condition itself depends on the treatment model's output?
7. Many Airbnb users browse logged out and on several devices before booking. How does this affect randomization and metric attribution? [Likely; 1]
    - Would you randomize by cookie, device, user ID or something else? What breaks in each case?
    - How would you handle a user who is assigned on a logged-out device and then logs in?
8. Request-to-book listings need host acceptance, and Instant Book listings do not. How do you define 'conversion' for a guest-side experiment when part of the outcome is the host's decision? [Likely]
    - What happens to your metric if treatment sends more requests to hosts with low acceptance rates?
9. What is a sample ratio mismatch (SRM)? How would you detect it, and what are its common causes on a web/mobile booking site? [Likely]
    - The treatment loads more slowly, so fewer treatment users fire the exposure log. What does that do to your results?
    - Would you ever analyze an experiment that fails the SRM check?
10. How would you run an A/A test, and what would you use it for on a mature platform? [Likely]
    - Your A/A tests show 9% significant results at alpha = 0.05 on one metric. What do you investigate?
11. Explain the p-value and the confidence interval of a difference in means to a product manager who wants a yes/no answer. [Likely]
12. How do you compute the sample size and minimum detectable effect (MDE) for a booking-conversion experiment? [Likely; 1]
    - Booking conversion is low and bookings are lumpy. How does that change the calculation?
    - The PM wants a two-week answer, but power analysis says six weeks. What options do you propose?
13. Your metric is a ratio, such as bookings per search or revenue per booking, where the analysis unit differs from the randomization unit. How do you compute its variance correctly? [Likely]
    - When would you use the delta method, and when a bootstrap?
    - What goes wrong if you treat each search as independent?
14. How do you handle heavy-tailed metrics like gross booking value, where a few long or luxury stays dominate? [Likely]
    - Would you winsorize/cap, log-transform or use a different estimator? What does each do to interpretation?
15. A treatment raises bookings per user but lowers booking conversion per search. How can both be true, and which metric do you trust for the launch decision? [Likely]
16. Airbnb's 2014 post describes an experiment whose result flipped depending on when it was checked, and a case where a browser-specific bug drove the result. What

general lessons do you take from such cases? [Likely; 1]
    - How would you build a platform guard against "result depends on the day you looked"?
    - How would you segment by platform/browser without going on a fishing expedition?
17. How long should a booking experiment run given weekly seasonality and a lead time between search and check-in of days to months? [Likely; 1]
    - What do you lose if you stop after full weeks but before guests actually stay?
18. How would you design guardrails that block a launch automatically? What would the guardrail metrics be for a search change at Airbnb? [Likely; 4]
    - How do you set the guardrail threshold so it is neither too noisy nor too lax?
    - Who should be able to override a guardrail, and how should overrides be logged?
19. Airbnb has written about designing experimentation guardrails. Beyond a guardrail that fires on a large negative impact, why might a platform also need a guardrail that checks whether each guardrail metric is adequately powered, and how does it change how long teams must run tests? [Likely; 4]

B. Metric choice: north stars, proxies and long-term value

20. What should the primary success metric be for a search-ranking experiment at Airbnb: bookings, booking value, nights booked, uncancelled bookings or something else? Defend your choice. [Likely; 2][5]
    - Why might "uncancelled bookings" or "booking with no host cancellation" be better than raw bookings?
    - How would you weigh guest-side gains against host-side outcomes?
21. Bookings are slow and rare, so the team wants a faster proxy metric (listing clicks, wishlist saves, contacts). How do you validate that a proxy is aligned with the north star? [Likely; 2]
    - Show me a scenario where the proxy improves and bookings drop.
    - How would you use historical experiments to measure proxy-to-north-star correlation, and what are the pitfalls?
22. Airbnb has written about putting a monetary future value on actions so that teams can compare trade-offs. How would you use such a metric in an experiment, and how could it mislead you? [Likely; 6]
    - How do you validate a model-based "future value" metric before letting it decide launches?
23. How would you measure long-term effects such as repeat booking, host churn or review quality that appear months after a test ends? [Likely]
    - What are the trade-offs of long-running holdouts?
    - How would you use surrogate indices or a long-term holdback group?
    - After launch, how would you use a long-term holdback to measure whether the feature's effect decays? How large should it be and how long should it run? [Likely]
24. Design an overall evaluation criterion (OEC) for a pricing-suggestion feature shown to hosts. [Likely; 7]
    - What are the host-side metrics, the guest-side metrics and the marketplace-level metrics?

    - Host earnings rise but occupancy falls. Is that a win?
25. How do you choose between a mean metric (average bookings per user) and a proportion metric (percentage of users who book)? [Likely]
26. How would you test whether a change shifts the 90th percentile of a metric, such as search latency or booking lead time, rather than the mean? How do you get valid confidence intervals for a quantile difference at platform scale? [Likely; 30]
27. You track 50 metrics, and three moved significantly in the wrong direction. How do you decide whether to worry? [Likely]
    - How do you control for multiple comparisons without destroying power?
    - Which metrics would you pre-register as primary, secondary and guardrail?
28. How would you measure the quality of search results beyond conversion, for example relevance, diversity or "guest found what they wanted"? [Likely; 8]
    - How would you test a diversity objective whose value is not captured by short-term bookings?
29. Airbnb has published work on managing diversity in search and on learning to rank diversely. How would you run an experiment on a diversity change whose benefit is that some guests find a match they would otherwise miss, while per-search booking rate for the majority may dip slightly? [Likely; 25][26]
    - Which metric captures 'guests who would otherwise have booked nothing'?
30. Cancellation and refund outcomes happen after the experiment window closes. How do you build a metric that accounts for them without waiting months? [Likely]

C. Marketplace interference and two-sided experiments

31. Why can a standard user-randomized A/B test give a biased estimate in a two-sided marketplace like Airbnb? Explain it with an example. [Likely; 12][13]
    - In which direction is the bias usually, for a ranking change that helps treatment guests book popular listings?
    - When is the interference negligible?
32. A treatment ranking model steers guests toward high-quality listings. Treatment and control guests compete for the same limited inventory on the same dates. What does the A/B test overstate, and why? [Likely; 12]
    - How would you estimate the size of this cannibalization bias?
33. Compare guest-side (demand) randomization with listing/host-side (supply) randomization. When would you use each? [Likely; 13]
    - Which side do you randomize for a host pricing tool, and what interference does that leave?
    - What is the interference pattern when you randomize listings for a ranking boost?
34. Explain two-sided randomization (randomizing both guests and listings, then analyzing the cells). What bias does it reduce, and at what cost? [Likely; 13]
35. Write down the estimand you actually care about for a marketplace launch (the global treatment effect: everyone treated versus no one treated). How does it differ from the difference in means in a user-randomized test under interference? [Likely; 13]
36. Airbnb collaborated on research about reducing interference bias in marketplace pricing experiments. How would you design a pricing experiment that limits interference? [Likely; 12]

    - How would cluster randomization by market or by "similar listings" help, and what does it do to variance?
    - How would you build listing clusters (for example, by geography and listing similarity) so that most competition stays within a cluster?
37. Compare switchback (time-based) designs, geo/market-level randomization and cluster randomization for an Airbnb search or pricing change. [Likely]
    - Why are switchbacks awkward when there is a long lead time between booking and stay?
    - With only a few hundred markets, how do you get enough power from a market-level experiment?
38. How would you detect interference empirically, for example by running the same treatment at two allocation levels (10% versus 50%)? [Likely; 13]
    - What would you conclude if the effect at 50% allocation is half the effect at 10%?
39. Describe an 'A/B test of A/B tests': randomly assigning clusters to either individual-level or cluster-level randomization and comparing the two estimates to detect interference. How would you adapt this for Airbnb listings clustered by market and date? [Likely; 27]
    - How many clusters would you need for the comparison to have any power?
40. Market conditions matter. When supply is tight (peak dates), interference bias grows. How would you account for supply/demand balance in the analysis? [Likely; 13]
    - Would you report the effect separately for supply-constrained and demand-constrained markets?
41. A change makes guests book earlier. Total bookings over the quarter may not change, but the A/B test shows a lift. How would you detect this kind of intertemporal substitution (pull-forward)? [Likely]
42. A host-side change (for example, better calendar sync) increases the number of available nights. How would you measure its marketplace effect, given that extra supply in treatment can take bookings from control listings? [Likely]
43. How would you experiment on a host-side product, such as a new listing-quality nudge, when the outcome you care about (bookings) is driven by guests who are not in your experiment? [Likely]
44. Network effects among guests are weak on Airbnb, but interference through inventory is strong. Contrast this with social networks, and explain how the remedies differ. [Likely]
45. Airbnb Experiences has far smaller inventory than Homes, with fixed time slots that sell out. How would interference and power differ from a Homes ranking test, and how would you design an Experiences ranking experiment? [Likely; 38]
46. How would you use a market-level holdout ("launch everywhere except these N cities") to validate the global effect of a ranking system after launch? [Likely]
    - How do you choose holdout markets, and how do you analyze them (difference-in-differences, synthetic control)?
47. When randomization at the right level is impossible, how would you estimate the causal effect of a marketplace-wide change? [Likely; 14]
    - How does a counterfactual model (predict what would have happened without the change) work, and how do you validate it?
    - What assumptions does synthetic control rely on, and how would you check them?

D. Variance reduction: CUPED, stratification and regression adjustment

48. Explain CUPED. What is it, why does it reduce variance, and why does it not bias the treatment effect estimate? [Likely; 15]
    - How do you choose the covariate, and what is the optimal theta?
    - Why must the covariate be measured before the experiment starts?
49. Booking is rare, and many users have no pre-period activity (new or logged-out users). How does CUPED perform for them, and what would you do instead? [Likely]
    - Could you use a model-predicted covariate, for example a predicted booking probability (CUPAC-style)? What are the risks?
50. DoorDash describes CUPAC, which uses an ML model's prediction as the CUPED covariate. What must be true of that model (training window, features, version freezing) for the adjusted estimator to stay unbiased? What happens if the model was trained on data that overlaps the experiment period? [Likely; 28]
51. How do CUPED and regression adjustment with several covariates relate? When would you use one over the other? [Likely]
52. Explain stratified sampling versus post-stratification for variance reduction. Which strata would you use for Airbnb (market, platform, new versus returning guests)? [Likely]
53. You applied CUPED and the variance fell by only 3%. Why might that happen for a booking metric, and what else would you try? [Likely]
54. How does variance reduction change your sample-size calculation and your experiment duration? [Likely]
55. Can you combine CUPED with ratio metrics and with the delta method? What do you need to be careful about? [Likely]
56. You cap gross booking value at the 99th percentile and also apply CUPED. Should you cap before or after the adjustment? What does each order do to bias, variance and interpretation? [Likely]
57. When would you choose a more sensitive design over variance reduction in the analysis? Examples include interleaving, paired designs and within-user designs. [Likely; 8]

E. Interleaving for search ranking

58. What is interleaving, and why is it much more sensitive than an A/B test for comparing rankers? [Likely; 8]
    - Compare team-draft interleaving with balanced interleaving.
    - How do you assign credit for a booking, as opposed to a click, in an interleaved result list?
59. Airbnb has described using interleaving to speed up search-ranking experiments. How would you set it up so that the interleaving verdict agrees with A/B results? [Likely; 8]
    - How would you measure agreement between interleaving and A/B tests over a set of historical launches?
    - What would you do if interleaving says ranker B wins but the A/B test is flat?
60. Airbnb search is not a single click-and-done session. Guests search many times over days, and listings appear on map views and across pages. What complications does this create for interleaving? [Likely; 8]

    - How would you handle the same listing appearing in both rankers' lists, and duplicates across searches?
    - How do you attribute a booking that happens days later?
61. Interleaving is a within-user design. How would you compute its significance (for example, a per-user preference sign test or a bootstrap over users), and why is per-impression analysis wrong? [Likely; 8]
62. What can interleaving not tell you? Give examples of ranking changes that must still be A/B tested (for example, changes to price display, diversity or result-set size). [Likely; 8]
63. Can you run interleaving and A/B tests on the same search traffic at the same time? How would you allocate traffic so that neither contaminates the other? [Likely; 8]
64. Design a two-stage experimentation funnel: offline evaluation, then interleaving, then A/B test. What are the gates between stages? [Likely; 2][8]
    - Offline NDCG improves but interleaving is flat. What is your hypothesis?

F. Offline-online gap and ML-specific experimentation

65. Your new ranking model improves offline NDCG by 2%, but the A/B test shows no booking lift. What are the possible reasons, and how do you debug? [Likely; 2][5]
    - Could position bias or feedback loops in the training data explain it?
    - How would you tell a modeling problem apart from an experiment-sensitivity problem?
66. Airbnb's deep-learning search papers describe many launches that improved offline metrics but failed online. How would you build an offline metric that better predicts online wins? [Likely; 2][5]
67. When training data come from the control model's own rankings, the treatment model is evaluated on data biased toward control. How does this affect A/B results, and how would you mitigate it? [Likely]
    - What role do exploration traffic or randomized-result buckets play?
68. How would you use off-policy evaluation (counterfactual estimators such as inverse propensity scoring) to pre-screen ranking models before an online test? What does it require from logging? [Likely]
69. Treatment and control models share a training pipeline. After launch, the new model's data feeds back into retraining. How would you measure that feedback-loop effect? [Likely]
70. Your ranking model is retrained weekly with the same architecture on fresh data. Do you A/B test every refresh? Design a policy for automated model refreshes with lighter-weight validation and a rollback trigger. [Likely; 2][5]
71. How would you A/B test a model whose outputs are cached or precomputed, such as listing embeddings or nightly batch scores? [Likely; 16]
    - What contamination can occur if the treatment's embeddings are also used by control's similar-listings carousel?
72. How would you run an experiment on an ML model that serves several surfaces (search, similar listings, email recommendations) at once? [Likely]
73. A new home-page browsing surface (such as Airbnb Categories) changes which searches guests run next. How would you separate its effect on search metrics from changes in ranking quality, and how would you test a ranking change that runs alongside it? [Likely; 39]

74. How would you test a latency-increasing model (larger neural net) when latency itself hurts conversion? [Likely]
    - How would you separate the model-quality effect from the latency effect?
75. How would you A/B test an LLM-powered customer-support feature, such as suggested replies for agents, where outcomes include resolution time, CSAT, escalations and policy errors? [Likely; 40]
    - Do you randomize by agent, by ticket or by guest, and what interference does each choice leave?
    - How do you combine offline LLM evaluation with the online test?
76. How would you experiment on a fraud or trust model, where false positives block real guests and the true outcome (fraud, chargebacks) arrives weeks later? [Likely; 17]
    - Is holding out a random set of transactions from the fraud model ethical or practical? What would you propose?
77. Treatment and control both generate logs that feed the next model's training data through the feature store. How do you tag and filter experiment traffic in feature and training pipelines so that one arm does not contaminate the next model? [Likely; 37]

G. Sequential testing, peeking and decision rules

78. What goes wrong when a team peeks at an A/B test every day and stops as soon as $p < 0.05$? Quantify it roughly. [Likely; 1]
79. Explain sequential testing approaches (group sequential with alpha spending, always-valid p-values or mSPRT). Which would you build into an experimentation platform, and why? [Likely; 18]
    - What do you give up in power compared with a fixed-horizon test?
    - How would you explain an "always-valid" confidence interval on a dashboard?
80. Airbnb's 2014 post proposed a dynamic p-value threshold that is stricter early on. How does that compare with modern sequential methods? [Likely; 1]
81. When is early stopping for harm appropriate, versus early stopping for benefit? [Likely]
82. A test has run for its planned duration, and the p-value is 0.07. The PM wants two more weeks. What do you advise? [Likely]
83. A bug in treatment is found and fixed on day 6 of a 21-day test. Do you restart, analyze only the post-fix period or analyze everything? What are the trade-offs? [Likely]
84. How would you apply a Bayesian A/B testing approach to booking conversion? When does it help decision-making, and when does it mislead? [Likely]
    - How would you set priors from Airbnb's history of past experiments?
85. When would you replace an A/B test with a multi-armed bandit, for example for email subject lines or the order of home-page categories? What do you lose for inference and for long-term measurement? [Likely; 34]
86. Explain the winner's curse in experimentation. Airbnb has written about correcting for selection bias when summing the impact of launched winners. How would you correct it? [Likely; 19]
    - Leadership adds up the measured lifts of 40 launched experiments, and the total far exceeds the actual business growth. Why?
87. How would you control the false discovery rate across hundreds of experiments and metrics on a platform? [Likely]

88. Your platform runs about 1,000 experiments a year at alpha = 0.05 and 80% power, and only about 10% of ideas truly work. Roughly what fraction of 'significant' wins are false positives? How should that change launch criteria? [Likely; 35]

H. Novelty, primacy, learning and time effects

89. How do you detect novelty or primacy effects in an Airbnb experiment, for example a redesigned listing page? [Likely]
    - How would you plot and test the treatment effect over time or by days since first exposure?
    - Why are novelty effects harder to see when most guests visit Airbnb only a few times a year?
90. How do hosts learn and adapt to a new host-side feature, such as changing prices after a pricing tip? How would you design the experiment to capture the post-adaptation equilibrium? [Likely; 7]
91. How would you handle seasonality and holidays, for example a test that spans the summer peak or the New Year period? [Likely]
    - Would you pause, extend or reweight?
92. A city regulation forces a change (for example, a new fee display) on every listing in that market at once. How would you estimate its impact with a quasi-experiment, and what are the main threats (few units, parallel trends, spillover into nearby markets)? [Likely; 31][14]
93. Experiments run during an external shock (for example a pandemic-scale demand drop or a major event in a city). How do you decide if the results still generalize? [Likely]
94. Should Airbnb keep a universal holdout group that sees none of a year's launches? What would you measure, how large should it be, and what does it cost the business and the users in it? [Likely; 32]

I. Heterogeneous effects, segments and the launch decision

95. The overall effect is flat, but new guests show +3% and returning guests show -2%. How do you decide whether to launch, and whether to personalize? [Likely]
    - How do you avoid Simpson's paradox in this reading?
    - How do you know the segment effect is not a multiple-testing artifact?
96. How would you estimate heterogeneous treatment effects with ML (for example causal forests or uplift models), and how would you validate them? [Likely]
97. You estimated heterogeneous treatment effects and want to show the feature only to segments that benefit. How do you turn the estimates into a targeting policy, and how do you validate the policy itself with a new experiment? [Likely; 33]
98. A treatment helps in large markets and hurts in small markets. What could explain this in a marketplace, and what would you do? [Likely]
99. The experiment improves bookings but reduces revenue per booking because guests choose cheaper stays. How do you make the launch call? [Likely; 6]
100. Results are significant on iOS and web but not on Android. How do you reason about it? [Likely; 1]
101. Walk me through the launch-review discussion you would lead when the primary metric is up, one guardrail is slightly down and the result is borderline significant.

[Likely; 4] - What would you write in the experiment review document? - How do you communicate uncertainty to a VP who wants a single number?

102. How would you size the actual business impact of a launch, as opposed to the experiment's measured lift? Consider trigger rate, ramp-up, interference and decay. [Likely; 19]

J. Experiment infrastructure and ML-platform design for experiments

103. Design the assignment service for an experimentation platform that serves search, mobile apps and email, with consistent bucketing across surfaces. [Likely; 3] - How do layers or mutually exclusive groups work, and when do you need them? - How do you handle ramping from 1% to 50% without reshuffling users?
104. Airbnb's mobile apps fetch experiment configuration at startup, and many users run older app versions or open the app offline. How do you handle version eligibility, cached assignments and late exposure logging in the analysis? [Likely; 36]
105. Design the metrics computation pipeline behind a system like ERF: daily aggregates for thousands of metrics across hundreds of experiments. [Likely; 3][11] - How do you make it incremental, backfill-able and consistent with the company's metric definitions? - How do you make CUPED and the delta method efficient at that scale?
106. Two notifications experiments (email and push) run at once, and both are subject to a global frequency cap. How does the cap create interference between the experiments and between arms, and how would you design around it? [Likely]
107. How do you detect and handle interactions between concurrent experiments, for example two ranking experiments that both touch the same feature? [Likely]
108. How would you design an experiment-quality monitoring system that automatically checks SRM, logging gaps, pre-period imbalance and metric anomalies? [Likely; 4]
109. How would you architect shadow and canary deployment for a new ranking model before it enters an A/B test? [Likely] - What would you compare in shadow mode, and what kinds of failure would make you abort?
110. How would you store and expose historical experiment results so that teams can run meta-analyses, such as proxy validation or calibrating priors? [Likely]
111. How would you build self-serve power analysis into the platform, given that metric variances differ by market and season? [Likely]

K. Pricing, Smart Pricing and host-side experiments

112. Design an experiment for a change to a host price-suggestion model. What is randomized, and what is measured? [Likely; 7][12] - Hosts may ignore suggestions. How does adoption affect your estimand (intent-to-treat versus compliers)? - How do you prevent treatment hosts' price changes from affecting control hosts' bookings?
113. How would you estimate the price elasticity of demand from experiments without running illegal or unfair price discrimination on guests? [Likely] - Why do companies like Airbnb generally avoid randomizing the price shown to guests, and what do they do instead?
114. Design an experiment for a weekly or monthly-stay discount nudge shown to hosts, where guests book long stays months ahead and the stay itself spans weeks. When can you read results, and what interim metrics would you trust? [Likely; 7]

115. A pricing experiment shows host revenue up 4% in treatment, but marketplace-wide bookings are flat. How do you interpret this? [Likely; 12]
116. How would you test a change in guest fees or the display of total price (for example, showing all-inclusive prices)? Which interference and regulatory concerns apply? [Likely]

L. Fairness, ethics and trust in experimentation

117. How would you check that an experiment does not harm a protected group of guests or hosts, given that you may not observe protected attributes directly? [Likely; 20] - How might perceived race or gender be measured under privacy constraints (for example, in Airbnb's Project Lighthouse work), and how would that feed into experiment analysis?
118. How would you add a fairness guardrail, such as a gap in booking acceptance rates between groups, to every experiment's scorecard? What power do you realistically have to detect a small change in that gap? [Likely; 20]
119. Which experiments should not be run at all, or need special review, on a travel/housing marketplace? [Likely]
120. How would you experiment on a trust and safety intervention, such as party prevention or booking restrictions, where the main outcome is a rare, severe harm? [Likely; 17]

M. Past-project deep dive: experimentation probes

121. Tell me about an experiment you ran where the result surprised you. What did you do next? [Likely]
122. Tell me about a time an A/B result was wrong or misleading, for example because of a bug, SRM, interference or novelty. How did you find out? [Likely]
123. Describe a time you pushed back on launching a feature that had a "statistically significant" win. [Likely]
124. Describe a time you changed how your team or org runs experiments, such as a new metric, a new method or new tooling. What was the impact? [Likely]
125. What was the smallest effect you ever detected reliably, and what did you do to get enough sensitivity? [Likely]
126. How did you decide between running an experiment and shipping on offline evidence? Give a time you chose not to A/B test. [Likely]
127. Tell me about a time a limitation of your experimentation platform forced a workaround. What did you build, and what would you change if you owned the platform? [Likely]
128. How do you work with data scientists, PMs and engineers when you disagree about how to read an experiment? [Likely]

N. Quick-fire concept checks (likely as follow-ups)

129. What is the stable unit treatment value assumption (SUTVA), and where does it fail at Airbnb? [Likely; 13]
130. Why is the t-test usually fine for non-normal metrics at Airbnb scale, and when does it break? [Likely]

131. What is the difference between relative and absolute lift, and how do you build a confidence interval for relative lift? [Likely]
132. What is the difference between a cluster-randomized design's unit of randomization and its unit of analysis, and how do you compute the standard errors? [Likely]
133. What is Twyman's law, and how does it apply to experiment review? [Likely]

References

Status key: **verified** = fetched and read during the 2026-10-14 review. **Confirmed via index** = title, URL and year match the verified curated lists [23] or [24], but the page itself could not be opened (medium.com and arxiv.org were blocked). **Unverified** = listed from prior knowledge; confirm before relying on it. No source below is a candidate report of an Airbnb experimentation question.

1. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7 (unverified: medium.com blocked)
2. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv 2018 (KDD 2019). https://arxiv.org/abs/1810.09591 (confirmed via index [23])
3. "Scaling Airbnb's Experimentation Platform" (ERF), Airbnb Engineering (Medium), 2017. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166 (confirmed via index [23])
4. "Designing Experimentation Guardrails", Airbnb Engineering (Medium), 2021. https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669 (confirmed via index [23])
5. Haldar et al., "Improving Deep Learning for Airbnb Search", arXiv 2020 (KDD 2020). https://arxiv.org/abs/2002.05515 (confirmed via index [23]). Companion post: "Improving Deep Learning for Ranking Stays at Airbnb", https://medium.com/airbnb-engineering/improving-deep-learning-for-ranking-stays-at-airbnb-959097638bde (confirmed via index [24])
6. "How Airbnb Measures Future Value to Standardize Tradeoffs", Airbnb Engineering (Medium), 2021. https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5 (confirmed via index [23])
7. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing", KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219830 (unverified: dl.acm.org blocked)
8. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e (confirmed via index [23])
9. "Airbnb Interview Process & Questions", IGotAnOffer. https://www.igotanoffer.com/blogs/tech/airbnb-interview-process (unverified: domain blocked; used only for loop structure in the Format notes)
10. "Airbnb Machine Learning Engineer Interview Questions", Glassdoor (candidate reports). https://www.glassdoor.com/Interview/Airbnb-Machine-Learning-Engineer-Interview-Questions-EI_IE391850.0,6_KO7,32.htm (unverified: domain blocked; used only for loop structure)

11. "How Airbnb Achieved Metric Consistency at Scale" (Minerva metrics platform), Airbnb Engineering, 2021. No URL given because none could be confirmed. Search the title on the Airbnb engineering blog. (unverified)
12. Holtz, Lobel, Liskovich and Aral, "Reducing Interference Bias in Online Marketplace Pricing Experiments", working paper, 2020. Search the title on arXiv or SSRN. (unverified)
13. Johari, Li, Liskovich and Weintraub, "Experimental Design in Two-Sided Platforms: An Analysis of Bias", Management Science, 2022 (arXiv preprint 2020). Search the title on arXiv. (unverified)
14. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512 (confirmed via index [23])
15. Deng, Xu, Kohavi and Walker, "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data" (CUPED), WSDM 2013. Search the title in the ACM Digital Library. (unverified)
16. Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb", KDD 2018. https://www.kdd.org/kdd2018/accepted-papers/view/real-time-personalization-using-embeddings-for-search-ranking-at-airbnb (confirmed via index [23]; the earlier dl.acm.org link was replaced with the URL the index gives)
17. "Fighting Financial Fraud with Targeted Friction", Airbnb Engineering, 2018. Search the title on the Airbnb engineering blog. (unverified)
18. Johari, Pekelis and Walsh, "Always Valid Inference: Bringing Sequential Analysis to A/B Testing", arXiv 2015 / Operations Research 2022. Search the title on arXiv. (unverified)
19. "Selection Bias in Online Experimentation" (winner's curse correction), Airbnb Engineering, around 2018-2019. Search the title on the Airbnb engineering blog. (unverified)
20. Airbnb "Project Lighthouse" technical paper on measuring discrimination using perceived race under privacy constraints, 2020. Search via the Airbnb Newsroom. (unverified)
21. Kohavi, Tang and Xu, "Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing", Cambridge University Press, 2020. General background for SRM, A/A tests, Twyman's law and OEC questions. https://www.researchgate.net/publication/339914315_Trustworthy_Online_Controlled_Experiments_A_Practical_Guide_to_AB_Testing (link as listed in the verified khangich/machine-learning-interview README; page not opened)
22. allaboutjst, "AirBnB Interview Questions" (community-collected from Glassdoor and GeeksForGeeks, last updated 2020). https://github.com/allaboutjst/airbnb (verified 2026-10-14). Contains only coding questions and no experimentation questions, which is why no question here is tagged [Reported].
23. Eugene Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production", GitHub. https://github.com/eugeneyan/applied-ml (verified 2026-10-14; its "Validation and A/B Testing" section is the index used to confirm references above and below)
24. Engineer1999, "A Curated List of ML System Design Case Studies", GitHub. https://github.com/Engineer1999/A-Curated-List-of-ML-System-Design-Case-Studies (verified 2026-10-14)

25. Abdool et al., "Managing Diversity in Airbnb Search", arXiv, 2020. https://arxiv.org/abs/2004.02621 (confirmed via index [23])
26. "Learning To Rank Diversely", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621 (confirmed via index [23][24])
27. "Detecting Interference: An A/B Test of A/B Tests", LinkedIn Engineering, 2019. https://engineering.linkedin.com/blog/2019/06/detecting-interference--an-a-b-test-of-a-b-tests (confirmed via index [23])
28. "Improving Experimental Power through Control Using Predictions as Covariate (CUPAC)", DoorDash Engineering, 2020. https://doordash.engineering/2020/06/08/improving-experimental-power-through-control-using-predictions-as-covariate-cupac/ (confirmed via index [23])
29. "Overtracking and Trigger Analysis: Reducing sample sizes while INCREASING sensitivity", Booking.com (booking.ai), 2022. https://booking.ai/overtracking-and-trigger-analysis-how-to-reduce-sample-sizes-and-increase-the-sensitivity-of-71755bad0e5f (confirmed via index [23])
30. "Comparing quantiles at scale in online A/B-testing", Spotify Engineering, 2022. https://engineering.atspotify.com/2022/03/comparing-quantiles-at-scale-in-online-a-b-testing (confirmed via index [23])
31. "Key Challenges with Quasi Experiments at Netflix", Netflix Tech Blog, 2020. https://netflixtechblog.com/key-challenges-with-quasi-experiments-at-netflix-89b4f234b852 (confirmed via index [23])
32. "Universal Holdout Groups at Disney Streaming", Disney Streaming (Medium), 2021. https://medium.com/disney-streaming/universal-holdout-groups-at-disney-streaming-2043360def4f (confirmed via index [23])
33. "Analyzing Experiment Outcomes: Beyond Average Treatment Effects", Uber Engineering, 2018. https://eng.uber.com/analyzing-experiment-outcomes/ (confirmed via index [23])
34. "Multi-Armed Bandits and the Stitch Fix Experimentation Platform", Stitch Fix, 2020. https://multithreaded.stitchfix.com/blog/2020/08/05/bandits/ (confirmed via index [23])
35. "Interpreting A/B Test Results: False Positives and Statistical Significance", Netflix Tech Blog, 2021. https://netflixtechblog.com/interpreting-a-b-test-results-false-positives-and-statistical-significance-c1522d0db27a (confirmed via index [23])
36. "Reliable and Scalable Feature Toggles and A/B Testing SDK at Grab", Grab Engineering, 2018. https://engineering.grab.com/feature-toggles-ab-testing (confirmed via index [23])
37. "Zipline: Airbnb's Machine Learning Data Management Platform" (talk), Airbnb, 2018. https://www.youtube.com/watch?v=Tg5VEMEsC-0 (confirmed via index [23])
38. "Machine Learning-Powered Search Ranking of Airbnb Experiences", Airbnb Engineering (Medium), 2019. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789 (confirmed via index [23][24])
39. "Building Airbnb Categories with ML and Human-in-the-Loop", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb (confirmed via index [23])
40. "How AI Text Generation Models Are Reshaping Customer Support at Airbnb", Airbnb Engineering (Medium), 2022. https://medium.com/airbnb-engineering/how-ai-text-gen

eration-models-are-reshaping-customer-support-at-airbnb-a851db0b4fa3 (confirmed via index [24])

05. ML Fundamentals and Modeling Depth: Ranking, GBDT vs. NN, Embeddings, Calibration, Features, Offline/Online Gap

> **Research note (read first).** Adversarially reviewed 2026-10-14. The shared web-search budget was used up, so no new search queries could run. The proxy could not resolve arxiv.org, dl.acm.org, medium.com, prachub.com, teamblind.com or trueinterview.io. GitHub (pages and raw files) was the only reachable host. **Fetched and read on review:** the TrueInterview-derived Airbnb MLE guide (3 candidate reports, mainly 2026 Q1-Q2) [1]; the dated Airbnb question index [2]; the Airbnb company interview-process page [3]; and about 100 posts in a GitHub mirror of 1point3acres Airbnb interview posts. Only two of those posts contain ML rounds [4] [5]. The other posts were checked and contain no ML content.
>
> **How references were checked.** arXiv, ACM and Medium could not be opened directly. Airbnb paper and blog URLs marked "(URL confirmed)" were checked against the curated applied-ML index [30], which lists the same titles and URLs. URLs marked "(not verified)" are cited from prior knowledge or from Chapter 09. Check those before relying on them. The PracHub report [29] could not be opened. Its topics are taken from Chapter 09's summary and are tagged [Likely], not [Reported].
>
> **Tags.** [Reported] means a fetched source names the prompt, the follow-up or the topic for an Airbnb MLE interview. [Likely] means an inference from the Senior MLE role, the reported round structure and Airbnb's published systems. A citation next to a [Likely] tag points to the paper or post behind the topic. It does **not** mean anyone was asked the question.

Format notes

- **There is no reported standalone "ML theory" round.** Reported MLE loops have a phone screen, one coding round, one or two ML system design rounds (sometimes an extra one), an experience deep dive, core values and a hiring-manager conversation [1][3]. Fundamentals come up as depth probes inside the design rounds and the deep dive. The design rounds are "conversations without code", and "the first minutes of scoping are graded" [1].
- **The design rounds decide the loop.** Rejections in the reports were attributed to design depth, not to coding. A borderline signal led to an extra design round instead of a pass [1]. One ML rejection was blamed on "jumping straight to model ideas"; the bar is "problem and label before model" [3].

- **Expected skeleton:** decision supported → label → data → features → model → offline and online metrics → serving → experiment [1]. Interviewers assess problem framing, label definition and time window, concrete marketplace features (reviews, host responsiveness, booking completion, photos, location encodings), the link from offline to online metrics, and production behaviour (candidate generation, latency, freshness, cold start for new listings, monitoring) [1].
- **Interviewers steer hard.** Reported examples: a lifetime-value round defined the label and then moved from regression to bucketized classification; a relisting round went into geohashing; search-ranking rounds pushed on latency, freshness and experimentation [1]. Treat a redirection as a hint and restate the narrower scope.
- **Reported prompts these probes hang off:** search ranking end to end, ranking without a text query, personalized search ranking, user-embedding semantic search, listing LTV, relisting detection, cover-photo selection, high-quality listings, recommendations, and GenAI support retrieval and ranking [1][2]. 1point3acres loops add customer LTV and host pricing [4], plus listing recommendation and price prediction [5].
- **Senior (G9) bar:** design carries more weight at Senior and above. Scope is scored separately from modeling difficulty, and a deep project with narrow reach "reads weaker" than a simpler one with wide impact [1]. Expect a "why" on every choice: label, GBDT vs. NN, metric, and what should move online. Practise each answer in 2-4 minutes, tied to an Airbnb surface.
- **Role context:** Airbnb MLE postings stress training-serving skew, A/B testing, and feature and model selection. Shared infrastructure includes Chronon, Airbnb's open-source feature platform for consistent online and offline features [1][25].
- **Public Airbnb context interviewers know well:** search moved from GBDT to deep NNs [6][7]; in-session listing embeddings [8]; Experiences ranking [9]; diversity in ranking [10][21]; multi-task journey ranking [11]; a customized pricing regression loss [12]; interleaving [13]; embedding-based retrieval [14].

Questions

A. Problem framing, labels and targets

1. You are asked to estimate the lifetime value of a listing (property). Would you frame it as regression or as bucketized classification, and why? [Reported] [1][2]
    - The interviewer pushes you toward bucketized classification. What do you gain and lose? [Reported] [1]
    - How do you choose bucket boundaries, and how do you turn bucket probabilities back into an expected value? Would you use an ordinal loss or a plain softmax?
    - How do you handle the heavy right tail and listings that churn off the platform (censoring)?
2. Before modeling LTV, agree on what "lifetime value" means here. Over what time window is the label defined, and how does that choice trade label delay against usefulness? [Reported] [1]
    - Which business decision does the LTV model support (host acquisition spend, host coaching, ranking), and how does that change the target and the metric? [Reported] [1]

3. Design a guest (customer) lifetime-value model. How do the unit of prediction, the label and the features differ from listing LTV? [Reported] [4]
    - How would you validate an LTV model whose label only matures a year after prediction? [Likely] [18]
4. How would you identify "high-quality listings"? What is your label, given that no ground truth exists? [Reported] [1]
    - Which proxies (reviews, rebook rate, cancellations, CS tickets, host responsiveness) would you combine? How do you avoid circularity with the ranking model's own exposure?
    - How would you use human raters to validate the proxy label, and how many ratings do you need? [Likely] [17]
5. Choose a listing's cover photo to lift bookings. Is this a supervised problem, a ranking problem or a bandit problem? [Reported] [1]
    - What label would you train on, given that the cover photo you showed also caused the clicks?
    - How do you handle cold start for a new listing with no engagement on any photo? [Reported] [1]
    - When would you pick a contextual bandit over a supervised model plus an A/B test? [Likely]
6. Detect relisting (a removed or banned listing reappears, possibly under a new account). What features and model family would you use? [Reported] [1][2]
    - Is this classification, similarity search or entity resolution? Defend the framing.
    - How does geohashing help you generate candidate pairs, and how does geohash precision trade recall against pair volume? [Reported] [1]
    - How would the model's outputs show up in online metrics, given that confirmed relistings arrive late and only for flagged cases? [Reported] [1]
    - How do you choose an operating threshold when false positives hurt legitimate hosts?
7. For search ranking, what is the training label: click, long click, wishlist, booking request, accepted booking or completed stay without cancellation? [Likely] [6][11]
    - How do you weight these in a single objective vs. a multi-task model?
    - How do you deal with label delay (booking days after search)?
8. Rank home search results when the user gives no text query (only location and dates, or nothing). What does "relevance" mean here, and what is the label? [Reported] [2]
    - With no query, popular listings dominate the logs. How do you avoid a popularity feedback loop and still give new listings exposure?
9. A home page combines search, calendar availability and ranking. How do availability and date constraints enter the model: as hard filters, as features, or through the label (unavailable listings as negatives)? [Reported] [2]
10. How would you define a negative example in Airbnb search: an impressed and not clicked listing, a clicked and not booked listing, or a random listing? What bias does each choice introduce? [Likely] [6][8]
11. A booking requires host acceptance. How does that change your label definition and your model? [Likely] [6]
12. You have far fewer bookings than clicks. How does label sparsity drive your choice of objective, model capacity and evaluation? [Likely] [6]

B. Learning to rank

13. Compare pointwise, pairwise and listwise learning-to-rank objectives. Which would you start with for Airbnb search, and why? [Likely] [6]
    - Airbnb's search team reported success with a pairwise formulation. Why might pairwise suit a booking-or-not label inside a search?
14. How do you build training pairs or lists from search logs (the booked listing vs. the other listings impressed in the same search)? [Likely] [6]
    - How many negatives per positive, and does sampling change the metric you are optimizing?
15. Explain how LambdaRank / LambdaMART connects a pairwise loss to NDCG. Why can't you optimize NDCG directly with gradient descent? [Likely]
16. Which offline metric would you use for search ranking (NDCG on the booked listing, MRR, recall@k, AUC), and why? Why might AUC improve while NDCG is flat? [Likely] [6]
17. How do you handle position bias in logged search data? [Likely] [7]
    - Compare position as a feature (with dropout, or set to a constant at inference), inverse propensity weighting and randomization-based estimates.
    - How would you estimate position propensities at Airbnb without hurting users?
18. Your ranker learns that cheap listings get booked. Why does pointwise use of price fail, and how would you encode price relative to the user's query and market? [Likely] [7]
19. A ranker trained on bookings learns the majority guest's preference and pushes one listing segment to the top for everyone. Why does that happen, and how do you correct it? [Likely] [21]
20. How do you rank diversely (avoid ten near-identical listings) without destroying relevance? [Likely] [10][21]
    - Is diversity a re-ranking step, a loss term or a list-aware model? What are the trade-offs?
    - How would you measure whether diversity helped bookings?
21. How would you design a two-stage system (retrieval + ranking) for Airbnb search, and what does each stage optimize? [Reported] [1]
    - How do you stop the ranker from never seeing listings that retrieval filters out (selection bias)? [Likely] [14]
22. Airbnb search is a multi-step journey (search → click → listing page → request → booking). How would you model it with multi-task learning? [Likely] [11]
    - How do you combine the task heads into a single ranking score?
    - How do you handle tasks with very different base rates and loss scales?
23. You have several ranking objectives (bookings, price competitiveness, host quality). Compare a weighted multi-objective loss with distilling the blended objective into one model. [Likely] [27]
24. How would you incorporate host-side objectives (acceptance likelihood, host preferences, fair exposure) into the ranking score? [Likely] [6]
25. Personalized search ranking: which user signals would you use, and how do you avoid overfitting to a user's last click? [Reported] [2]
26. How would you rank Airbnb Experiences differently from Stays, given much less data per item? [Likely] [9]
27. Listings are also shown as pins on a map, where there is no clear top-to-bottom order. How do your ranking metric and position-bias assumptions change? [Likely] [26]

C. GBDT vs. neural networks

28. When would you choose GBDT over a deep NN for a tabular ranking problem, and vice versa? [Likely] [6]
    - Airbnb moved search from GBDT to a NN and initially saw little gain. Why does that happen, and what made the NN eventually win?
29. Compare a factorization machine with an embedding + DNN ranker for an Airbnb ranking problem. Which would you ship, and why? [Likely] [29]
30. A NN on the same features as your GBDT performs worse. What do you check first: normalization, distribution shape, learning rate, capacity or label noise? How do you tell which one it is? [Likely] [6]
31. GBDTs are invariant to monotone feature transforms; NNs are not. What normalization would you apply to price, review count and distance before a NN, and why might a log or Gaussian-rank transform matter for the NN but not the trees? [Likely] [6]
32. How would you use a GBDT and a NN together (GBDT leaves as NN features, stacking or distillation)? [Likely]
33. Why might listing-ID embeddings overfit in Airbnb search, when the same idea works for e-commerce item IDs? What about the booking-per-listing distribution makes it hard? [Likely] [6]
34. Deep models are harder to interpret. How do you explain to a PM or a host why a listing ranked lower? [Likely] [6]
    - Compare SHAP, partial dependence plots and ablation-based importance. Which are trustworthy for a NN?
35. Your NN is more accurate offline but has 3x the inference latency, and the interviewer pushes on the latency budget. How do you decide, and what closes the gap (distillation, caching, fewer candidates, smaller model)? [Reported] [1]
36. How would you trade off precision, recall, latency and infrastructure cost when choosing a model for production ranking? [Likely] [29]
37. How would you enforce a monotonicity requirement (for example, all else equal, a higher price must not raise the score)? [Likely] [7]
38. You need to classify listings using text, photos and tabular attributes together. How would you design a multimodal model, and when is it worth it over a GBDT on hand-built features? [Likely] [22]

D. Embeddings and representation learning

39. How would you learn listing embeddings from user click sessions? [Likely] [8]
    - Why treat a session as a "sentence" and use skip-gram with negative sampling?
    - What is the effect of using the booked listing as global context?
    - Why sample negatives from the same market, and what goes wrong if you sample uniformly?
40. How do you produce an embedding for a brand-new listing with no sessions (cold start)? What are the failure modes of averaging nearby similar listings? [Likely] [8]
41. Bookings are too sparse to learn per-user embeddings. How would you learn user-type and listing-type embeddings in the same space? [Likely] [8]
42. How do you evaluate listing embeddings offline before they reach production? Which intrinsic checks (geography, price, style) and which extrinsic checks would you run?

[Likely] [8]

43. Design listing recommendations (similar listings, or listings for a guest). How would you evaluate an embedding-based recommender offline and online? [Reported] [1][5]
44. Use embeddings for real-time personalization: how do you turn a user's in-session clicks and skips into ranking features, and how do you weight recent vs. older signals? [Likely] [8]
45. Design user embeddings for semantic search (free-text queries such as "cabin with a hot tub near a lake"). [Reported] [2]
    - How would you align queries and listings in one embedding space?
    - Which training pairs and hard negatives would you use?
46. How would you build embedding-based retrieval for Airbnb search and combine it with hard filters (dates, price, map bounds, guests)? Pre-filter vs. post-filter with ANN: what are the recall and latency trade-offs? [Likely] [14]
47. Two-tower vs. cross-encoder: where in the stack would you use each, and why? [Likely] [14]
48. Prices, photos and availability change daily. How do you keep embeddings and features fresh, and how do you measure what staleness costs you? [Reported] [1]
    - What breaks downstream when you retrain the embedding model and the space rotates? [Likely]
49. Spatial features: compare geohash buckets, learned location embeddings and raw latitude/longitude. Why can raw lat/long be a poor NN input, and how does geohash granularity affect generalization? [Reported] [1][3]
50. How would you use image embeddings from listing photos (room type, amenity detection, photo quality) as ranking or classification features? [Likely] [15][16]
51. When does a pre-trained LLM or foundation-model embedding beat a domain-trained listing embedding, and when does it not? [Likely]

E. Calibration and probability quality

52. What is calibration, and why does it matter if you only use the score to rank? Name Airbnb consumers that need a calibrated probability (combining multi-task heads, expected-value LTV, trust thresholds). [Likely]
53. How do you measure calibration (reliability diagram, ECE, log loss vs. Brier score), and how do Platt scaling, isotonic regression and temperature scaling each fail? [Likely]
54. You down-sample negatives 10:1 for training. How do you correct the predicted probabilities? [Likely] [6]
55. Your model is well calibrated overall but miscalibrated for new listings and small markets. How do you detect and fix that? [Likely]
56. You combine pClick x pBook|click x pAccept into one ranking score. What happens if one head is miscalibrated? [Likely] [11]
57. Calibration drifts after a ranking change, because the model now trains on its own exposure. How do you monitor and recalibrate? [Likely]
58. In the bucketized LTV model, how do you check that the bucket probabilities are calibrated enough to compute an expected value? [Likely] [1]

F. Feature engineering

59. Name the most important feature families for Airbnb search ranking and justify each. Which are query-dependent (dates, guest count, location) and which are listing-only? [Likely] [6]
60. Which concrete marketplace signals (reviews, host responsiveness, booking completion, photos) would you turn into features for listing quality or LTV, and how? [Reported] [1]
61. How would you build a feature for "price relative to the market for these dates and this listing type"? [Likely] [7][12]
62. How do you create features from reviews (count, average, recency, text) and historical CTR or booking rate without leaking future information? How do you guarantee point-in-time correctness in the training pipeline? [Likely] [25]
63. How would you compute per-guest cumulative booking counts and first and last booking times as point-in-time-correct model features? [Likely] [2]
64. How do you handle new listings whose historical features are missing: imputation, a missingness indicator or a separate model? How would you estimate their engagement? [Likely] [7]
65. How do you handle high-cardinality categorical features (city, host ID, amenity sets) in a GBDT vs. a NN? [Likely]
66. Which features would you compute in real time (session clicks, current filters), and which in batch? What does each choice cost? [Likely] [8]
67. How do you detect and prevent training-serving skew (a feature computed differently online than offline)? What does a shared feature platform such as Chronon solve, and what does it not? [Likely] [1][25]
68. How do you decide whether a new feature is worth its engineering and latency cost? What ablation and online test would you run? [Likely]
69. Seasonality and lead time strongly affect demand. How would you encode check-in date, lead time and length of stay? [Likely] [12]
70. How do you handle features that the ranking model itself influences (for example, a listing's CTR depends on how highly it was ranked)? [Likely] [7]

G. Offline evaluation and the offline/online gap

71. How do you translate an offline metric (NDCG, AUC) into an expected online metric (bookings, revenue), and which A/B test would confirm it? [Reported] [1]
    - Have you validated the correlation between your offline metric and online lift across past launches?
72. Offline NDCG improved, but the online A/B test is flat or negative. List the hypotheses you would investigate, in order: training-serving skew, logging bias, metric mismatch, novelty, low power, interference between listings. How do you rule each out? [Likely] [6][29]
73. Why can a model trained on logs from the old ranker look good offline but fail online? How would off-policy evaluation (IPS, doubly robust) help, and what assumptions does it need? [Likely] [7][28]
74. When would you use interleaving instead of an A/B test to compare two rankers? [Likely] [13]
    - What does interleaving measure well, and what can it not tell you (revenue, long-term effects)?
    - How do you validate that interleaving outcomes agree with A/B outcomes?

75. How would you combine counterfactual (logged-data) evaluation with interleaving to filter ranking candidates before a full A/B test? [Likely] [28]
76. How do you build an offline evaluation set that represents the online traffic mix (markets, devices, new vs. returning users)? [Likely]
77. Your model wins in aggregate but loses in some markets or segments. How do you decide whether to launch? [Likely]
78. How do you guard against overfitting to the offline evaluation set after many iterations? [Likely]
79. Which primary metric should a ranking launch be judged on, and which guardrail metrics would block it? How do you make both robust to noise and gaming? [Reported] [2]
80. How would you set up a long-term holdout for ranking changes, and what does it show that a two-week A/B test cannot? [Likely]
81. What metric would you use when the goal is "a great stay", not just a booking (cancellations, reviews, rebookings)? [Likely] [11]
82. Airbnb converts short-term actions into a predicted future value so that teams can compare trade-offs. How would you build such a model, and how would you check it before using it as an experiment metric? [Likely] [19]
83. Once a model ships, which signals would you monitor to catch degradation before bookings drop? [Reported] [1]

H. Training, generalization and debugging

84. How would you detect and fix overfitting in a deep ranking model trained on one year of search logs? [Likely] [6]
85. How do you split data for time-dependent problems (random vs. temporal vs. by user/listing), and why does it matter at Airbnb? [Likely]
86. How do you handle class imbalance (bookings much rarer than impressions): re-weighting, sampling or focal loss? Which of these break calibration? [Likely]
87. How often would you retrain the ranker, and how do you decide (fixed schedule vs. drift-triggered)? [Likely]
88. Model quality dropped after a holiday season. How do you tell concept drift from a data-pipeline bug? [Likely]
89. How do you make training reproducible enough that a 0.5% offline gain is real and not noise? How many seeds or bootstrap resamples would you run? [Likely]
90. How do you choose the loss for a pricing or demand model where under- and over-prediction have asymmetric costs? [Likely] [12]

I. Regression, forecasting and pricing-flavoured modeling

91. Design a model that helps hosts set a price for each future date, using market demand. What exactly is the target: the booked price, the listed price, or a demand curve? [Reported] [4][5]
92. Predict the nightly price for a brand-new listing with no booking history. What features and model would you use, and how do you evaluate it? [Reported] [5]
93. Airbnb proposed a customized regression loss for price suggestions because true "optimal price" labels do not exist. How would you learn a price when you only see booked or not booked at a set price? [Likely] [12]

94. How would you estimate demand elasticity when price is confounded with listing quality and seasonality? [Likely] [12]
95. How would you evaluate a price-suggestion model offline when hosts can ignore the suggestion? [Likely] [12]
96. How would you forecast occupancy or demand for a listing 90 days out, and which uncertainty estimates would you provide? When would you use quantile regression instead of mean regression for host-facing estimates? [Likely]
97. Airbnb built an ML pipeline to predict the value of homes. How would you choose among candidate models for such a value model, and how would you check that it is fair across markets? [Likely] [18]

J. Classification, trust and quality models

98. Design the label and model for detecting fake or fraudulent listings, when positives are rare and arrive late. How do you handle labels that only exist for cases an earlier model or rule flagged (selection bias)? [Likely]
99. Predict party risk at reservation time. The label (a reported party) is rare and noisy. How would you build it, and which features would you refuse to use for fairness reasons? [Likely] [1]
100. A property-damage risk score feeds a protection or screening decision. How do asymmetric error costs shape the threshold, and why does calibration matter here? [Likely] [1]
101. Build a classifier that flags "family friendly" listings. Where do labels come from (weak supervision, rules, raters), and how do you measure precision on a long tail of listing types? [Likely] [29]
102. How do you evaluate a classifier when precision matters far more than recall (for example, removing a host's listing)? [Likely]
103. How would you use text (descriptions, messages) in a classifier: TF-IDF + GBDT, a fine-tuned transformer, or LLM embeddings? [Likely] [22]
104. You must classify guest-host messages by intent, but no taxonomy exists yet. How would you discover the intent classes and then train a supervised classifier? [Likely] [23]
105. How would you build an active-learning or human-in-the-loop process to label listing categories or quality? [Likely] [17]
106. How would you check a ranking or trust model for unfair outcomes across host or guest groups? [Likely]

K. Retrieval and LLM modeling for support (recent senior loops)

107. Recent senior loops asked ranking and retrieval questions about generative-AI support tools. How would you evaluate a retriever that finds the right policy passages for a support case (recall@k, nDCG on labeled pairs, LLM-as-judge)? [Reported] [1]
108. Generating customer-support instructions from company policies: when would you fine-tune an LLM, and when would retrieval-augmented generation be enough? How would you measure hallucination and policy compliance? [Reported] [1][2]
109. How would you rerank retrieved support documents, and what labels would you train the reranker on? [Likely] [1]

L. Experimentation-adjacent modeling (role focus)

110. Your new ranker is evaluated by a user-randomized A/B test. Why can supply competition between listings bias the estimate, and how does that affect how you read a model's lift? [Likely] [13]
111. How would you use a model-based surrogate metric (predicted booking value) to read experiments faster? What validation does the surrogate need? [Likely] [19]
112. How would you use pre-experiment covariates (CUPED or regression adjustment) to reduce variance, and which model features make good covariates? [Likely]
113. You cannot randomize a host-facing model (for example, a pricing tool hosts opt into). How would you estimate its causal effect, and how could an ML-based counterfactual help? [Reported] [2] - What are the failure modes of ML-predicted counterfactuals compared with a randomized control? [Likely] [20]
114. Heterogeneous treatment effects: how would you model which users or markets benefit from a ranking change (uplift models, causal forests)? [Likely]
115. If you can run only one experiment this quarter, how do you choose among three model improvements using offline evidence? [Likely]
116. Which guardrail metrics would you set for launching a new ranking model, and how do you avoid guardrails blocking every launch through false alarms? [Likely] [24]

M. Senior-level depth probes (deep dive and design rounds)

117. In the ML project you are walking us through, why did you choose this model, feature set or serving path over the alternatives? [Reported] [1]
118. In your most impactful model, which single modeling decision mattered most, and how do you know? [Likely] [1]
119. Tell me about a time a simpler model (heuristic, logistic regression, GBDT) beat a more complex one in production. Why did it win? [Likely]
120. What did you choose *not* to model, and why? [Likely]
121. How did you choose your offline metric, and did it ever mislead you? [Likely] [1]
122. Given twice the data, twice the features or twice the model capacity, which would you take for Airbnb search, and why? [Likely]
123. The interviewer redirects you mid-design (for example, from regression to classification). How do you restate the narrower scope and adapt your modeling plan? [Reported] [1]
124. How do you explain a ranking model's trade-offs to a PM who only cares about bookings? [Likely]

References

[1]-[5], [25] and [30] were fetched and read on 2026-10-14. URLs marked "(URL confirmed)" match the title and URL listed in [30]. They could not be opened directly because arxiv.org and medium.com were unreachable. URLs marked "(not verified)" could not be checked in this session.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, CC BY 4.0, built from 3 candidate reports, mainly 2026 Q1-Q2: LTV regression to bucketized steering, relisting and geohashing, cover photo with cold start, high-quality listings,

latency/freshness pushback, offline-to-online link, GenAI support retrieval and ranking, monitoring, Chronon and training-serving skew in postings). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md

2. "Airbnb" question index with report dates ("Listing Lifetime Value — Estimation", May 2026; "Design User Embedding Semantic Search", Apr 2026; "Rank Home Search Results Without a Text Query", Sep 2026; "Home Page — Search + Availability + Ranking", Mar 2026; "Model Cumulative Booking Metrics and First and Last Booking Times", Sep 2026; "Design A Personalized Search Ranking System"; "Design Airbnb Relisting Detection"; "Design robust primary and guardrail metrics"; "Design an A/B test with causal inference"; "Generate Customer-Support Instructions from Company Policies"). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
3. "Airbnb interview process" (MLE candidates get one or two ML design rounds on search ranking, property LTV, recommendations, or relisting detection with geohashing; "Problem and label before model"). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. 1point3acres post #40 (English mirror): project deep dive; coding LC 1298; ML design rounds on customer lifetime value and on helping hosts set prices. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
5. 1point3acres post #79 (English mirror): ML design rounds on listing recommendation and price prediction; ConnectN coding; an ML experience round that turned into an unplanned ML design question. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
6. Haldar et al., "Applying Deep Learning To Airbnb Search," arXiv 1810.09591 (KDD 2019). (URL confirmed) https://arxiv.org/abs/1810.09591
7. Haldar et al., "Improving Deep Learning For Airbnb Search," arXiv 2002.05515 (KDD 2020). (URL confirmed) https://arxiv.org/abs/2002.05515
8. Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb," KDD 2018, and the companion blog post "Listing Embeddings for Similar Listing Recommendations and Real-time Personalization in Search." (URLs confirmed) https://www.kdd.org/kdd2018/accepted-papers/view/real-time-personalization-using-embeddings-for-search-ranking-at-airbnb ; https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
9. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences," Airbnb Tech Blog, 2019. (URL confirmed) https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
10. "Learning To Rank Diversely," Airbnb Tech Blog, 2022. (URL confirmed) https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621 . Paper version: Abdool, Haldar et al., "Learning To Rank Diversely At Airbnb," arXiv 2210.07774. (not verified) https://arxiv.org/abs/2210.07774
11. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning," arXiv 2305.18431 (KDD 2023). (not verified) https://arxiv.org/abs/2305.18431

12. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing," KDD 2018. (not verified) https://dl.acm.org/doi/10.1145/3219819.3219830
13. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving," Airbnb Tech Blog, 2022. (URL confirmed) https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
14. "Embedding-Based Retrieval for Airbnb Search," Airbnb Tech Blog, and "Applying Embedding-Based Retrieval to Airbnb Search," arXiv 2601.06873, 2026. (not verified; taken from Chapter 09) https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/ ; https://arxiv.org/abs/2601.06873
15. "Categorizing Listing Photos at Airbnb," Airbnb Tech Blog, 2018. (URL confirmed) https://medium.com/airbnb-engineering/categorizing-listing-photos-at-airbnb-f9483f3ab7e3
16. "Amenity Detection and Beyond — New Frontiers of Computer Vision at Airbnb," Airbnb Tech Blog, 2019. (URL confirmed) https://medium.com/airbnb-engineering/amenity-detection-and-beyond-new-frontiers-of-computer-vision-at-airbnb-144a4441b72e
17. "Building Airbnb Categories with ML and Human-in-the-Loop," Airbnb Tech Blog, 2022. (URL confirmed) https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb
18. "Using Machine Learning to Predict Value of Homes On Airbnb," Airbnb Tech Blog, 2017. (URL confirmed) https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d
19. "How Airbnb Measures Future Value to Standardize Tradeoffs," Airbnb Tech Blog, 2021. (URL confirmed) https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5
20. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb," Airbnb Tech Blog, 2022. (URL confirmed) https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512
21. Abdool et al., "Managing Diversity in Airbnb Search," arXiv 2004.02621, 2020. (URL confirmed) https://arxiv.org/abs/2004.02621
22. "WIDeText: A Multimodal Deep Learning Framework," Airbnb Tech Blog, 2020. (URL confirmed) https://medium.com/airbnb-engineering/widetext-a-multimodal-deep-learning-framework-31ce2565880c
23. "Discovering and Classifying In-app Message Intent at Airbnb," Airbnb Tech Blog, 2019. (URL confirmed) https://medium.com/airbnb-engineering/discovering-and-classifying-in-app-message-intent-at-airbnb-6a55f5400a0c
24. "Designing Experimentation Guardrails," Airbnb Tech Blog, 2021. (URL confirmed) https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669
25. "Chronon: a data platform for AI/ML" (Airbnb's open-source feature platform), airbnb/chronon, GitHub. Fetched 2026-10-14. https://github.com/airbnb/chronon
26. Haldar et al., "Learning to Rank for Maps at Airbnb," arXiv 2407.00091 (KDD 2024). (not verified; taken from Chapter 09) https://arxiv.org/abs/2407.00091
27. Tang, Gao, He and Katariya, "Multi-objective Learning to Rank by Model Distillation," arXiv 2407.07181 (KDD 2024). (not verified; taken from Chapter 09) https://arxiv.org/abs/2407.07181

28. Zhang et al., "Harnessing the Power of Interleaving and Counterfactual Evaluation for Airbnb Search Ranking," arXiv 2508.00751, 2025. (not verified; taken from Chapter 09) https://arxiv.org/abs/2508.00751
29. "Airbnb Senior Machine Learning Engineer Interview Experience — Four ML System Design Rounds, Rejected on Calibration," PracHub, 2026. Not reachable in this session. The topics cited here (FM vs. embedding + DNN; precision/recall/latency/cost trade-offs; offline NDCG up but A/B flat; a "family friendly" classifier) come from Chapter 09's summary. "Calibration" in the title may refer to level calibration, not probability calibration. https://prachub.com/interview-experiences/airbnb-senior-machine-learning-engineer-interview-experience-four-ml-system-design-rounds-rejected-on-calibration
30. Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production" (used to confirm Airbnb paper and blog titles and URLs). eugeneyan/applied-ml, GitHub. Fetched 2026-10-14. https://github.com/eugeneyan/applied-ml

06. General Backend System Design: Booking, Availability, Search, Messaging, Payments and Feature Store

> **Research note.** This chapter was re-checked on 2026-10-14. Only GitHub-hosted sources could be reached; Glassdoor, Blind, LeetCode, Medium, Hello Interview, IGotAnOffer, PracHub, TrueInterview and the Airbnb Tech Blog did not resolve, and the web-search budget was used up. The chapter rests on aggregated 2025 to 2026 candidate reports on GitHub [1][2][3][4], a community prep guide [5], a crowd-sourced question list [6], the Tech Interview Handbook [7], Airbnb's open-source Chronon repository [8], and English and Chinese 1point3acres interview reports mirrored on GitHub [11]-[45]. A question is tagged [Reported] only when a cited source reports it, or a very close variant, for Airbnb. All other questions are [Likely]: inferences from Airbnb's products and the senior bar those sources describe. Most mirrored 1point3acres posts carry no date. Where a date is given, it is 2025 or 2026 except [21] (2023).

Format notes

- **Rounds and length.** Mid-level loops usually have one design round and senior loops two, each about an hour of discussion with no code. Senior loops can also move system design into the phone stage. A design round is sometimes added after the onsite when the signal is borderline [1]. Several candidates describe that extra round, for example a classic booking prompt focused on preventing double booking [25] [1]. The Tech Interview Handbook lists one "System Design/architecture" round in the general onsite [7]. One 2026 report for the 1-to-3-year level (G8) describes a system design screen plus a second system design round at the onsite [5].
- **Interviewer and track.** MLE candidates usually get one or two ML system design rounds instead (see chapter 03) [2][4]. Generic backend prompts still appear at senior level [1][3], and staff loops have included both a distributed job scheduler and Airbnb booking [29].
- **Senior bar.** Senior loops "expect depth one candidate called staff level" [1]. Several rejected senior candidates were told their design round fell short of the senior bar [22][32][21].
- **Common failures.** Running out of time while trying to cover everything (an in-memory SQL database prompt) [1]. "Chased perfection on the chat schema instead of aligning on trade-offs" [1]. Needing too many hints on API design [38]. Talking about scaling when the interviewer wanted to go deep on APIs and database design [16].
- **What gets probed.** Schemas and indexes that make the stated queries cheap (top-N chats, time and location filters); double booking under contention, idempotent refunds

and delivery guarantees [1]. Interviewers often treat the supporting services (pricing, search, notifications) as given and focus on the data schema and one or two core flows [19].

- **What this means for an MLE.** Expect a classic marketplace prompt (booking, waitlist, chat, notifications, ticketing, job scheduler). Expect the interviewer to probe whatever your background makes natural: feature stores, logging pipelines and experiment assignment.

Questions

A. Booking and reservation system

1. Design Airbnb's booking system: guests search for available listings and book them, and the system must never double-book a listing. [Reported] [1][3][5][25][27][29]
    - What functional requirements would you commit to in the first few minutes, and what would you explicitly leave out?
    - Is a booking request a strongly consistent write? Which parts of the flow can be eventually consistent?
    - How do you guarantee two guests cannot book overlapping dates on the same listing at the same moment?
    - Would you use pessimistic row locks, optimistic concurrency with version numbers, or a uniqueness constraint on (listing, night)? Why?
    - How do you paginate and index availability search results? [1]
    - Follow-up: a big local event creates a hot partition. Which keys get hot, and how do you shed or queue load without letting anyone book a listing that is already taken? [5]
2. Design a hotel booking system, with rooms that have inventory counts rather than unique homes. [Reported] [3]
    - How does inventory-count booking differ from Airbnb's one-unit-per-listing model?
    - How do you handle overbooking policies, if any are allowed?
3. Walk through the state machine of a reservation: request, pending host approval, accepted, paid, checked in, completed, cancelled or altered. [Likely]
    - Where do you persist state transitions, and how do you make them auditable?
    - How do you expire a booking request the host never answers within 24 hours?
4. How would you implement a temporary hold on dates while the guest is on the checkout page? [Likely]
    - How long should the hold last, and what releases it?
    - What happens if the payment succeeds after the hold has expired?
5. Design Instant Book and Request to Book. How do the two flows differ in consistency and latency requirements? [Likely]
6. How do you handle reservation alterations, such as changing the dates or guest count on an existing booking? [Likely]
    - How do you avoid releasing the old dates before the new dates are secured?
7. Design the cancellation flow, including refunds that depend on the cancellation policy and the time left before check-in. [Likely]

8. Design a waitlist system for a hot listing, with APIs to join, check position and leave, and notifications when spots open under high contention. [Reported] [1][3][20]
    - How do you notify the next person fairly, with a deadline to claim?
    - How do you avoid a thundering herd when a slot opens?
9. Design a listing booking system in which users can join a waitlist when a listing is booked, and the first 20 waitlisted users are notified when it becomes available. Pricing, search and notification services already exist. [Reported] [19]
    - What is your data schema for bookings and waitlist entries?
    - How do you guarantee that exactly the first 20 users are notified once, even if the release event is delivered twice?
10. How would you shard the reservations database as Airbnb grows across regions? [Likely]
    - Would you shard by listing, by guest or by host? Which queries become cross-shard?
11. How do you make the "Reserve" API idempotent when the mobile client retries on a flaky network? [Likely] [9]
12. Booking needs data from several services: listing, pricing, availability, payments and trust. How do you coordinate the transaction across them? [Likely] [9]
    - Would you use a saga, two-phase commit or an orchestrator? What are the compensating actions?
13. Design a flight booking system that supports connecting flights while prices change. [Reported] [13]
    - How do you search for itineraries with connections (graph search, precomputed routes)?
    - Under a partition, and in normal operation, do you favor availability or consistency for prices and for seats (PA versus PC)? Agree on this before you design.

B. Availability calendar

14. Design the availability calendar service for millions of listings, in which hosts block dates and set minimum and maximum nights and per-night prices. [Likely] [1][3]
    - What is the data model: one row per listing-night, interval ranges or bitmaps?
    - What are the read-to-write ratios for search, listing-page views and host edits?
15. Design Google Calendar (events, recurrence, invites, free/busy). [Reported] [3][39]
    - How do you store and expand recurring events efficiently?
    - What schema supports fast queries such as "is everyone free from 2 to 3pm" across many calendars?
16. Hosts sync calendars with other platforms through iCal. How do you import and export them without causing double bookings? [Likely]
    - What polling frequency would you use, and how do you resolve conflicts between an external block and a pending Airbnb request?
17. How do you answer "which listings in Paris are available from June 3 to June 7 for 4 guests" at search time? [Likely] [1]
    - Do you pre-compute availability into the search index, or filter after retrieval?
    - How stale can the search index's availability be? What happens if a user clicks a listing that has just become unavailable?

18. Design the split-stay feature, which offers two listings that together cover a guest's dates when no single listing does. [Likely] [1][25]
    - How do you search pairs of listings without an O(n squared) blow-up?
    - What changes if you allow three-way splits?
19. How do you handle time zones and check-in and check-out boundaries for listings around the world? [Likely]
20. A host edits the calendar while a guest is booking the same dates. Which write wins, and how do you guarantee correctness? [Likely]
21. How would you propagate availability changes to search, pricing and ML ranking features within seconds? [Likely]
    - Would you use change data capture (CDC) to a log, dual writes or an outbox pattern?
22. Walk through the data flow after a user modifies their profile information: which services, caches and indexes must update, and in what order? [Reported] [12]

C. Search and discovery backend

23. Design the Airbnb home page: search, availability and ranking. [Reported] [1][3]
    - What are the latency budgets for retrieval, filtering and ranking?
    - How does the page behave when the ranking service is slow or down?
24. Design ranking for home search results when the user has typed no text query (only location, dates and guests). [Reported] [3]
25. Design a query system over user activity that filters by time window and geography. [Reported] [1][3]
    - What index structures would you use for the geo part (geohash, S2 cells, quadtrees) and for the time part?
    - How do you support queries such as "views in this bounding box in the last 7 days" at scale?
26. Design a map-based search: as the user pans and zooms, return the listings within the viewport. [Likely]
    - How do you avoid hammering the backend on every pan event?
    - How do you cluster pins when thousands of listings fall in the viewport?
27. Design a full-text search system: index building, query processing and integration with the underlying storage. [Reported] [13]
    - How do you optimize query latency and the efficiency of index updates?
    - When would you do a full reindex rather than incremental updates for listings, prices and availability?
28. Design typeahead (autocomplete) search that returns matching results in real time as the user types a prefix, for example for destinations. [Reported] [34]
    - What data structures and storage would you use, and how do you rank and cache prefix results?
29. Design semantic search over user and listing embeddings. [Reported] [3]
    - How do you choose and operate an approximate nearest-neighbor index (HNSW, IVF-PQ) next to structured filters?
    - How do you re-embed and re-index without downtime?
30. A host with more than 100 listings finds the host listings page very slow. Redesign the aggregation from the bottom up. [Reported] [1][3][25]
    - What would you pre-compute, cache or paginate?

    - How do you avoid N+1 calls across service boundaries?
31. Design cursor-based pagination for search results that stays stable when ranking or availability changes between page loads. [Likely] [1][41]
32. How would you cache search results, given that many users query the same location and dates but prices and availability keep changing? [Likely]
33. Design the wishlist service, including shared wishlists among group members. [Likely]

D. Messaging and notifications

34. Design a group chat system (WhatsApp- or Slack-like): create groups, add members, send messages and view history. [Reported] [3][4][5][36][37][45]
    - The interviewer focuses on the database tables. What schema makes the required queries cheap?
    - How do you guarantee per-conversation ordering and exactly-once display?
    - How do you support presence and read receipts? [1]
35. Design group chat with an inbox that shows a user's top 20 chats, ordered by the most recent message or by activity over a configurable window. [Reported] [1][32][33] [34][35]
    - How do you maintain this inbox in real time without scanning every group's messages?
    - How do you notify online and offline users of each new message (long polling, WebSockets, an asynchronous queue)? [35]
    - How do you route messages across many WebSocket servers, and how do you recover when one fails? [32]
36. Design a group-only instant messaging system with groups of 10,000 to 100,000 members. [Reported] [17]
    - Fan-out on write or fan-out on read? How do you avoid write amplification, and how do you shard?
    - Which delivery semantics do you choose, and how do you de-duplicate with message IDs?
    - How do you enforce ordering within a group (partition by group ID, a sequence generator)?
37. Design guest-host messaging on Airbnb: messages tied to a listing or reservation, with read receipts and translation. [Likely]
    - How do you store the conversation so it is linked to the reservation and visible to support agents?
    - Where would you hook in fraud or off-platform-payment detection without adding latency to sending?
38. Design a unified notification platform for guests, hosts and employees, covering event-triggered sends (an email when a guest books, a door lock code by SMS on check-in morning) and batch campaigns (promotions to prospective hosts). [Reported] [1][3][4] [26][40]
    - How do you prioritize transactional notifications over marketing?
    - How do you de-duplicate, rate-limit per user and respect quiet hours across time zones?
    - What happens when a downstream provider (APNs, FCM, an SMS gateway) is slow or failing?

39. How would you design per-user notification preferences and channel fallbacks (push, then email, then SMS)? [Likely]
40. Design a customer-support ticketing system that routes tickets to the right agent queue. [Reported] [1][3][26][27]
    - How do you implement agent claim rules by language and location, and measure first-response time? [1]
    - How do you combine rules-based routing with an ML classifier, and fall back when the model is down?
41. Design the support ticket lifecycle: users file tickets, tickets move through created, pending assignment, assigned and resolved, and users and agents chat inside the ticket. [Reported] [25]
    - The prompt is broad. Which 70 percent would you design first if time ran short?
42. Design a ticket system with a watchlist or dashboard of open tickets and search by status, priority and creation date. [Reported] [21]
43. Design a system that generates customer-support instructions from company policies, for example with an LLM. [Reported] [3]
    - How do you keep generated instructions in sync when a policy changes?
    - How do you evaluate correctness before showing them to agents?
44. Design a system that sends reminder messages (check-in instructions, review requests) at specific times relative to each reservation. [Likely] [26]

E. Payments, payouts and financial correctness

45. Design a payment system for hundreds of millions of daily active users, covering authentication, transaction processing and concurrency. [Reported] [15]
    - What is an idempotency key? Who generates it, and how long do you keep it? [9]
    - How do you separate the local DB transaction from the remote payment-provider call?
46. A payment service runs on two databases and can no longer handle the traffic. How do you add sharding? [Reported] [14]
    - How do you choose the shard key and migrate live data without double-charging or losing writes?
    - How do you handle cross-shard transfers and reporting queries after the split?
47. Design a digital wallet service (Google Wallet-style): stored balances, top-ups, transfers and transaction history. [Reported] [16][18]
    - Expect the interviewer to go deep on the APIs and the database schema rather than on scaling. What tables and constraints keep balances correct?
48. Design host payouts: scheduling, splits among co-hosts, multiple currencies and payout methods. [Likely] [1]
49. Design refunds across multiple payment methods and dates, honoring a priority order. [Likely] [10]
50. How do you make refunds idempotent when the refund request, the provider callback and an agent's manual action can all arrive more than once? [Likely] [1]
51. Design a double-entry ledger for Airbnb's payments platform. [Likely]
    - How do you reconcile your ledger against the payment processor's settlement files?
52. Design a banking-like service with deposit, withdraw, transfer and balance-history queries by time. How would you turn it into a distributed service that handles

concurrent transactions? [Likely] [1][16]

53. Design a currency-conversion and price-display service that shows consistent prices from search through checkout. [Likely]
    - How do you lock an exchange rate for a quote?
54. Design a retryer or retry library for calls to unreliable downstream services. [Likely] [1][19]
    - How do you decide which errors are retryable?
    - How do you combine exponential backoff, jitter, deadlines and circuit breakers?
55. How would you design fraud checks into the payment path without hurting checkout latency? [Likely]

F. Data, logging and analytics infrastructure

56. Design a highly interactive data analysis system. [Reported] [6]
    - What latency problem appears if you start with batch Spark processing?
    - How would you use Flink with Kafka for streaming aggregation?
    - How does recovery work when a Flink job fails (checkpoints, offsets, exactly-once)?
57. Design an ad-click (or listing-click) event aggregation system. [Reported] [3]
    - How do you handle late and duplicate events?
    - How do you reconcile real-time counts with batch-corrected counts?
58. Design a user event aggregation system, including its API, database, monitoring and SLOs. [Reported] [11]
    - Which SLOs would you commit to (freshness, completeness, query latency), and how would you alert on each?
59. Model cumulative booking metrics, including the first and last booking times per user or listing. [Reported] [3]
    - Would you compute this incrementally or recompute it? How do you backfill when the logic changes?
60. Design the logging pipeline that captures search impressions, clicks and bookings for ranking training data. [Likely]
    - How do you join impressions to bookings that happen days later?
    - How do you guarantee the logged features match what the model saw at serving time?
61. Design a metrics and dashboard system for marketplace health (bookings, nights, supply) with daily and real-time views. [Likely]
62. How would you design data-quality checks and alerting for core tables that feed models and experiments? [Likely]
63. An existing production system has a high error rate. Design changes to make it more reliable and faster. [Reported] [16]
    - How do you find where the errors come from before you redesign anything?

G. Feature store and ML platform infrastructure

64. Design a feature store that serves offline training and low-latency online ranking from one feature definition. [Likely] [8]
    - How do you guarantee point-in-time correctness in training backfills?
    - How do you measure and alert on online/offline consistency?

65. Design the pipeline that measures online/offline feature consistency by logging online feature fetches, replaying their keys and timestamps as a backfill, and comparing the results. [Likely] [8]
    - At what sampling rate would you log, and how do you attribute a mismatch to a specific feature or pipeline?
66. How would you support windowed aggregations (for example a listing's bookings in the last 7 days) in both streaming and batch? [Likely] [8]
    - How do you handle very skewed keys, such as a popular listing or a bot user, in the aggregation?
67. Design the online serving layer of a feature store for a ranking request that fetches features for 1,000 candidate listings in under 50 ms. [Likely] [8]
    - Which key-value store would you choose? How do you batch, cache and tolerate tail latency?
68. A team adds a new feature to a ranking model already in production. Design the backfill, validation and rollout path. [Likely] [8]
69. How do you version features, and what happens to running models when a feature definition changes? [Likely]
70. Design a model-serving platform that supports canary releases, shadow traffic and rollback. [Likely]
71. Design an embedding-serving service that keeps listing embeddings fresh as listings and behavior change. [Likely]
72. Design the serving backend for host price suggestions: which parts are batch-computed, which are computed per request, and how do you keep them consistent with the calendar? [Likely] [42][43]
73. Design the serving path for listing recommendations ("similar listings"): what is precomputed, what is retrieved online, and how is it kept fresh? [Likely] [43]
74. Design a job scheduler for about ten million users. [Reported] [1][3][4][28][30][31]
    - How do you guarantee each job runs at least once, and when do you need exactly-once?
    - How do you scale the dispatcher and avoid one hot time bucket (for example midnight UTC)?
    - How would you shard the job database? How do queue features such as visibility timeouts and delayed delivery (for example in SQS) work underneath? [28]
75. Design a distributed job scheduler with task dependencies, priorities, high availability, retries and monitoring, for example for ML training and backfill DAGs. [Reported] [28][29]
76. A cache holds derived values that depend on other cache entries (a DAG). Design the invalidation and update path so that dependents are always correct when an input changes. [Likely] [44]

H. Experimentation platform (backend view)

77. Design an A/B testing platform. [Reported] [3]
    - How do you assign users to variants deterministically and consistently across web, iOS and Android?
    - How do you handle logged-out users who later log in?
    - How do you log exposures so analysis is not diluted?

78. Design the assignment service's API and latency budget, given that every page load may evaluate dozens of experiments. [Likely]
    - Do you evaluate on the server or in a client SDK? How do you push config changes?
79. How would you support mutually exclusive experiment layers and holdouts? [Likely]
80. Design the metrics pipeline that computes experiment scorecards daily for hundreds of experiments and thousands of metrics. [Likely]
81. How would you design a kill switch or automated guardrail that stops an experiment when a key metric degrades? [Likely]
82. Design a network-aware experiment for a "Wi-Fi badge" on listings, where treatment effects can spill over between listings. [Reported] [3]
    - How would the platform support cluster or market-level randomization?

I. Generic distributed-systems building blocks (asked at Airbnb)

83. Design a distributed key-value store. [Reported] [1][23][24]
    - Walk through the CAP trade-offs, consistent hashing and vector clocks. [1]
    - How do you support get, put, delete and a paginated prefix scan (get(key*)) under eventual consistency and high availability? [23]
    - How do you handle node failure, rebalancing and fault recovery? [24]
84. Design a highly reliable, highly available distributed key-value system that supports bulk upload, multiple versions per key and TTLs. [Reported] [22]
    - How does a bulk upload become visible atomically, and how do you garbage-collect expired versions?
85. Design an in-memory SQL engine. Agree on scope first. [Reported] [1]
    - Which subset of SQL (select, where, join, group by) do you commit to, and why?
86. Design an online two-player chess game. [Reported] [3][4][38]
    - What does the API look like? How do you sync game state in real time and handle disconnects and clocks?
87. Design a rate limiter for Airbnb's public APIs and internal services. [Likely]
88. A partner API complains that your service calls it too often. Redesign the integration to batch the calls without breaking freshness guarantees. [Likely] [19][41]
89. Design a distributed cache in front of the listing service, and explain invalidation when a host edits a listing. [Likely]
90. Design account merge and user de-duplication for people who create multiple accounts. [Likely] [1]
    - How do you merge reservations, reviews and payment methods safely?

J. Other Airbnb product surfaces

91. Design the reviews system: double-blind reviews revealed only when both parties submit or the window closes. [Likely]
    - How do you implement the reveal-at-deadline logic reliably?
92. Design the listing creation and photo upload pipeline, including image processing and moderation. [Likely]
93. Design Experiences booking with time slots, group capacity and a waitlist. [Likely] [3]
94. Design a multi-traveler itinerary service that combines stays and experiences, for example filling a layover with experiences. [Likely] [10]

95. Design a relisting detection system that catches a banned host relisting the same property under a new account. [Reported] [2][3]
    - How do you use geohashing and photo or text similarity at scale?
96. Design a listing lifetime-value estimation service and its batch and online parts. [Reported] [2][3][42]
97. Design the trust and safety screening pipeline that scores every booking before confirmation. [Likely]
    - How do you keep it off the critical path where possible, and do you fail open or closed when it is down?

K. Cross-cutting senior probes (apply to any prompt)

98. What are your back-of-the-envelope numbers for this design (QPS, storage, peak-to-average ratio)? [Likely] [1][17]
99. You have 45 minutes. Which two components would you deep-dive, and why those? [Likely] [1]
100. Write out the API and the core tables before you draw any boxes. Which queries does each index serve? [Likely] [1][16][19]
101. How do you deploy a schema change to the reservations table with zero downtime? [Likely]
102. How would you run this system across multiple regions? What happens during a region failover in the middle of a booking? [Likely] [17]
103. What metrics, alerts and SLOs would you define for this service? [Likely] [11]
104. Which part of this design would you A/B test, and what could go wrong with the test because of the architecture (caching, sticky assignment, logging gaps)? [Likely]
105. How would you migrate from the existing monolith to this service-oriented design incrementally? [Likely]
106. If traffic grew 10x tomorrow, what breaks first? [Likely]
107. What would you simplify if you had to ship a first version in two weeks? [Likely] [1]
108. How do you secure personal data (addresses, IDs, payment details) in this design, and who can access it? [Likely]

References

All sources were fetched on 2026-10-14 except [9], which could not be reached (medium.com did not resolve) and is cited from memory. Sources [11]-[45] are individual Airbnb interview reports from 1point3acres, translated and mirrored in the FlowSight/coding GitHub repository. Each one was read in full.

1. "Airbnb Software Engineer Interview Process" (based on 41 candidate reports, mostly 2025 Q4 to 2026 Q2), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
2. "Airbnb Machine Learning Engineer Interview Process", kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md

3. "Airbnb" question list (system design titles with report dates, Nov 2025 to Sep 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
4. "Airbnb" company overview (prompts by round type), kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/README.md
5. "Airbnb" company prep page (Jan 2026 senior SWE booking and hot-partition report, Jan 2026 group chat report, G8 loop), jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
6. "Airbnb interview questions" (Problem 10: data analysis system design), warrenzhu25/system-design, GitHub. https://github.com/warrenzhu25/system-design/blob/master/airbnb_interview_questions.md
7. "Interview formats of top companies", Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
8. "Chronon: a data platform for AI/ML" (Airbnb open-source feature platform: point-in-time backfills, windowed aggregations, online/offline consistency measurement), airbnb/chronon, GitHub. https://github.com/airbnb/chronon
9. "Avoiding Double Payments in a Distributed Payments System", The Airbnb Tech Blog (Medium), 2019. Not verified in this session (domain unreachable). https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb
10. Airbnb 1point3acres coding prompts, including "Refund Allocation by Payment Method and Date Priority" and "Fill Layover Hours by Booking Experiences" (folder listing), FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a
11. Airbnb onsite report: "Design a user event aggregation system, including API, database, monitoring, and SLO", FlowSight/coding (1p3a #101). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/101.md
12. Airbnb phone screen report: 5-minute design question on the data flow after a user modifies their information, FlowSight/coding (1p3a #30). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/30.md
13. Airbnb infrastructure-team onsite report: full-text search system design, and flight booking design with an L7 interviewer (PA versus PC trade-off), FlowSight/coding (1p3a #70). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/70.md
14. Airbnb onsite report: "payment service with two databases ... How do we add shards?", FlowSight/coding (1p3a #71). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/71.md
15. Airbnb onsite report: payment system design for 300M DAU, FlowSight/coding (1p3a #17). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/17.md
16. Airbnb onsite report, Oct to Dec 2025: system design rounds on Google Wallet and on improving a system with a high error rate; the interviewer went deep on APIs and database design, FlowSight/coding (1p3a #16). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/16.md
17. Airbnb report with a detailed group-only IM design prompt (large groups, fan-out, delivery semantics, ordering, disaster recovery), FlowSight/coding (1p3a #6). https://

github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/6.md

18. Airbnb onsite report: "SD: Wallet service", FlowSight/coding (1p3a #67). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/67.md
19. Airbnb onsite report: listing booking with a waitlist notifying the first 20 users, with pricing, search and notification services given; retryer coding; code review on pagination and batching calls to a partner API, FlowSight/coding (1p3a #69). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/69.md
20. Airbnb onsite report: "Design a hot listing waitlist API", FlowSight/coding (1p3a #58). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/58.md
21. Airbnb (China office) senior report, 2023: ticket system with a watchlist or dashboard and filter search; failed on design against the senior bar, FlowSight/coding (1p3a #76). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/76.md
22. Airbnb Infrastructure senior report: distributed KV system with bulk upload, multiple versions and TTL; rejected because the design was not at senior level, FlowSight/coding (1p3a #62). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/62.md.md
23. Airbnb senior onsite report: distributed key-value store with paginated get(key*), eventual consistency and high availability, FlowSight/coding (1p3a #29). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md
24. Airbnb staff-level report: key-value store design (sharding, fault recovery) in the first round, FlowSight/coding (1p3a #94). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/94.md
25. Airbnb report, Oct to Nov 2025 (4.5 YOE): ticketing system with routing, user-agent chat and statuses; an added design round on booking and double-booking prevention; slow page for hosts with more than 100 listings; split-stay screen, FlowSight/coding (1p3a #11). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md
26. Airbnb senior report: customer ticket management system design, plus the Airbnb notification-system prompt (booking email, batch host promotions, lock-code SMS on check-in day), FlowSight/coding (1p3a #13). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
27. Airbnb report: two system design rounds, on a booking system and a customer ticket handling system, FlowSight/coding (1p3a #18). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/18.md
28. Airbnb onsite report, Jul to Sep 2025: job scheduler (DB sharding, SQS visibility timeout and delay scheduling) with dependencies, priorities and HA, FlowSight/coding (1p3a #24). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/24.md
29. Airbnb Staff SWE (L6) report: "Design Distributed Job Scheduler" and "Design Airbnb's booking system", FlowSight/coding (1p3a #78). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/78.md
30. Airbnb onsite report: system design rounds on group chat and a job scheduler, FlowSight/coding (1p3a #80). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/80.md
31. Airbnb report: "System design, job scheduler", FlowSight/coding (1p3a #85). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/85.md

32. Airbnb senior report: chat system with heavy schema focus and top-N group chat optimization, WebSocket routing and disaster recovery; failed against the senior bar, FlowSight/coding (1p3a #10). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
33. Airbnb report: chat design focused on the schema for "only displaying the top 20 recent chats", FlowSight/coding (1p3a #72). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/72.md
34. Airbnb onsite report: typeahead search design, and group chat with an inbox ordered by the most recent message, FlowSight/coding (1p3a #93). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/93.md
35. Airbnb onsite report, 2025: group chat schema for the top 20 chats, and notifications to online and offline users (long polling, WebSockets, async queues), FlowSight/coding (1p3a #104). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/104.md
36. Airbnb onsite report (August): group chat design focused on database tables and the query functions, FlowSight/coding (1p3a #28). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/28.md
37. Airbnb senior report, booking and orchestration team: group chat system with a focus on DB design, FlowSight/coding (1p3a #73). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/73.md
38. Airbnb onsite report: two-player chess game design; feedback was that the API design needed too many hints, FlowSight/coding (1p3a #39). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/39.md
39. Airbnb system design question: Google Calendar with recurring events and a schema for fast queries, FlowSight/coding (1p3a #102). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/102.md
40. Airbnb onsite report: architecture round on notification systems, FlowSight/coding (1p3a #99). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/99.md.md
41. Airbnb code review round report: PRs on adding pagination, input validation and batched calls to an external API, FlowSight/coding (1p3a #88). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/88.md
42. Airbnb MLE report: ML design rounds on LTV and a host pricing system, FlowSight/coding (1p3a #40). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
43. Airbnb MLE report: two ML design rounds, on listing recommendation and price prediction, FlowSight/coding (1p3a #79). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
44. Airbnb infrastructure report: coding on updating a cache with dependent entries (a DAG), FlowSight/coding (1p3a #35). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/35.md
45. Airbnb onsite report: "classic design group chat", approached like a WhatsApp design, FlowSight/coding (1p3a #3). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md

07. Past Project / Technical Experience Deep Dive: Reported Prompts and Probing Follow-ups for Senior MLEs

> Research note (read first). This chapter was re-verified on 2026-10-14. The shared web-search budget was used up, so no new search queries could run. The network proxy could not resolve arxiv.org, medium.com, airbnb.tech, careers.airbnb.com or trueinterview.io, and the Glassdoor, Blind, LeetCode, IGotAnOffer and InterviewQuery hosts were unreachable earlier. GitHub was reachable, so this chapter is built from pages fetched live there: a mirror of about 100 Airbnb interview posts from 1point3acres (the FlowSight mirror, in Chinese and English), the TrueInterview Airbnb guides (CC BY 4.0), which are based on 3 MLE reports and 41 SWE reports from 2025 Q4 to 2026 Q2, and the TrueInterview Airbnb question list (dated entries to September 2026). In this review, [1]-[4], [5], [6]-[10], [14], [15], [17], [18], [19], [21], [23], [24], [28] and [34]-[37] were re-fetched and checked against the questions that cite them. Three tags were corrected: the whiteboard prompt (now [Likely]), the API and database probe (now [Likely], because that report came from a system design round), and the time-zone and "code to project" prompts (still [Reported], but from the core values round). [30]-[33] could not be fetched (DNS failure) and are cited from prior knowledge. Tags: **[Reported]** means a cited source says a candidate was asked this, or something very close to it, at Airbnb. **[Likely]** means an inference from the role, the senior bar and Airbnb's products; a citation on a [Likely] question only supplies context. Most deep-dive reports come from SWE and infrastructure loops, but the MLE guide says the MLE track runs the same round format [1].

Format notes

Aspect	What sources report
Name on the schedule	"Experience deep dive", "Technical Experience Interview", "Project Deep Dive", "Resume deep dive", "Project presentation", or for ML roles "ML Experience" [1][2][6][7][9][16][17]
Length	45-60 min. One candidate's round was booked for 1 hour and ran about 90 minutes [1][8][9][14]
Interviewer	Usually a senior IC or an engineering manager, and sometimes a senior manager from a neighbouring team, who walks through the project step by step [16][21][25][36]. A separate hiring-manager (HM) call often repeats the deep dive and adds behavioral questions [9][22][23]
Shape	One project, taken apart: architecture, the decisions that were yours, trade-offs, measured impact. Interviewers keep asking "until they heard what they wanted to hear" [1][6][16]. In at least one loop the deep dive and a coding exercise shared a single session [26]. Behavioral and values questions are scored in separate rounds run by employees from any function [5]
Senior / L5 bar	**The perceived scope of your work is scored on its own.** Lead with problem size, ownership and outcome, not model details [1]. The round "also drives leveling" [2]. Scope judgements here can overturn strong technical rounds [2][3]

Aspect	What sources report
Weight	One recruiter told a candidate "the deep dive round is the most important", and Airbnb would not re-run it [18]. Several senior rejections were pinned on this round alone [8][18][20]
Reported failure modes	Opening walkthrough too long: keep it near 10 minutes, and one candidate's 20+ minutes ate the follow-up time [2]. A narrow project reads as below level, and a polished success story with no real mistakes is marked down [3]. A project too complex to explain or abstract quickly [8][14]. Generic answers without technical detail [18]. "No technical challenge visible" [10]. Background judged a poor match for the team [20]
MLE-specific twist	One MLE candidate's "ML Experience" round turned into an unexpected ML design question [7]. Prepare to whiteboard your project's system the way you would a design round
Prep advice from reports	Know every line on your resume: one candidate was grilled on a project that took one line [12]. Bring metrics, a conflict and a rejected alternative [2][3]. At staff level, compare your system with how Airbnb solved a similar problem on its tech blog, and say why you chose your approach [24]

How to use this chapter: choose **one** flagship ML project that touched an experiment or online metric, with clear cross-team reach. Practise a 10-minute arc, then drill every question below against that project. Prepare a backup project for questions that ask for a different story, such as a failure or a conflict.

Questions

A. Opening, project selection and the 10-minute arc

1. Walk me through the project you are proudest of. [Reported] [16][11][29]
    - Why this project and not another one on your resume?
    - Can you give me the two-minute version before we go deep? [Likely] [3]
2. Pick one recent project and walk me through it from start to finish. [Reported] [19] [25]
    - Where do you want to start: the problem, the system or the result?
3. Tell me about the most interesting project you have worked on. [Reported] [22]
4. Show me a project you are proud of that was cross-organizational or cross-functional. [Reported] [11]
    - Which teams outside yours had to change something for it to ship?
5. Walk me through the projects on your resume. What were you responsible for on each? [Reported] [13]
    - You wrote one line about this older project. Tell me about it in detail. [Reported] [12]
6. Walk through one ML project end to end, covering the decisions that were yours and the impact you measured. [Reported] [1]
    - Who used its output, and how many users, listings or dollars did it touch? [Likely] [1]
    - Which metric moved, and which teams did you have to align with? [Likely] [1]
7. If you had to describe the project's scope in one sentence (problem size, your ownership, outcome), what would it be? [Likely] [1]
8. Can you draw the overall system on the whiteboard while you explain? [Likely] [14]
9. Walk me through your background, and why Airbnb specifically? Which project best shows why you fit this team? [Reported] [3]

B. Problem framing and business context

10. What problem was the project solving, and why did it matter to the business at that time? [Reported] [17][6]
    - What would have happened if nobody had done this project?
11. Why did the project start, and how did you get stakeholders to commit to it? [Reported] [2][3]
    - Which stakeholders outside your team had to commit, and what did they give up? [Reported] [3]
12. Set the implementation aside: what was the user problem, and how did the product change for users? [Reported] [2]
    - How would you explain the impact to a product manager who never looked at the model? [Likely]
13. What decision or product surface did the model's output feed into? [Likely] [1]
    - Who consumed the score: a ranker, a rule engine, an ops team or a user-facing UI?
14. How did you turn a vague product goal into a measurable prediction problem? [Likely] [1]
    - What was the label, and why that label rather than a nearby proxy?
15. What was the scale: traffic, QPS, data volume, number of entities? [Reported] [10]
    - What about that scale made the problem hard?
16. Why did you split the work into a short-term plan and a long-term plan? Why was the long-term plan needed at all? [Reported] [10]
17. Was ML actually necessary here? What was the heuristic or rules baseline? [Likely]
    - By how much did ML beat the baseline, and was the extra complexity worth it?
18. How did you size the opportunity before you started? [Likely]
19. How did you choose the training objective or loss, and how closely did it track the business metric you cared about? [Likely]
    - In a two-sided marketplace, whose outcome did the objective favour, guests or hosts, and how did you balance the two? [Likely]

C. Ownership: what was yours versus the team's

20. What was your specific role on the project? [Reported] [15][13]
    - Which decisions did you personally make, and which did you inherit?
21. How was the work divided across the team? [Reported] [9]
    - Who else was on the project, and what did each person own?
22. Did you lead the project, or contribute to it? How many people did you direct or influence? [Likely] [1]
23. Were the metrics you analysed already there, or did you add them? [Reported] [10]
24. What did you do that nobody asked you to do? [Likely]
25. If you had left halfway through, what would not have happened? [Likely]
26. Whom did you influence outside your own group, and how? [Likely] [2]
27. After launch, who owned the model in production (on-call, retraining, deprecation)? Did ownership move to another team? [Likely]

D. Data, labels and features

28. Where did the training data come from, and what were its biggest quality problems? [Likely] [1]
    - How did you detect and handle label leakage?
    - How did you handle delayed or censored labels (for example a booking that happens weeks after a search)?
29. How did you build the label, and how noisy was it? [Likely] [1]
    - Did you consider regression versus a bucketed classification framing? [Likely] [1]
30. Which features mattered most, and how do you know? [Likely]
    - Did any "obviously useful" feature turn out to hurt?
31. How did you keep training and serving features consistent? [Likely]
    - Did you ever ship with training-serving skew? How was it caught?
32. How did you handle cold start for new users or new items? [Likely] [1]
33. Did you use spatial or geographic signals? How did you encode them? [Likely] [1]
34. Did your training data come from logs produced by an earlier model? How did you deal with feedback-loop and position bias? [Likely] [30]
35. For a ranking or recommendation project, how did you build the training examples: impressions, negatives, sampling? [Likely] [34]
36. Did the project use personal or sensitive data? What privacy, retention or fairness constraints shaped the features? [Likely]

E. Modeling decisions and trade-offs

37. Walk me through the architecture and the key design points. [Reported] [6][23]
    - Why this design over the obvious alternative?
38. What were the main trade-offs in your ML project? [Reported] [1][9][37]
    - What did you give up, and how did you decide it was acceptable?
39. What alternatives did you consider and reject? [Reported] [2][3]
    - What evidence made you reject each one?
40. Why this model family (for example GBDT versus a neural network)? [Likely] [1][30]
    - When did you decide to move to something more complex, and what convinced you?
    - Why this feature set and this serving path rather than the alternatives? [Likely] [1]
41. What was the single modeling change that gave the biggest gain? [Likely]
    - What did you try that did not work? [Likely] [30]
42. How did you tune the model, and how did you avoid overfitting to your validation set? [Likely]
43. How did you trade model quality against latency, cost or interpretability? [Likely]
44. Did downstream consumers depend on calibrated scores? How did you check calibration, and did it hold after launch? [Likely]
45. If the project used embeddings or an LLM, how did you evaluate output quality, and how did you control inference cost? [Likely] [34]
46. If the interviewer cannot follow your domain, how would you simplify the core idea in two minutes? [Reported] [8][14]

F. Architecture, serving and production

47. Walk me through the implementation details end to end, from data pipeline to serving. [Reported] [6]
48. What were the latency and throughput requirements at serving time, and how did you meet them? [Likely]
49. Was inference batch, online or both? Why? [Likely]
50. How did you deploy new model versions safely (shadow, canary, rollback)? [Likely]
51. How did you make sure of system quality after deployment? [Reported] [17]
    - What monitoring and alerting did you set up for data drift and model decay?
52. What broke in production, and how did you find out? [Likely]
53. Which part of the system would break first at 10x scale? [Likely]
54. Did you build or reuse a platform (feature store, training pipeline, experiment framework)? Why? [Likely]
55. Walk me through an incident on this project. What was the root cause, and what was the fix? [Reported] [2]
56. How often was the model retrained, and how did you choose that cadence? How were data and model versions tracked so that a run could be reproduced? [Likely]
57. What did the project cost in compute and infrastructure, and how did you justify that cost? [Likely]

G. Offline-to-online evaluation and experimentation (A/B testing focus)

58. How did you evaluate the model offline, and which metric did you trust? [Likely] [1]
    - How well did the offline metric predict the online result?
59. How did an offline metric gain become an online metric gain? [Likely] [1]
60. How did you know it worked? Was there an A/B test? [Likely] [31]
    - What was the primary metric, and what were the guardrails? [Likely] [34]
    - What was the randomization unit, and why?
    - How long did it run, and how did you choose the sample size?
61. What were the confidence intervals on the main result? Were any segments surprising? [Likely]
62. Did offline gains ever fail to show up online? How did you debug it? [Likely] [30]
63. Did your experiment face interference effects (for example a two-sided marketplace or shared inventory)? How did you handle it? [Likely] [31][34]
    - Would cluster-level or market-level randomization have changed the answer? [Likely] [34]
64. Did you consider faster methods such as interleaving for a ranking change? Why or why not? [Likely] [32]
65. Did you use variance reduction such as CUPED? What did it buy you? [Likely]
66. How did you handle novelty effects, seasonality or ramp-up effects when you read the result? [Likely]
67. Did you check for sample ratio mismatch or logging bugs before you trusted the result? [Likely]
68. Did you keep a long-term holdout? What did it show compared with the initial test? [Likely]
69. Tell me about an experiment result that surprised you. What did you do next? [Likely]
70. Who made the launch decision, and what would have made you not launch? [Likely]
71. How did you choose the minimum detectable effect, and did you check beforehand that the metric was sensitive enough to detect it? [Likely]

72. How did you handle many metrics and segments at once, and how did you avoid peeking or stopping the test early? [Likely]
73. The true outcome (for example a completed stay, a review or long-term value) arrives weeks later. Which proxy or surrogate metric did you use in the test, and how did you validate it? [Likely] [6][34]
74. If a clean A/B test was not possible (for example for pricing or a policy change), which quasi-experimental method did you use, and what assumptions did it rest on? [Likely] [34]
75. The primary metric was flat but a secondary metric improved. What did you recommend, and why? [Likely]

H. Impact and metrics

76. What was the measured impact, in business terms? [Reported] [2][3]
    - How did you attribute the impact to your work and not to other launches?
77. How did you ensure the users actually benefited after deployment? [Reported] [17]
78. What metrics did you use to define success, and who agreed to them? [Reported] [2]
    - How did you align Product and Data Science on the success metric? [Likely]
79. Were there negative side effects on another metric or another group (for example hosts versus guests)? [Likely]
80. If the impact was smaller than expected, why? [Likely]
81. Did you measure segment-level harm, for example on new hosts, small markets or new users? What did you find? [Likely]

I. Challenges, failures and debugging

82. What was the most challenging part of the project? [Reported] [16][15]
    - Was the challenge technical, organizational or about reliability?
83. What technical challenges did you hit, and how did you solve them? [Reported] [6][14]
    - Where exactly is the technical challenge in this project? [Reported] [10]
84. Tell me about a project or plan that missed its ETA, or whose scope had to shrink. [Reported] [9]
    - How did you identify the root cause? [Reported] [9]
    - Did you have a fallback plan to unblock the team? [Reported] [9]
85. Tell me about a model that worked offline but failed in production. [Likely]
86. Tell me about a bug or data problem that invalidated results. How was it found? [Likely]
87. What was the hardest debugging problem on the project? [Likely]
88. Tell me about a failure on this project and what you did next. [Reported] [3][4]
    - Tell me about a real mistake you made, not the polished version of the story. [Reported] [3]

J. Planning, milestones, collaboration and conflict

89. How did you plan the project? [Reported] [15]
    - How did you set and deliver milestones? [Reported] [17][11][1]
90. Describe the project's milestones, a conflict over scope, and how it was resolved. [Reported] [1]

91. Describe a conflict during the project, and which of the rejected alternatives you would choose now. [Reported] [2]
92. How did you scope the project to fit a deadline? [Reported] [2]
    - When requirements changed mid-project, how did you re-plan and rescope? [Reported] [2][3]
93. How was the engineering effort structured? [Reported] [11][29]
94. How did you persuade stakeholders to back the project? [Reported] [2]
    - Who opposed it, and what changed their mind?
95. Did you have a conflict on the project? How did you resolve it? [Reported] [15][16][9]
    - What would the other person say about how it was resolved?
96. How did you work with your teammates? [Reported] [16]
97. Tell me about collaborating with another team where their delay affected you. [Reported] [9]
98. How did you keep teams in different time zones or cultures aligned from design through implementation? What problems came up? (asked in the core values round) [Reported] [9]
99. How did you handle a tight deadline or a complex project you owned? [Reported] [4]
100. How did you work with data scientists on metric definitions and experiment reading? [Likely]
101. How did you influence the technical direction without formal authority? [Likely]

K. Retrospective, learning and next steps

102. If you redid the project, what would you change? [Reported] [15][17][3]
    - What would you change if you had to redesign it from scratch? [Reported] [17]
103. What did you learn from the project? [Reported] [16]
104. What would the next iteration of the system look like? [Likely]
105. If you had half the time, what would you cut? [Likely]
106. What technical debt did you leave behind, and why was it acceptable? [Likely]
107. How did you decide the project was finished, and what did you stop doing? [Likely]
108. What was the biggest obstacle in moving from writing code to owning a whole project? (asked in the core values round) [Reported] [9]

L. Leadership and senior/staff calibration

109. Walk me through your experience, a past project and your leadership on it. [Reported] [21][36]
110. How did you grow or mentor others on this project? [Likely]
111. What is your current scope at work, and what scope are you looking for? [Reported] [10]
    - Are you looking to be a tech lead, or a strong IC? [Reported] [10]
112. How did this project change how your team or organization works? [Likely]
113. Which decision on this project would you defend most strongly to a skeptical staff engineer? [Likely]
114. What was the strongest pushback in design review, and did it change the design? [Likely]
115. How did you decide what not to build? [Likely]

M. Mapping your project to Airbnb

116. How does your system compare with how Airbnb solved a similar problem, and why did you choose your approach? [Reported] [24]
    - Have you read Airbnb's engineering blog posts on search ranking, pricing or experimentation? How does your work differ? [Likely] [24][30][31]
117. How would your project transfer to a two-sided marketplace like Airbnb's (guests and hosts)? [Likely]
118. What in your background maps directly onto this team's problems? [Reported] [20][2]
    - You have also worked in an unrelated area (for example hardware). How is that relevant here? [Reported] [20]
119. If you joined, what would you build in your first six months, based on this experience? [Likely]
120. Tell me about a project connected to your personal or professional values. How did it embody or challenge them? [Reported] [19][33]

N. Pressure probes reported in the room

These are the push-back styles candidates describe. Practise staying concise and concrete under each one.

121. "I still don't understand. Can you explain the task again, more simply?" [Reported] [14]
122. "Why did you do that at all?" (repeated challenges to the rationale for a phase of the project) [Reported] [10]
123. "Can you be more specific? What were the actual technical details?" (asked because answers sounded generic) [Reported] [18]
124. Interruptions and nitpicking mid-answer: how do you regain the thread? [Reported] [27]
125. "Let's switch: how would you design an ML system for X?" (the experience round turns into ML design) [Reported] [7]
126. "Give me the specific API and database design of your system." (a detail-level probe instead of scaling talk; reported in an Airbnb system design round, not the deep dive) [Likely] [28]

References

[1]-[29] and [34]-[37] were fetched from GitHub; the subset listed in the research note was re-fetched on 2026-10-14. [6]-[29] and [35]-[37] are English or Chinese 1point3acres posts mirrored on GitHub. The mirror does not keep original post dates, though several posts carry 2025 timestamps. [30]-[33] could **not** be fetched (DNS failure on 2026-10-14) and are cited from prior knowledge. Verify them before relying on them.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview, CC BY 4.0; based on 3 candidate reports, 2026 Q2). GitHub mirror kevin-2023-code/Tech-Interview-Questions, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md

2. "Airbnb Software Engineer Interview Process" (TrueInterview, CC BY 4.0; 41 candidate reports, 2025 Q4 to 2026 Q2). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
3. "Airbnb interview process" (TrueInterview general guide). GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. "Airbnb" company guide (Technical Experience round, behavioral list; updated October 2026). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. "Interview formats of top companies" (Airbnb section). Tech Interview Handbook, GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
6. 1p3a post #40 (MLE loop): project deep dive covering architecture, implementation, challenges and solutions; ML design on LTV and host pricing. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
7. 1p3a post #79 (MLE loop): "ML Experience" round expected as a project deep dive but turned into an ML design question; two ML design rounds. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
8. 1p3a post #10 (senior, rejected on design and resume-project rounds): 1-hour round ran 90 min on one project; advice to keep it under 20 min and simplify. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/10.md
9. 1p3a post #11: VO3 "Tech Experience Interview" (45 min; "trade-offs, conflicts"); HM round with resume deep dive plus BQ (missed ETA or reduced scope, root cause, division of work, fallback plan); core values round (cross-time-zone design-to-implementation, moving from writing code to owning a project). https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md
10. 1p3a post #13 (senior): HM scope call (TL or IC); project deep dive challenged on the long-term plan, traffic level, new versus existing metrics, and the lack of a technical challenge. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
11. 1p3a post #8: "project experience: most proud project, milestone, eng structure, conflict"; "BQ: show a proud project (cross organizational/functional)". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/8.md
12. 1p3a post #3: experience BQ round asked about a project described in a single resume line. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md
13. 1p3a post #30: phone round with a 30-minute resume deep dive on responsibilities. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/30.md
14. 1p3a post #39: 1-hour resume round felt like a stress interview ("couldn't understand"); advice to focus on role, decisions, technical challenges and conflicts. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/39.md
15. 1p3a post #58: project deep dive covering role, planning, challenges, conflicts and redo improvements. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/58.md
16. 1p3a post #69: Technical Experience Interview covering proudest project, most challenging part, teammates, conflicts and lessons. https://github.com/FlowSight/codi

ng/blob/main/company%20wise/airbnb/1p3a/69.md

17. 1p3a post #84 (senior backend): project presentation covering scope, milestones, post-deploy quality and user benefit, and redesign. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/84.md
18. 1p3a post #86: failed the deep dive for "generic answers without any technical details"; recruiter said the deep dive is the most important round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/86.md
19. 1p3a post #93: experience round as a deep dive into a recent project; values-linked project prompt. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/93.md
20. 1p3a post #7: deep-dive feedback said the candidate's background did not fit the team. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/7.md
21. 1p3a post #78 (staff SWE, L6): senior manager from another group ran experience, past projects and leadership. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/78.md
22. 1p3a post #29 (senior): HM went over the most interesting project in detail; VO behavior round focused on one project. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md
23. 1p3a post #62 (infrastructure senior): HM interview on resume projects, key design points and challenges. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/62.md.md
24. 1p3a post #94 (staff): HM round deep on the resume; advice to read Airbnb's tech blog and compare your system with Airbnb's. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/94.md
25. 1p3a post #99: experience interview "dug into a project from start to finish". https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/99.md.md
26. 1p3a post #104: project deep dive and coding combined in one round. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/104.md
27. 1p3a post #31: a manager in the behavioral and in-depth project round kept interrupting and nitpicking. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/31.md
28. 1p3a post #6: "BQ and past experience deep dive" round (no questions listed); the API and database detail probe came from the system design rounds. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/6.md
29. 1p3a post #18: "most proud project, milestone, eng structure, conflict", asked in fine detail. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/18.md
30. Haldar et al., "Applying Deep Learning to Airbnb Search", arXiv / KDD 2019 (includes lessons on failed attempts and offline/online gaps). Not fetched. https://arxiv.org/abs/1810.09591
31. "Experiments at Airbnb", Airbnb Engineering & Data Science (Medium), 2014. Not fetched. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
32. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving", Airbnb Tech Blog (Medium), 2022. Not fetched. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e

33. "Life at Airbnb" (core values), Airbnb Careers. Linked from [4]; not fetched. https://careers.airbnb.com/life-at-airbnb/
34. "Airbnb" company question list (TrueInterview mirror; experiment design prompts including primary and guardrail metrics, a network-aware experiment, causal regression, and LTV and embedding-search ML design; dated entries to September 2026). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
35. 1p3a post #16: onsite round 2 "BQ and past experience deep dive"; notes that senior candidates are usually pushed on scaling. FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/16.md
36. 1p3a post #75: a senior manager walked through the candidate's project experience step by step and gave feedback afterwards. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/75.md
37. 1p3a post #21: experience round on the candidate's project, mostly trade-offs, conflicts and standard behavioral questions. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/21.md

08. Core Values Interviews: Champion the Mission, Be a Host, Embrace the Adventure, Be a Cereal Entrepreneur, plus Cross-Functional and Leadership Behavioral Questions

> Research note (read first). The shared web-search budget ran out before this chapter was written, so no new search-engine queries were possible. DNS lookups also failed for glassdoor.com, leetcode.com, interviewing.io, hellointerview.com, prepfully.com, igotanoffer.com, medium.com, careers.airbnb.com and news.airbnb.com, and reddit.com refused fetches. The chapter is therefore built from pages that could be read directly on GitHub. These include two TrueInterview-derived Airbnb process guides (one for the MLE track, one for the SWE track), a 2026 company guide that aggregates Hello Interview, interviewing.io, PracHub and LeetCode Discuss, the Tech Interview Handbook's Glassdoor-sourced Airbnb list, and a mirror of more than 100 1point3acres (1p3a) Airbnb interview posts. Every 1p3a post was downloaded and searched for values, behavioral (BQ), hiring-manager (HM) and culture content. Chinese-language posts are paraphrased in English here. Tags: **[Reported]** means a cited source says a candidate was asked this, or something very close to it, at Airbnb. **[Likely]** means an inference from the values, the Senior MLE / experimentation role, or Airbnb's products. It does not mean anyone reported it.

Format notes

Aspect	What sources report
Number and length	One core-values round of about 45 minutes is reported for the MLE track [1]. The SWE track reports one or two rounds of about 45 minutes each [2][3]. Hello Interview's G8 guide (as aggregated) calls it a "30-min mandatory gatekeeper" [4]. Rounds sometimes end early: one 2026 candidate finished in about 20 minutes and still passed [4][10]
Timing	Often scheduled **after** the technical onsite has passed, as a separate gate [4][8][10]. One candidate had it after hiring-manager calls during team matching [18]
Interviewer	An employee from **any function**, outside the hiring team, often a non-engineer [1][2][4][5][7]. One senior candidate's values round was with an engineering manager [15]
What is scored	Authentic connection to Airbnb's four values: Champion the Mission, Be a Host, Embrace the Adventure, Be a "Cereal" Entrepreneur [5][12]. Personal stories count as much as work stories [2]. Answers that relabel engineering wins with value names, or that sound rehearsed or framework-driven, land poorly [1][2][3]. Rapport with the interviewer affects the outcome [1]

Aspect	What sources report
Weight	Scored in the same debrief as the technical rounds [2]. It "can reject you on its own" [4][5]. One candidate failed two behavioral rounds and was told they "answered too quickly, without their own pace" [16]
Related rounds	**Experience / project deep dive** (45-60 min): cross-functional influence, conflict, milestones and "who outside your team you had to align" are probed here. It drives leveling [1][2][17][18][19]. **Hiring manager conversation**: scope, past projects, team fit, and sometimes BQ questions about missed cross-team ETAs [1][8][20]
Senior bar	No source defines a separate senior bar for the values round itself [1][2]. For Senior, scope, cross-team influence and ownership are judged mainly in the deep dive and HM rounds [1][2][20]. Expect values interviewers to probe for the same signals: mentoring, hosting newcomers, and leading through ambiguity [Likely]

Preparation signals from sources: prepare one or two stories per value, including non-work ones such as a cause you support, a time you hosted or built community, and feedback that stung [2][3][4]. Use plain language for a non-engineer interviewer [5]. Pace your answers and do not rush [16].

Questions

A. Opening, background and "Why Airbnb?"

1. Walk me through your background. [Reported] [8]
    - What thread connects your career moves?
    - What do you do outside work?
2. Why Airbnb? [Reported] [1][3][4][6][8][11]
    - Why now, and why this team rather than a bigger-name ML org?
    - What specifically drew you to Airbnb as a company, beyond the product? [Reported] [1]
3. What do you think of Airbnb? [Reported] [6]
    - What does Airbnb do better than any other travel company?
4. What excites you about the company? [Reported] [6]
5. Tell me something about yourself and why you would be a good fit. [Reported] [6]
6. What are your hobbies outside work, or what fun thing from school would you share? [Reported] [8]
7. Is there anything you really want to do in the future? [Reported] [4]
    - Where do you want to be in five years, and how does Airbnb help?
8. What is something you do not want to repeat from your last job? [Reported] [6]
9. What obstacles, challenges or uncertainty do you expect if you join Airbnb? [Reported] [8]
    - What would make you leave Airbnb after a year?

B. Champion the Mission (belonging, community, causes)

10. What does "belong anywhere" mean to you? [Reported] [6]
    - When have you felt you did not belong, and what changed it?
    - When have you created belonging for someone else?
11. What part of our mission resonates most with you? [Reported] [6]

12. How does Airbnb impact our guests and hosts? [Reported] [6]
    - How does it affect the neighborhoods and cities where hosts operate?
13. What large problems in the world would you solve today? [Reported] [1][6]
    - Why that problem, and what have you personally done about it?
14. What cause do you actively support, and how do you contribute to it? [Reported] [1] [3]
    - How much time do you give it, and what have you learned from it?
15. When did you last volunteer, and what did you take away from it? [Reported] [8]
16. How have you contributed to a community, or what volunteer work have you done? [Reported] [4][5]
17. Tell me a story about your community and the connections you have built. [Reported] [11]
18. Describe a time you stood up for something you believe in. [Reported] [4]
    - What did it cost you, and would you do it again?
19. Who has inspired you most, and why? [Reported] [11]
20. What is Airbnb's hardest problem today, in your view? [Reported] [2]
    - How could ML or experimentation help solve it?
21. Tell me about a project that reflected your personal or professional values. [Reported] [14]
    - How did the project embody or challenge those values?
22. Name an organization (not Airbnb) that you admire, and explain why. [Reported] [4] (intern prompt, flagged unverified by the source)
23. Tell me about a time you chose the right outcome for users even when an internal metric pointed the other way. [Likely]
    - In ML terms: when did you decline to ship a model that won the A/B test?
24. How would you make sure a ranking or pricing model treats hosts and guests fairly across demographics and markets? [Likely]
    - How would you explain that trade-off to a non-technical interviewer?
25. How do you keep the people affected by your models (hosts, guests, support agents) in mind when you only see aggregate metrics? [Likely]

C. Be a Host (care for guests, teammates and newcomers)

26. Give me an example of when you have been a good host. [Reported] [2][6]
    - It does not have to be a listing: an event, a dinner, a visitor, a new hire? [Reported] [11]
27. Do you have any experience as a host, of any kind? [Reported] [11]
28. Tell me about your most memorable experience traveling with Airbnb. [Reported] [8]
    - What did the host do that made it memorable?
29. What are your tips for choosing an Airbnb listing? Have you ever had a bad experience? [Reported] [8]
    - How would you fix that bad experience with product or ML?
30. Share one of your trips with us. / Tell me about your most memorable trip. [Reported] [4][6]
31. Name a situation where a company's customer service impressed you. [Reported] [6]
    - What would it take for Airbnb support to match it?
32. What is the best gift you have ever given or received? [Reported] [6]

33. If you had an unlimited budget to buy one gift for one person, what would you buy and for whom? [Reported] [6]
34. Was there a senior colleague who helped you a lot early in your career? [Reported] [8]
    - How have you paid that forward?
35. Tell me about a time you made a new teammate feel welcome and productive. [Likely]
    - What did you change in onboarding docs, tooling or pairing as a result?
36. Describe a time you mentored an engineer or scientist to a measurable improvement. [Likely]
    - How did you adapt your approach to their style?
37. Tell me about a time you went out of your way for an internal customer, such as a data scientist or product team that consumed your model or platform. [Likely]
38. Tell me about a time you supported a teammate going through a hard period. [Likely]
39. How do you make people who disagree with you feel heard in a design review or experiment readout? [Likely]
40. Tell me about a time you had to give someone terrible news. [Reported] [5][6]
    - How did you prepare, and how did they react?

D. Embrace the Adventure (curiosity, risk, discomfort, resilience)

41. What adventurous moments or decisions have you had in life? How do you take risks? [Reported] [8]
42. Tell me about a calculated risk you took. [Reported] [2]
    - How did you size the downside before you committed?
43. Tell me about a time you were uncomfortable and how you dealt with it. [Reported] [5][6]
44. If you had an unlimited budget and could go anywhere, where would you go? [Reported] [6]
45. What is something new you can teach your interviewer in a few minutes? [Reported] [6]
46. Tell me about a self-learning experience: how you learned something, and whether it helped your later work. [Reported] [13]
47. What is the most challenging project, in or out of work, that you have done recently? [Reported] [6][11]
48. How do you manage the gap between expectations and reality, at work or in life? [Reported] [8]
49. Tell me about a time a major change you did not choose (reorg, layoff, pivot) hit your team. How did you respond? [Likely]
50. Tell me about a time you worked in a domain you knew nothing about, for example moving from recommendation to causal inference. [Likely]
    - How quickly were you useful, and what did you get wrong first?
51. Tell me about an experiment or model idea that failed. What did you learn, and what did you do next? [Likely]
    - How did you tell the team?
52. When have you changed your mind because of data that contradicted a strong belief? [Likely]
53. If you were a gerbil, which gerbil would you be? (a reported curveball used to test spontaneity) [Reported] [6]

E. Be a "Cereal" Entrepreneur (resourcefulness, scrappiness, creativity)

54. Tell me about a time you were scrappy and built something valuable with very limited resources. [Likely] [5][12]
    - What did you deliberately leave out?
55. What is something you would like to remove from the Airbnb experience? [Reported] [5][6]
    - How would you test whether removing it helps?
56. Tell me about a time you saw an unowned problem and took it on without being asked. [Likely]
57. Describe a time you shipped a quick heuristic or a small experiment instead of waiting for the perfect model. [Likely]
    - How did you know when it was time to replace it?
58. Tell me about a time you found a creative workaround for a data, compute or labeling constraint. [Likely]
59. Have you ever started something of your own, such as a side project, club, startup or open-source tool? [Likely]
    - What did it teach you about customers?
60. Tell me about a time you turned a one-off analysis or script into a tool other teams adopted. [Likely]
61. Tell me about a time you reduced cost (infrastructure, experiment duration, labeling spend) without hurting quality. [Likely]
    - Which metrics showed it, and who signed off?
62. Pitch me one product or ML idea for Airbnb that nobody has asked you to build. [Likely]
    - How would you validate it in two weeks?

F. Feedback, failure and self-reflection

63. Have you given or received hard feedback? Tell me about it. [Reported] [2][3][11]
    - What did you change afterwards, and how would the other person describe it?
64. Have you received negative feedback from others, and how did you handle it? [Reported] [13]
65. Tell me about a time you did not meet expectations, and how you improved using customer feedback. [Reported] [13]
66. What could you have done better in a past experience? [Reported] [8]
67. What was your most difficult task, and how did you get through it? [Reported] [11]
68. Tell me about a time you failed. [Likely] [3]
    - What would you do differently, and what did you change in your process?
69. What would your last manager say is your biggest area for growth? [Likely]
70. Tell me about a time you made a mistake that affected users or a launch, such as a bad model push or a wrong experiment readout. [Likely]
    - How did you communicate it, and what guardrail did you add?

G. Cross-functional collaboration and communication

71. Give an example of communicating with your own team or with a different team. [Reported] [8]

72. How do you make sure a design lands well with teams in different time zones or cultural backgrounds? What problems came up? [Reported] [8]
73. In an urgent situation, how do you make sure you get time with senior people? [Reported] [8]
74. How did you work with senior management on large projects and with multiple internal teams? [Reported] [6]
75. Tell me about a time another team's internal ETA slipped and affected you, so the timeline moved or scope shrank. [Reported] [8]
    - How did you identify the root cause? [Reported] [8]
    - How was work split across the team? [Reported] [8]
    - Did you have a backup plan to unblock yourself? [Reported] [8]
76. Show me a project you are proud of that crossed organizations or functions. [Reported] [17]
    - How were milestones set, how was engineering structured, and what conflicts arose? [Reported] [17]
77. Tell me about a cross-team collaboration challenge on a past project. [Reported] [18]
78. Tell me about a conflict, failure, tight deadline or complex project you owned. [Reported] [4]
79. How did you set milestones on your project, and how did you ensure system quality and user benefit after deployment? [Reported] [19]
    - What would you change if you redesigned it? [Reported] [19]
80. Tell me about a time you disagreed with a product manager about the success metric for an experiment. [Likely]
    - Whose metric won, and why?
81. Tell me about a time a data scientist and you read the same A/B test differently. How was it resolved? [Likely]
82. How do you explain a model's behavior or an experiment's uncertainty to a non-technical stakeholder, such as an operations or policy partner? [Likely]
    - Explain a confidence interval to me as if I were your values interviewer.
83. Tell me about a time you influenced a decision with no formal authority over the people making it. [Likely]
84. Tell me about a time you had to say no to a partner team's request. [Likely]
85. How do you work with designers and PMs when a model's output changes the user experience, for example in ranking or pricing suggestions? [Likely]
86. Describe a time you resolved friction between two teammates or two teams. [Likely]

H. Senior-level ownership and leadership (MLE and experimentation flavor)

87. What is the biggest obstacle in moving from writing code to owning an entire project? [Reported] [8]
88. What is your current scope, and what scope do you expect here? [Reported] [20]
    - Do you want to be a tech lead, or a strong IC on a team that already has one?
89. What was the hardest technical or organizational challenge you faced? [Reported] [20]
90. Tell me about a time you set technical direction for a group larger than your own team. [Likely]

91. Tell me about a time you stopped a launch, or argued against shipping, because the experiment evidence was weak. [Likely]
    - How did leadership react, and what happened next?
92. Tell me about a time you raised the bar for experimentation rigor on your team, for example with pre-registration, guardrails or a review process. [Likely]
93. How do you balance short-term booking or revenue metrics against long-term guest and host trust? [Likely]
94. Tell me about a time you had to deliver an inconclusive or negative result to a team that badly wanted a win. [Likely]
95. How do you grow other engineers' judgment about when ML is the wrong tool? [Likely]
96. Tell me about a time you made a decision with incomplete data under time pressure. [Likely]
97. How do you prioritize when three teams all need your help this quarter? [Likely]
98. Tell me about a time you handed off something you built so that it no longer depended on you. [Likely]

I. Hiring-manager conversation (behavioral portion)

99. Walk me through your most interesting project. What were the key design points and challenges? [Reported] [20][21]
100. Which problems on our roadmap map onto your background, and where would you need to ramp up? [Reported] [1]
101. Who outside your team did you have to align on your biggest project, and how? [Reported] [1][2]
102. How did you scope work when requirements changed mid-project? [Reported] [2]
103. Why did you choose a long-term plan over a short-term fix, and how did you justify it? [Reported] [20]
104. Which metrics did you add yourself, and which existed already? [Reported] [20]
105. What do you need from a manager to do your best work? [Likely]
106. What kind of team culture do you create when you are the most senior IC in the room? [Likely]

References

All URLs below were fetched or downloaded for this chapter, except where noted. The 1p3a mirror posts are English translations or original Chinese posts reposted on GitHub. The original 1point3acres threads were not reachable.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview guide, mirrored; based on 3 candidate reports, mainly 2026 Q2). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb Software Engineer Interview Process" (TrueInterview guide, mirrored; based on 41 candidate reports, 2025 Q4 to 2026 Q2). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md

3. "Airbnb Interview Process" (overview). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. "Airbnb" company guide (aggregates Hello Interview G8 guide, interviewing.io, PracHub Jan 2026, LeetCode Discuss post 7621156 from Mar 2026, Glassdoor intern snippets). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/companies/airbnb.md
5. "Behavioral: other companies" (Airbnb section: values, format, practice questions). jugaldb/faang-interview-prep, GitHub, 2026. https://github.com/jugaldb/faang-interview-prep/blob/main/docs/behavioral/other-companies.md
6. "Behavioral interview questions" (Airbnb list, sourced from Glassdoor). Tech Interview Handbook (yangshun), GitHub, undated (older SWE reports). https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/behavioral-interview-questions.md
7. "Interview formats of top companies" (Airbnb: two cross-functional rounds with employees from any discipline). Tech Interview Handbook (yangshun), GitHub. https://github.com/yangshun/tech-interview-handbook/blob/main/apps/website/contents/interview-formats-top-companies.md
8. 1p3a post #11 (two posts in one file: a core values round opening with "When did you last volunteer?", and a 4.5 YOE candidate's full VO with HM BQ and a 15-question Core Values list). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/11.md
9. Airbnb 1point3acres interview-post mirror (folder index). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/tree/main/company%20wise/airbnb/1p3a
10. 1p3a post #3 (passed onsite and core values; values round scheduled after onsite and ended in about 20 minutes; posted 2026-03). FlowSight/coding, GitHub, 2026. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/3.md
11. 1p3a post #57 (core values questions: why Airbnb, hosting experience, hard feedback, community, who inspired you, most difficult task). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/57.md
12. 1p3a post #55 (G10 full-stack; core value round "essentially behavior" demonstrating hospitality and entrepreneurial spirit; 2025-07). FlowSight/coding, GitHub, 2025. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/55.md
13. 1p3a post #45 (BQ opening: self-learning, missed expectations and customer feedback, negative feedback). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/45.md
14. 1p3a post #93 (core value round listed; project tied to personal or professional values). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/93.md
15. 1p3a post #75 (core value interview scheduled with an engineering manager). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/75.md
16. 1p3a post #7 (onsite with culture and behavior rounds; failed both BQ rounds for answering too fast). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/7.md
17. 1p3a post #8 (BQ: proud cross-organizational project; milestones, eng structure, conflict). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/c

ompany%20wise/airbnb/1p3a/8.md
18. 1p3a post #28 (experience round with cross-team collaboration questions; HM calls, then core value round; 2025-10). FlowSight/coding, GitHub, 2025. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/28.md
19. 1p3a post #84 (senior backend; project discussion on milestones, quality and user benefit after deployment, redesign). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/84.md
20. 1p3a post #13 (HM round on current scope and TL versus IC expectations; deep dive challenged long-term plan, metrics and technical challenge). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/13.md
21. 1p3a post #29 (Senior; HM interview with behavioral questions on the most interesting project). FlowSight/coding, GitHub. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/29.md

09. Airbnb ML Context: Papers, Platforms and Recent AI Products as Interview Questions

This chapter turns Airbnb's public ML record into questions you could be asked. That record covers the KDD/CIKM papers, the Bighead / Zipline / Chronon platform line, Minerva and ERF, interleaving, and the 2024-2026 LLM customer-support and AI-search launches. The chapter has no round of its own. Its questions come up inside the ML system design rounds, the experience deep dive (past-project round) and the experimentation discussion. They work best as design prompts ("design X at Airbnb"), so the system-design framing is kept throughout.

> **Research note (read first).** This chapter went through an adversarial review on 2026-10-14. The shared web-search budget was used up, so no new search queries could run. The proxy could not resolve prachub.com, teamblind.com, 1point3acres.com, interviewquery.com, trueinterview.io, arxiv.org, medium.com, airbnb.tech, infoq.com, cxtoday.com, zenml.io, altexsoft.com or syncedreview.com. Twelve of the previously cited URLs were tried and all failed to resolve. GitHub was the only reachable host.
>
> - **Fetched and read on 2026-10-14:** [1]-[7]. These are the TrueInterview-derived Airbnb MLE guide (3 candidate reports, mostly 2026 Q1-Q2), the dated Airbnb question index (entries to Sep 2026), the general Airbnb process page, two 1point3acres MLE reports mirrored on GitHub, the Chronon repository, and the applied-ml index.
> - **URL confirmed:** [8]-[27]. Title, URL and year match the applied-ml index [7], but the page itself was not opened.
> - **Unverified:** [28]-[54]. These are carried over from the previous version of this chapter and could not be opened. Check them before you rely on them.
>
> **Tags. [Reported]** means a fetched source [1]-[5] says a candidate was asked this at Airbnb, or something very close to it. **[Likely]** means an inference from Airbnb's published work, the role and the senior bar. A citation on a [Likely] question only gives context.
>
> **Retagging in this review.** Six questions were moved from [Reported] to [Likely] because their only sources (PracHub, Blind, CleverPrep) could not be opened: FM vs. DNN, offline-NDCG-up-but-A/B-flat, precision/recall/latency/cost, host fairness, the "family friendly" classifier and the ML-platform code review. The topics are plausible but unconfirmed.

Format notes (how this material surfaces in the loop)

- **Where it shows up:** Airbnb MLE loops have one or two ML system design rounds, sometimes more. The prompts are open-ended marketplace problems with no code: search ranking, a property's lifetime value, recommendations, relisting detection, cover photos [1][3]. The loop is decided mainly in these rounds, and a borderline design signal can earn an extra design round [1].
- **Experience round can turn into design:** One MLE candidate expected a past-project deep dive in the "ML experience" round, but the interviewer asked an unplanned ML design question instead [5]. Prepare to whiteboard an Airbnb problem in any ML round.
- **Senior bar:** At Senior (G9) and above, design carries more weight. In the reports, rejection feedback pointed to design depth or overall match, never to coding [1]. One ML rejection cited "jumping straight to model ideas"; the guide's advice is "Problem and label before model" [3].
- **Recent senior loops:** These loops added ranking and retrieval prompts around generative-AI customer-support tools and a document-redaction exercise [1]. In design rounds, interviewers push on candidate generation, latency, freshness, cold start for new listings and monitoring [1].
- **Interviewer steering:** Treat a redirection as a hint and restate the narrower scope before going on [1].
- **Unverified prep advice:** This advice comes from search-index excerpts of Blind and PracHub [52][53] and could not be re-checked: "be very familiar with Airbnb business and product," because questions often reflect a real problem the team is working on.
- **How to use this chapter:** You will rarely be asked to recite a paper. You are more likely to get the paper's problem as an open design prompt, then a probe on whether you know how Airbnb approached it.

---

1. Search ranking at Airbnb (KDD 2019 / KDD 2020 deep learning line)

1.1 Search-ranking prompts

1. [Reported] Rank home search results when the guest gives no text query, only location, dates and guest count. Reported Sep 2026. [1][2][3]
    - What is the label: click, wishlist, booking request, or an uncancelled booking that the host accepted?
    - How do you handle the fact that most of a guest's searches never end in a booking?
    - What retrieval stage feeds the ranker, and how large is the candidate set?
    - Walk through candidate generation, offline evaluation, online evaluation and deployment. [1]
2. [Reported] Design a personalized search ranking system for Airbnb. [1][2]
    - [Likely] How does ranking Experiences differ from ranking Homes (smaller inventory, time-slot availability, cold start)? [11]
    - How would you surface new or high-quality Experiences that have little engagement history?

3. [Reported] In your ranking design, the interviewer pushes on latency, feature freshness, cold start for new listings and monitoring. Which of these sets your architecture, and what would you give up first? [1]
4. [Likely] In Airbnb search ranking, how do you balance relevance, personalization and host fairness?
    - How would you measure fairness or exposure for hosts, and which guardrail would you set on it?
5. [Likely] Compare a factorization machine with an embedding + DNN ranker for this problem. Which would you ship, and why? [52]
    - Explain the tradeoffs in interaction modeling, serving latency and debuggability.
6. [Likely] Your new ranker improves offline NDCG, but the online A/B test is flat or negative. What do you do? [1][52]
    - Possible causes include position bias in training logs, label leakage, distribution shift, a metric mismatch with bookings, and set-level effects.
7. [Likely] How do you trade off precision, recall, latency and infrastructure cost in a production ranking system? [52]

1.2 Moving from GBDT to neural ranking ("Applying Deep Learning to Airbnb Search")

8. [Likely] Airbnb's GBDT search ranker plateaued, and the team moved to neural networks [8]. When is that a good decision, and how would you de-risk it?
    - Why start with a single-hidden-layer network to validate the pipeline before trying deeper models?
    - Which feature-engineering habits from GBDTs (raw counts, unnormalized prices) hurt neural nets, and how do you fix them?
9. [Likely] Airbnb reported that listing-ID embeddings overfit badly in search ranking [8]. Why would listing IDs overfit in a marketplace where each listing books only a few times a year?
    - What features would you use in place of high-cardinality IDs?
10. [Likely] A listing can be booked by only one guest per night. How would you build a "booking probability" label from search logs under that constraint?
    - How do you deal with listings that were unavailable, or that the host rejected?
11. [Likely] How do you check whether a neural ranker learned the expected relationship with price, for example a preference for cheaper listings when everything else is equal?
    - What would you do if partial-dependence plots showed the model preferring more expensive listings?
12. [Likely] Airbnb search is a two-sided marketplace. How would you include host preferences (acceptance, rejection, Instant Book) in the ranking objective?

1.3 Position bias, cold start and architecture ("Improving Deep Learning for Airbnb Search")

13. [Likely] Explain how you would correct for position bias in training data from search logs. [9]
    - How does a shallow position "tower" whose logit is added to the relevance tower work, and how do you serve without it?

    - What assumption about factorizing click probability does this rely on?
    - When would you use dropout on the position feature instead?
14. [Likely] New Airbnb listings have no engagement history. How do you rank them fairly without flooding results with untested inventory? [9]
    - How would you predict engagement features for a new listing from its attributes and nearby listings?
    - How do you A/B test a cold-start change when its effects show up on the supply side?
15. [Likely] Airbnb found that a deeper or more complex network did not automatically win [9]. How do you decide when to stop increasing model capacity?
16. [Likely] How would you design a two-tower "query/guest" vs "listing" architecture for ranking, and what is lost compared with a full cross-feature network?

1.4 Diversity and set-level ranking

17. [Likely] Ranking each listing by its own booking probability ignores how similar the listings are to each other. How would you diversify Airbnb search results? [12][13]
    - How do you define similarity: price, location, room type, or a learned embedding?
    - Why might diversity optimization break interleaving-based evaluation? [18][37]
18. [Likely] How does a set-level (listwise or contextual) ranker differ from a pointwise one, and how would you serve it within latency budgets?

1.5 Multi-task and multi-objective ranking (Journey Ranker, KDD 2023; distillation, KDD 2024)

19. [Likely] Guests research for days or weeks before booking, and hosts can reject or cancel. How would you design a ranker that uses intermediate "milestones" such as clicks, wishlists, contact host and booking request? [28]
    - How do you treat negative milestones such as host rejection or guest cancellation?
    - How would you use guest state (early vs late in the journey) and query context as inputs?
    - How would you make the architecture modular so that other surfaces (email, Experiences, recommendations) can reuse it?
20. [Likely] Search ranking must maximize bookings while keeping cancellations, low reviews and customer-support contacts down. How would you combine these objectives? [29]
    - What are the drawbacks of a hand-tuned weighted sum of task heads?
    - How would distilling several single-objective "teacher" models into one "student" ranker work?
    - How do you add a non-differentiable business objective?
    - How do you show that the secondary-objective constraints still hold after launch?
21. [Likely] How would you choose the primary metric for a multi-task ranker: uncancelled bookings, booking value, or something else?

1.6 Maps, home page and split stays

22. [Likely] On a map there is no ranked list, and attention does not decay by rank position. How would you evaluate and train ranking for map pins? [30]
    - Why is NDCG a poor fit for maps, and what attention model (uniform, tiered pins, spatial decay) would you use?
    - How would you decide which listings get full price pins and which get mini-pins?
23. [Likely] How should list ranking and map results stay consistent when a guest pans or zooms?
24. [Reported] Design the Airbnb home page: search, availability and ranking working together. Reported Mar 2026. [2]
    - Where does availability filtering happen relative to retrieval and ranking, and how stale can availability be?
    - How do you rank home-page modules or carousels against each other, not only listings within a module?
25. [Likely] When no single listing is free for all of a guest's dates, Airbnb can suggest a split stay across two listings. How would you rank listing pairs for that? A split-stay combination problem appears in the coding pool [2].
    - How do you score a pair (distance between the listings, combined price, quality of each) and keep the candidate space tractable?

---

2. Retrieval, location and semantic search

26. [Likely] Design the first embedding-based retrieval (EBR) stage for Airbnb search, which narrows eligible homes before the expensive ranker runs. [31]
    - How do you build training pairs from search sessions, and which negatives do you use (random, same-market, or hard negatives from booked-vs-viewed)?
    - Why compute the listing tower offline daily and the query tower online?
    - How would you choose an ANN index (IVF vs HNSW) given that availability and price filters change constantly?
    - How do you combine EBR with hard filters such as dates, guest count and price?
27. [Likely] Airbnb reported that EBR increased bookings and also cut compute [31]. How could a retrieval change save compute and improve conversion at the same time?
28. [Reported] Design semantic search over user embeddings. Reported Apr 2026. [2]
    - What does a user embedding encode (stays, searches, wishlists, reviews), and how is it refreshed?
    - What is the query: a free-text description, another user, or a listing? How does that change the index design?
    - How do you evaluate semantic relevance when there is no clean labeled set?
29. [Likely] For a query like "Lake Tahoe," which geographic areas should search retrieve from? How would you move from heuristic bounding boxes to a learned approach? [32]
    - How do you evaluate location recall, meaning whether the area included the listing the guest eventually booked?
    - Why might you use reinforcement learning or bandit exploration for location retrieval?
30. [Likely] Suppose you represent the world as discrete location cells for retrieval. How would you do it, and what tradeoff do you face between cell size, precision and recall?

[32]
31. [Likely] How would you support flexible-date or "anywhere" searches, where there is no fixed location or date?

---

3. Embeddings, content understanding and recommendations

32. [Likely] Walk through how you would learn listing embeddings from click sessions with a skip-gram style model. [10][33]
    - Why use the booked listing as a global context in every session window?
    - Why sample negatives from the same market as well as at random?
    - How would you check embedding quality offline (same-market neighbors, similar price, similar listing type)?
33. [Likely] Bookings are too sparse per user and per listing to learn ID embeddings directly. How did Airbnb's user-type and listing-type embeddings address this, and how would you build them? [10]
    - How do you put host rejections into the embedding space?
34. [Reported] Design Airbnb listing recommendation. [5]
    - [Likely] Make it concrete as the "similar listings" carousel on a listing page [10] [33]. What offline metric and what online A/B metric would you use?
    - How would you handle a new listing with no clicks?
35. [Likely] How would you use short-term (in-session) and long-term (past bookings) interest embeddings as real-time ranking features?
    - What latency and freshness requirements apply, and where do the session features come from?
36. [Likely] Airbnb trained a two-tower model on guest engagement to produce listing embeddings for host-side and similar-listing surfaces [33]. How do you correct sampling bias in in-batch negatives?
37. [Likely] How would you build one multimodal embedding per listing from many photos and text fields, without just averaging them? [34]
    - How would you use LLMs or CLIP-style models to align text and images?
    - How would you compress the embedding (PCA, quantization) to fit a serving memory budget?
    - How does this embedding help with zero-shot free-text search and cold-start listings?
38. [Likely] Design a room-type classifier for listing photos (bedroom, kitchen, bathroom) and its labeling pipeline. Airbnb's Photo Tour depends on this kind of classification. [20]
    - How would multi-task learning, ensembling or distillation improve accuracy within latency limits?
39. [Reported] Choose a listing's cover photo so that it raises bookings, including for new listings with no engagement data. [1]
    - Do you predict a photo-quality score, run a bandit over candidate photos, or rank photos by expected click-through? What is the label?
    - How do you avoid optimizing for click-bait photos that lead to worse reviews?

    - How would you A/B test this when the treatment changes what other guests see in search?
40. [Likely] Design amenity detection from listing photos, and use it to check that listings really have the amenities they claim. [21]
    - How do you label at scale, and what precision do you need before showing a detected amenity to guests?
41. [Likely] Design Airbnb Categories (collections such as "Treehouses" or "Amazing pools"), where ML proposes listings and humans review them. [24]
    - How do you choose which candidates go to human review, and how do reviewer decisions retrain the model?
    - How do you rank inside a category versus across categories?
42. [Likely] How would you build a knowledge graph of places, Experiences and listing attributes, and use it in search or recommendations? [23]

---

4. Listing quality, pricing, value and trust models

43. [Reported] Design a system that finds high-quality listings (asked at the phone screen in one MLE loop). [1]
    - How do you define "quality" without circular reliance on bookings: reviews, host behavior, photos, cancellations?
    - How would the score feed search, badges or curated collections?
44. [Reported] Design a model that predicts or recommends the nightly price for an Airbnb listing, helping hosts set prices from market demand. [4][5]
    - Listings are not identical products, so how do you estimate demand without a clean demand curve? [35]
    - How would a "booking probability, then price strategy" two-stage design work, and why might you use a custom loss with an asymmetric penalty instead of MSE? [35]
    - How do you evaluate price suggestions when you never see what would have happened at a different price?
    - How do you handle hosts who do not accept the suggestion (selection bias in adoption)?
45. [Reported] Estimate a property's lifetime value. First agree with the interviewer on what lifetime value means here. Reported May 2026. [1][2][3]
    - Would you frame it as regression or as bucketized classification, and why might bucketizing be the better choice for steering decisions? [1]
    - What horizon and discounting do you use, and how do you handle censoring for new listings? [25]
    - Which teams would consume the estimate (host acquisition, onboarding, support priority), and how would you check that it changes their decisions?
46. [Reported] Design a system that calculates customer (guest) lifetime value and builds its ML pipeline. [4]
    - How does guest LTV differ from listing LTV in labels, horizon and use?
    - How would you use it to pick users for a promotional campaign and measure incremental impact? [32]

47. [Reported] Design relisting detection: catch a host who deletes a listing and relists the same property, for example to shed bad reviews. Show how the outputs show up in online metrics. [1][2][3]
    - How would geohashing help generate candidate pairs, and how do you handle nearby but distinct units in the same building? [1]
    - Which signals would you combine (photo embeddings, text similarity, host/device/payment links)?
    - What action follows a match (block, merge reviews, manual review), and how do you measure false positives?
48. [Likely] Design a "family friendly" listing classifier. [52]
    - Where do the labels come from (amenities, reviews, host text, photos), and how noisy are they?
    - How would you use an LLM or vision-language model to bootstrap labels, and how do you audit it?
49. [Likely] Design a model that predicts guest or host cancellations at booking time.
    - How would that prediction be used: in ranking, in host tools, or in support?
50. [Likely] Design a real-time booking-time risk model for offline risks such as property damage or unauthorized parties. [36]
    - How do you set the decision threshold when false positives block legitimate guests?
    - How do you get labels when most incidents are never reported?
    - How do you measure the impact of a risk model in an A/B test without letting fraud through in control?
51. [Likely] Design a system that detects fake listings or scam messages using graph signals, embeddings and device IDs. [36]
52. [Likely] How would a guest-host match quality model feed into both search ranking and Instant Book eligibility?
53. [Likely] Forecast vacant nights or occupancy for each listing so that host tools can suggest price or availability changes. "Average Vacant Days" appears as an Airbnb SQL prompt [2].
    - How do you handle listings the host blocks on purpose, which look the same as nights with no demand?

---

5. ML platform: Bighead, Zipline, Chronon

5.1 Bighead (end-to-end ML platform)

54. [Likely] Airbnb built Bighead because shipping a model took weeks to months [38]. If you joined a team without a unified ML platform, which component would you build first, and why?
    - How would you standardize the path from notebook to production?
    - How do you make training reproducible: environment pinning with Docker, data snapshots, versioned features?
55. [Likely] How would you design model serving for a search ranker that scores hundreds of listings per request under a strict p99 latency budget?
    - What do you batch, cache or precompute?

    - How do you safely roll out a new model version (shadow, canary, A/B)?
56. [Likely] What should a production ML monitoring stack watch for an Airbnb ranker (feature drift, prediction drift, calibration, business KPIs)? [1]
57. [Likely] How would you review an ML pipeline pull request for correctness and training/serving consistency? A multi-PR code-review round is reported in Airbnb SWE loops (Jun 2026) [2], and one unverified post describes it for an ML-platform role [54].

5.2 Zipline and Chronon (feature platform)

58. [Likely] What is training/serving skew, and how does a "define once, compute offline and online" feature platform such as Chronon prevent it? Airbnb postings stress training/serving skew [1]. [6][19]
    - What does point-in-time correctness mean for a feature like "number of views of this listing in the last 7 days"?
    - What goes wrong if you join today's snapshot to last month's labels?
59. [Likely] Explain how a Chronon `GroupBy` (windowed aggregations such as SUM, COUNT, AVERAGE keyed by an entity) and a `Join` (a left event source plus several GroupBys) produce a point-in-time training set. [6]
    - How do you backfill a new feature over a year of history efficiently, without snapshotting every timestamp? [6][19]
    - How do batch and streaming paths combine to serve a fresh value online?
60. [Likely] Chronon measures online/offline consistency by logging online fetches, replaying them as a backfill and comparing the values [6]. How would you set alert thresholds on that metric, and what would you do when it drifts?
61. [Likely] Design the real-time feature pipeline that supplies "listings this guest viewed in the current session" to the search ranker within a few hundred milliseconds.
    - Which store (key-value store, in-memory cache) do you use, and what happens on a cache miss?
62. [Likely] Chronon is open source, and its repository lists other companies such as Stripe among its adopters [6]. What tradeoffs would you weigh between an in-house feature platform and an off-the-shelf feature store?
63. [Likely] How would you reuse a feature platform for LLM applications, for example to fetch real-time user and reservation context for a support agent? [39]

---

6. Metrics and experimentation context: Minerva, ERF, interleaving

64. [Likely] Airbnb built Minerva so that a metric is defined once and used for analytics, experimentation and reporting [40][41]. Why does that matter for ML launch decisions?
    - What goes wrong when Data Science and Finance compute "bookings" with different logic?
    - How would you add a new metric for your model launch without breaking downstream consumers?

65. [Reported] Design the data model behind cumulative booking metrics, including each user's first and last booking times. Reported Sep 2026. [2]
    - [Likely] How would this metric layer serve both experiment readouts and dashboards consistently (the Minerva problem)? [40]
66. [Reported] Design an A/B testing platform. Reported Apr 2026. [2]
    - [Likely] Using Airbnb's ERF as the reference [14], how do you join assignment logs with metric events and compute results for thousands of metric/experiment pairs?
    - At what unit do you randomize for a search ranker: user, device or search session? What happens with logged-out users?
    - How do you deal with users who switch devices between searching and booking?
67. [Reported] Design robust primary and guardrail metrics for an experiment. [2]
    - [Likely] Airbnb escalates a launch when it moves another team's metrics, for example a test that raised bookings but lowered review ratings [15]. Which guardrails would you set for a ranking launch: cancellations, review ratings, CS contacts, host earnings, latency?
    - How do you set an escalation threshold, and what is a "power guardrail"? [15]
68. [Reported] Design an A/B test that uses causal inference. [2]
    - [Likely] When you cannot randomize (a policy change in one market, a host-side launch), how would an ML-based counterfactual estimator such as Airbnb's ACE work, and how do you check it for bias? [17]
69. [Reported] Design an experiment for a listing badge (for example a "fast Wi-Fi" badge) that accounts for network effects. [2]
    - Why does a listing-level badge violate SUTVA when guests compare badged and unbadged listings in the same search?
    - Would you randomize by listing, by guest, by market, or with a two-sided (bipartite) design? What bias does each leave?
70. [Likely] Bookings have high variance and long latency. How would you make ranking experiments more sensitive? [42]
    - How could a model of leading indicators (a surrogate or value function) give credit for the eventual booking to earlier steps?
    - What are the risks of optimizing a surrogate metric instead of bookings?
71. [Likely] Airbnb puts a monetary "future value" on actions so that teams can compare tradeoffs on a common scale [16]. How would you estimate the long-term value of a cancellation, a bad review or a support contact, and use it in a launch decision?
72. [Likely] Airbnb uses interleaving to screen search-ranking candidates on a small share of traffic before A/B testing [18][37]. Explain team-draft interleaving and how you would attribute bookings to one ranker.
    - When is interleaving invalid (set-level diversity, results shared with the map)?
    - What is online counterfactual evaluation, and when would you use it instead?
    - How do you show that interleaving outcomes predict A/B outcomes?
73. [Likely] In a two-sided marketplace, treatment-group and control-group guests compete for the same listing-nights, so a ranking test can move bookings from control to treatment. How do you detect and correct this interference?
    - When would you use market-level or time-split (switchback) randomization?
74. [Likely] A model launch helps guests but lowers host earnings in some markets. How do you decide whether to ship?

7. 2024-2026 Airbnb AI: customer-service LLMs

75. [Reported] Design retrieval and ranking for a generative-AI tool that helps customer-support staff answer guest and host issues (recent senior loops). [1]
    - What does the knowledge base contain (policies, help articles, past resolved tickets), and how do you chunk and index it?
    - How do you evaluate retrieval quality and answer faithfulness before agents see the answers?
    - How do you stop the system from citing an outdated or region-specific policy?
76. [Reported] Generate customer-support instructions from company policies. Reported Sep 2026 in senior loops. [1][2]
    - [Likely] How would you turn policies and workflows into a format an LLM can follow reliably, such as an intent-context-action structure? [46]
    - How would you generate synthetic fine-tuning or evaluation data from those instructions?
    - How do you detect when a policy change makes generated instructions stale?
77. [Reported] Redact sensitive information (PII, contact details, payment data) from documents or messages. In recent senior loops this was reported as a coding exercise; here it is asked as a design question. [1]
    - How do you trade off recall (missing PII) against precision (over-redacting useful content)?
    - Would you use regex, NER, an LLM, or a cascade, and why? How does Airbnb-scale automated data classification inform this? [27]
78. [Reported] Design a customer-support ticketing and routing system. Reported Jun 2026 as a system design prompt. [2]
    - [Likely] Where does ML enter: contact-reason classification, priority and urgency scoring, agent-skill matching, deflection to self-serve? [22]
    - How do you measure whether a routing model shortened resolution time and did not just move work between queues?
79. [Likely] Airbnb's support agent was reported to be built on many models (13, per the CEO) and to resolve a large share of issues without a human [43][44]. How would you architect a multi-model support agent?
    - Which components would you expect: intent/contact-reason classifier, retriever, policy-reasoning LLM, tool/action executor, guardrail model, escalation model?
    - Which model do you route each sub-task to, and how do you balance cost, latency and quality?
80. [Likely] Airbnb's automation platform evolved from predefined conversational workflows [26] to a version that combines LLM reasoning (chain of thought, tool selection) with those workflows plus a guardrails layer [39]. When should an action (refunds, cancellations) be handled by a deterministic workflow, and when by an LLM?
    - How do you design guardrails against off-topic, unsafe or policy-violating responses?
    - How do you handle hallucination and latency?
81. [Likely] What is the right primary metric for an AI support agent: self-resolution rate, CSAT, reduction in contacts to humans, repeat-contact rate, or refund accuracy? How

would you A/B test it?
    - Why are "share of tickets handled by AI" and "reduction in people needing a human" not the same metric? [43][44]
82. [Likely] Design the voice support pipeline: speech-to-text, contact-reason detection, self-serve article retrieval and escalation-intent detection. [45]
    - What latency budget applies to intent detection on a live call?
    - How do you deal with transcription errors before classification?
83. [Likely] Airbnb's earlier work fine-tuned text-generation models (for example MT5) for help-center content recommendation, agent QA assistance and chatbot paraphrasing [47]. How would you redo that system with today's LLMs, and what would you keep?
84. [Likely] How would you fine-tune a support model (SFT, preference optimization, distillation) on millions of past agent conversations without learning bad agent behavior? [48]
    - How do you filter or weight the transcripts, and how do you protect PII in the training data?
85. [Likely] How would you build an offline evaluation harness for a support agent, with LLM-as-judge, policy-compliance checks and human review sampling? [1]
    - How do you calibrate the LLM judge against expert agents?
86. [Likely] A guest says the listing was not as described and asks for a refund. Walk through how the AI agent should gather evidence, apply policy and decide whether to escalate.
87. [Likely] Design intent classification for guest-host messages (check-in questions, pet requests, cancellations, off-platform payment attempts). [22]
    - How do you discover new intents without labels, and how do you route a high-risk intent to trust and safety?

---

8. 2025-2026 Airbnb AI: natural-language and AI search

88. [Likely] Airbnb has rolled out natural-language / AI-assisted search across Homes, Experiences and Services, which accepts text or voice and creates filters on the fly [49][50]. Design this system end to end.
    - How do you parse a query like "quiet cabin near a lake for 6 with a hot tub, under $300" into structured filters and soft preferences?
    - Which constraints are hard filters and which are ranking signals?
    - How do you keep classic filters and AI-generated filters consistent?
89. [Likely] How would you combine LLM query understanding with the existing EBR and Journey Ranker stack, rather than replace it?
    - Where does free-text-to-listing matching happen: retrieval (multimodal embeddings), reranking, or both? [34]
90. [Likely] AI search usage is opt-in and selection-biased. How would you A/B test it against classic search?
    - Which metrics separate "more engagement" from "more bookings"?
91. [Likely] How do you prevent an LLM-powered search from claiming that listings have amenities they lack? [21]
92. [Likely] Chesky has described an app that "doesn't just search for you; it understands you" [50]. How would you use a guest's past stays and reviews for personalization in

AI search while respecting privacy?
93. [Likely] How would you handle LLM latency and cost for search at Airbnb's query volume (caching, small models, distillation, query-class routing)?
94. [Likely] Airbnb is reportedly building an AI agent to keep travel discovery on its own platform rather than in third-party AI assistants [51]. What ML and product risks follow from that strategy?

---

9. Cross-cutting senior-level probes using Airbnb context

95. [Reported] Walk through one ML project end to end: the problem, your decisions, the trade-offs, the scope and the measured impact. Then the interviewer asks about milestones and a scope conflict. [1][4]
    - [Likely] Be ready for the interviewer to drop the project and ask an ML design question instead [5]. How would you link your project to an Airbnb problem?
96. [Likely] Which Airbnb ML paper or blog post do you know best? What would you have done differently, and what would you try next?
97. [Likely] Pick one Airbnb surface (search, similar listings, pricing, support). In your view, what is the current bottleneck in its ML system, and what is your 6-month roadmap?
98. [Likely] Airbnb's published ranking gains are often small, such as about 0.3% booking conversion [29][31]. How do you decide whether an improvement that small is real and worth the complexity?
    - How do you account for long-term effects and novelty?
99. [Likely] Airbnb inventory is seasonal, and travel demand changed sharply in 2020-2021. How would you make ranking and pricing models robust to large distribution shifts?
100. [Likely] How do you prioritize between model improvements, feature-platform investment and experimentation-speed investment such as interleaving?
101. [Likely] How would you design an ML system that serves Homes, Experiences and Services, which have very different inventory and booking patterns? Do you share models or separate them?
102. [Likely] Airbnb's audience-expansion work uses extreme classification for marketing in a two-sided marketplace [32]. How would you choose which users to target for a promotional email, and how would you measure incremental impact?
103. [Likely] Describe a time your offline evaluation and online results disagreed. How did you diagnose it, and what did you change in your process? This is the project-deep-dive version of question 6. [1]
104. [Likely] An Airbnb ranker underperforms in a single market, such as a new country. How do you debug it? - What do you check first: feature coverage, label sparsity, currency normalization, or local inventory differences?
105. [Likely] A host claims their listing "disappeared" from search. How would you explain the ranking change to them? - What tooling (ranking dashboards, explanations) would you build for internal teams? [11]

References

Status key: **verified** = fetched and read on 2026-10-14; **URL confirmed** = title, URL and year match the applied-ml index [7], but the page was not opened; **unverified** = the host could not be reached in this review, and the entry is carried over from the previous version of this chapter.

1. "Airbnb Machine Learning Engineer Interview Process" (TrueInterview-derived guide, 3 candidate reports, mostly 2026 Q1-Q2: search ranking, property lifetime value, relisting with geohashing, cover photo with cold start, high-quality listings, generative-AI support retrieval and ranking, document redaction, policy-to-instructions). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Verified. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb" question index with report dates (Rank Home Search Results Without a Text Query, Sep 2026; Generate Customer-Support Instructions from Company Policies, Sep 2026; Model Cumulative Booking Metrics, Sep 2026; Design Customer Support Ticketing System, Jun 2026; Code Review Multi-PR, Jun 2026; Listing Lifetime Value, May 2026; Design User Embedding Semantic Search, Apr 2026; Design an A/B Testing Platform, Apr 2026; Home Page Search + Availability + Ranking, Mar 2026; undated: personalized search ranking, relisting detection, primary and guardrail metrics, A/B test with causal inference, network-aware Wi-Fi badge experiment). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Verified. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
3. "Airbnb interview process" (MLE candidates get one or two ML system design rounds on search ranking, a property's lifetime value, recommendations, or relisting detection with geohashing; "Problem and label before model"). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Verified. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. 1point3acres post #40, English mirror (MLE loop: project deep dive; ML design on customer lifetime value and on helping hosts set prices; coding LC 1298). FlowSight/coding, GitHub. Verified. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
5. 1point3acres post #79, English mirror (MLE loop: two ML design rounds on listing recommendation and price prediction; an "ML experience" round that turned into an unplanned ML design question; ConnectN coding). FlowSight/coding, GitHub. Verified. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
6. "Chronon: a data platform for AI/ML" (GroupBy and Join, point-in-time backfills, online/offline consistency measurement, adopters list). airbnb/chronon, GitHub. Verified. https://github.com/airbnb/chronon
7. Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production." eugeneyan/applied-ml, GitHub. Verified; used to confirm [8]-[27]. https://github.com/eugeneyan/applied-ml
8. Haldar et al., "Applying Deep Learning To Airbnb Search," arXiv 1810.09591 (KDD 2019). URL confirmed. https://arxiv.org/abs/1810.09591
9. Haldar et al., "Improving Deep Learning For Airbnb Search," arXiv 2002.05515 (KDD 2020). URL confirmed. https://arxiv.org/abs/2002.05515
10. Grbovic, "Listing Embeddings in Search Ranking," Airbnb Tech Blog, 2018 (companion to Grbovic and Cheng, "Real-time Personalization using Embeddings for

Search Ranking at Airbnb," KDD 2018). URL confirmed. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
11. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
12. Abdool et al., "Managing Diversity in Airbnb Search," arXiv 2004.02621, 2020. URL confirmed. https://arxiv.org/abs/2004.02621
13. "Learning To Rank Diversely," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621 . Paper version, arXiv 2210.07774, unverified: https://arxiv.org/abs/2210.07774
14. "Scaling Airbnb's Experimentation Platform" (ERF), Airbnb Tech Blog, 2017. URL confirmed. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166
15. "Designing Experimentation Guardrails," Airbnb Tech Blog, 2021. URL confirmed. https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669
16. "How Airbnb Measures Future Value to Standardize Tradeoffs," Airbnb Tech Blog, 2021. URL confirmed. https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5
17. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512
18. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
19. Simha, "Zipline: Airbnb's Machine Learning Data Management Platform" (2018) and "Zipline - A Declarative Feature Engineering Framework" (2020), conference talks. URL confirmed. https://www.youtube.com/watch?v=Tg5VEMEsC-0 ; https://www.youtube.com/watch?v=LjcKCm0G_OY
20. "Categorizing Listing Photos at Airbnb," Airbnb Tech Blog, 2018. URL confirmed. https://medium.com/airbnb-engineering/categorizing-listing-photos-at-airbnb-f9483f3ab7e3
21. "Amenity Detection and Beyond — New Frontiers of Computer Vision at Airbnb," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/amenity-detection-and-beyond-new-frontiers-of-computer-vision-at-airbnb-144a4441b72e
22. "Discovering and Classifying In-app Message Intent at Airbnb," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/discovering-and-classifying-in-app-message-intent-at-airbnb-6a55f5400a0c
23. "Contextualizing Airbnb by Building Knowledge Graph," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/contextualizing-airbnb-by-building-knowledge-graph-b7077e268d5a
24. "Building Airbnb Categories with ML and Human-in-the-Loop," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb

25. "Using Machine Learning to Predict Value of Homes On Airbnb," Airbnb Tech Blog, 2017. URL confirmed. https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d
26. "Intelligent Automation Platform: Empowering Conversational AI and Beyond at Airbnb," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/intelligent-automation-platform-empowering-conversational-ai-and-beyond-at-airbnb-869c44833ff2
27. "Automating Data Protection at Scale, Part 1," Airbnb Tech Blog, 2021. URL confirmed. https://medium.com/airbnb-engineering/automating-data-protection-at-scale-part-1-c74909328e08
28. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning" (Journey Ranker), arXiv 2305.18431 (KDD 2023). Unverified. https://arxiv.org/abs/2305.18431
29. Tang, Gao, He and Katariya, "Multi-objective Learning to Rank by Model Distillation," arXiv 2407.07181 (KDD 2024). Unverified. https://arxiv.org/abs/2407.07181
30. Haldar et al., "Learning to Rank for Maps at Airbnb," arXiv 2407.00091 (KDD 2024). Unverified. https://arxiv.org/abs/2407.00091
31. "Embedding-Based Retrieval for Airbnb Search," Airbnb Tech Blog, and "Applying Embedding-Based Retrieval to Airbnb Search," arXiv 2601.06873, 2026. Unverified. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/ ; https://arxiv.org/abs/2601.06873
32. "Academic Publications: Airbnb Tech 2025 Year in Review" (location retrieval, audience expansion, interleaving), Airbnb Tech Blog, 2025, and "Transforming Location Retrieval at Airbnb," arXiv 2408.13399. Unverified. https://airbnb.tech/infrastructure/academic-publications-airbnb-tech-2025-year-in-review/ ; https://arxiv.org/abs/2408.13399
33. "Learning and Applying Airbnb Listing Embeddings" (KDD 2024 workshop paper), Airbnb Tech. Unverified. https://airbnb.tech/wp-content/uploads/sites/19/2024/12/Learning-and-Applying-Airbnb-Listing-Embeddings-.pdf
34. Guy, Grbovic, Tan and Zhao, "BiListing: Modality Alignment for Listings," arXiv 2508.20396 (CIKM 2025). Unverified. https://arxiv.org/abs/2508.20396
35. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing," KDD 2018. Unverified. https://dl.acm.org/doi/10.1145/3219819.3219830
36. "Senior Machine Learning Engineer, Trust & Safety," Airbnb Careers (job posting; may have closed). Unverified. https://careers.airbnb.com/positions/7185448/
37. Zhang et al., "Harnessing the Power of Interleaving and Counterfactual Evaluation for Airbnb Search Ranking," arXiv 2508.00751, 2025. Unverified. https://arxiv.org/abs/2508.00751
38. "Bighead: Airbnb's End-to-End Machine Learning Platform," Data Council talk. Unverified. https://datacouncil.ai/talks/bighead-airbnbs-end-to-end-machine-learning-platform
39. "Automation Platform v2: hybrid LLM conversational platform" (third-party summary of a 2024 Airbnb Tech Blog post), ZenML LLMOps database. Unverified. https://www.zenml.io/mlops-database/airbnb-chronon-internal-dataai-app-platform-conversational-ai-platform-automation-platform-v2-hybrid-llm-conversational
40. "Ep 1: Robert Chang on Scaling Metrics @ Airbnb" (Minerva), dbt Roundup. Unverified. https://roundup.getdbt.com/p/ep-1-robert-chang-on-scaling-metrics
41. "How did Airbnb achieve metric consistency at scale" (Minerva), Analytics India Magazine. Unverified. https://analyticsindiamag.com/ai-features/how-did-airbnb-achie

ve-model-metric-consistency-at-scale
42. Deng, Du and Matlin, "Continuous Attribution of Episodical Outcomes for More Efficient and Targeted Online Measurement," arXiv 2210.16373. Unverified. https://arxiv.org/abs/2210.16373
43. "AI Agents Handle 30% of Airbnb Customer Support Tickets as Company Expands Automation," CX Today, 2026. Unverified. https://www.cxtoday.com/?p=212503
44. "Airbnb Claims 'Best AI Self-Solve in All of Travel' After AI Resolves 40% of Customer Issues," CXM World, 2026. Unverified. https://cxm.world/customer-experience/airbnb-claims-best-ai-self-solve-in-all-of-travel-after-ai-resolves-40-of-customer-issues/
45. "Listening, Learning, and Helping at Scale: How Machine Learning Transforms Airbnb's Voice Support Experience," Airbnb Tech Blog. Unverified. https://airbnb.tech/ai-ml/listening-learning-and-helping-at-scale-how-machine-learning-transforms-airbnbs-voice-support-experience/
46. "LLM-Friendly Knowledge Representation for Customer Support," arXiv 2510.10331, 2025. Unverified; the venue claimed in the previous version (COLING 2025) did not fit the October 2025 arXiv ID and has been dropped. https://arxiv.org/abs/2510.10331
47. "LLM integration for customer support automation and enhancement" (third-party summary of Airbnb's "How AI Text Generation Models Are Reshaping Customer Support"), ZenML LLMOps database. Unverified. https://zenml.io/llmops-database/llm-integration-for-customer-support-automation-and-enhancement
48. "Principal Machine Learning Engineer - LLM Fine-tuning and Optimization," Airbnb Careers (job posting; may have closed). Unverified. https://careers.airbnb.com/positions/7955579/
49. "Airbnb Adds AI-Powered Search and Social Features," Let's Data Science, 2026. Unverified. https://letsdatascience.com/news/airbnb-adds-ai-powered-search-and-social-features-dd4ce7ea
50. "Airbnb's AI Strategy for 2026 Is Already Taking Shape," Rental Scale-Up, 2026. Unverified. https://www.rentalscaleup.com/airbnbs-ai-strategy-2026/
51. "Airbnb Is Building an AI Agent to Preserve Customer Traffic," AltexSoft, 2025. Unverified. https://www.altexsoft.com/travel-industry-news/airbnb-is-building-an-ai-agent-to-preserve-customer-traffic
52. "Airbnb Senior Machine Learning Engineer Interview Experience — Four ML System Design Rounds, Rejected on Calibration," PracHub, 2026. Unverified (host unreachable); its topics are used only as [Likely] context. https://prachub.com/interview-experiences/airbnb-senior-machine-learning-engineer-interview-experience-four-ml-system-design-rounds-rejected-on-calibration
53. "Airbnb senior MLE interview prep," Blind. Unverified (host unreachable). https://www.teamblind.com/post/airbnb-senior-mle-interview-prep-n1mdsd3b
54. "Interviewing for Airbnb ML platform role," Blind, 2025. Unverified (host unreachable). https://www.teamblind.com/post/interviewing-for-airbnb-ml-platform-role-8in7czso

10. Whiteboard-Ready ML System Design Diagrams: Search Ranking, Embeddings, Pricing, Trust, LLM Support Routing and AI Search

Research note (read first). Written on 2026-10-14 and reviewed the same day. The shared web-search budget for this run was already used up, so **no new search queries could run**, in the first pass or in the review. In the review, only GitHub (through raw.githubusercontent.com) could be reached. arxiv.org, medium.com, dl.acm.org, airbnb.tech, careers.airbnb.com, YouTube, zenml.io and letsdatascience.com were blocked by the proxy. trueinterview.io, prachub.com, glassdoor.com, interviewquery.com, igotanoffer.com and tryexponent.com did not resolve. The review re-fetched and re-read the candidate-report and official pages: the TrueInterview-derived Airbnb MLE loop summary [1], the dated Airbnb question index [2], the Airbnb interview-process page [3], two translated 1point3acres onsite posts [4][5], Airbnb's Chronon README [6], and the Airbnb Software Engineer process page [26], which is new in the review. It also scanned 91 other translated 1point3acres Airbnb posts from the same mirror; none of them contained an ML design prompt. Two curated indexes, applied-ml [7] and an ML case-study list [31], were fetched to confirm the titles and URLs of Airbnb papers and blog posts. A general ML-design template [8] was also read. Glassdoor, Blind, LeetCode Discuss, Reddit, IGotAnOffer, Exponent, InterviewQuery and Prepfully could not be searched or opened. Before the onsite, cross-check this chapter against those sites.

Review changes. Several [Reported] tags claimed more than the source says, and were retagged [Likely] or reworded: questions 1 (first probe), 36, 101 and 124. A job posting that could not be opened (reference 26 in the previous version) was replaced by the fetched team list in [1]. Two uncited and unreachable references were removed: a PracHub report and the Glassdoor page. The review added 30 questions, and the whole chapter was renumbered. Ten are [Reported] from the fetched index [2] and the process pages [1][3][26]. The rest are [Likely] and tied to confirmed Airbnb posts.

About the diagrams. No source reports that interviewers hand out diagrams. The diagrams below are **study scaffolds**: one plausible whiteboard layout per core Airbnb prompt. They are built from the reported round skeleton [1][3] and from Airbnb's public papers and posts [9]-[29]. They are **not** Airbnb's actual production architecture. Every latency number is an **illustrative budget to defend or change**, not a published figure. Use each diagram to practise drawing the boxes in the first 10 minutes. Then work through the questions underneath it, which are what interviewers probe.

How the tags work

Every question carries exactly one tag. The same scheme is used in chapters 01-09, so the whole pack reads the same way in the PDF.

- **[Reported] [n]**: a fetched source [n] says this prompt, or a close variant, was asked in an Airbnb interview. Most are MLE loops. [2] does not always give the role.
- **[Likely]**: a well-grounded inference from the Senior MLE role, the reported round structure and Airbnb's published systems. A citation next to [Likely] points to the paper or post behind the topic. It does **not** mean a candidate reported the question.
- **Follow-up probes** are the indented bullets under a question. They carry no tag of their own. A probe inherits its parent's tag only if the source names that probe; where a source does name a probe, it is cited inline.
- **Reference status** (see References): **fetched** = read on 2026-10-14; **URL confirmed** = title and URL match the applied-ML index [7] or the case-study index [31], but the page itself was not opened; **unverified** = carried over from earlier chapters, host unreachable. Check unverified links before relying on them.

Format notes

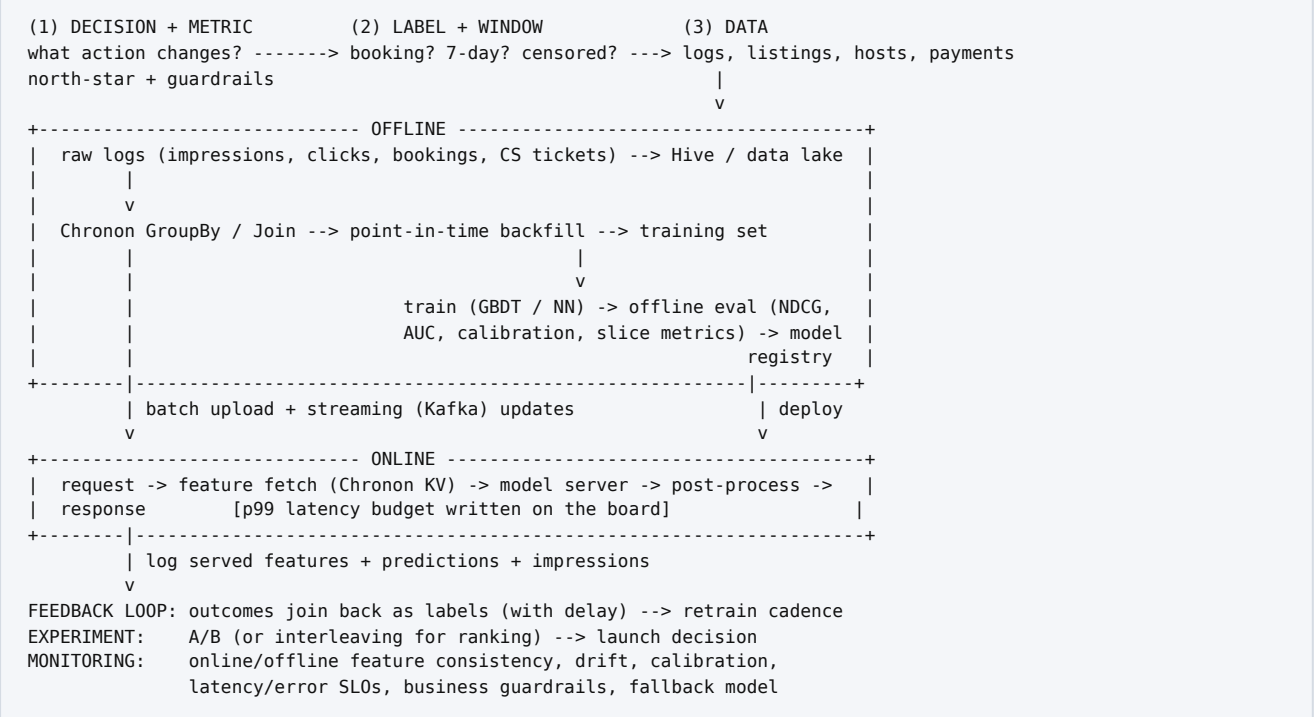
- **Number of rounds.** MLE candidates get "one or two ML system design rounds" in place of the standard system design round [3]. Design is "filtered by design depth, not by code" [1]. Two 1point3acres MLE loops each had two ML design rounds: customer LTV plus host pricing [4], and listing recommendation plus price prediction [5]. In [5], the scheduled "ML experience" round also turned into an unplanned ML design question.
- **Length and medium.** Around 45-60 minutes over video with a shared doc or whiteboard (chapter 03, from [1]). The rounds are "conversations without code, and the first minutes of scoping are graded" [1]. Being able to draw a clean block diagram fast frees time for depth.
- **Interviewer.** Usually a senior or staff MLE from an ML-heavy team such as Search, Pricing, Trust, Customer Support/GenAI or ML platform [Likely]. Interviewers steer hard. Reported examples: an LTV round moved from regression to bucketized classification, a relisting round went into geohashing, and search-ranking rounds "pressed on latency, freshness, and experimentation" [1].
- **What is graded** [1][3]: framing (the business decision and success metric) before any model; the label and its time window; concrete marketplace features; how offline metrics link to online results through an A/B test; and production behaviour: candidate generation, latency, freshness, cold start for new listings, and monitoring. "Problem and label before model" [3]. One ML rejection was blamed on "jumping straight to model ideas" [3].
- **Senior (G9) bar.** At Senior and above, design "carries more weight". A strong coding result does not offset a weak design, and a borderline design signal can add another design round [1]. An added late design round is "a warning sign" [3].
- **Reported prompts covered here:** end-to-end search ranking [1][3]; ranking home results without a text query [1][2]; home page search + availability + ranking [2]; personalized search ranking [2]; user-embedding semantic search [2]; listing

recommendation [5]; host pricing / price prediction [4][5]; relisting detection [1][2]; and generative-AI support retrieval and ranking, including turning company policies into customer-support instructions [1][2]. The fraud/ATO and natural-language AI search prompts are [Likely].

- **Adjacent prompts from the standard design round** [2][3][26], each reused below with an ML layer: a homepage backend "with ranking signals"; booking with availability search, pagination and indexing; a support ticketing system "with agent claim rules by language and location and first-response metrics"; a host dashboard for 100+ listings; ad-click event aggregation; and a time-and-geo activity query system. The base prompt was reported. The ML extension is this chapter's addition, and the tags say which part is which.

Diagram 0. The skeleton every diagram follows

Draw this first in any round. Then specialise the boxes. The flow follows the reported grading skeleton [1][3] and a standard nine-step template [8].



Questions on the skeleton

1. In the first five minutes, which business decision, success metric and label would you write on the board before drawing any model box? [Reported] [1][3]
   - You catch yourself naming model architectures before the label is agreed. How do you recover? [Likely] (One ML rejection was blamed on "jumping straight to model ideas" [3].)
   - How do you restate scope after the interviewer redirects you, so that the new scope is explicit? [Reported] [1]
2. Where in the diagram does point-in-time correctness matter, and what goes wrong if the training join uses today's feature values? [Likely] [6]

    - How would you measure training-serving skew once the model is live, not just prevent it? [Likely] [6]
3. Which boxes are batch, which are streaming, and which are request-time? How do you decide for each feature? [Likely] [6]
4. What do you log at serving time so that the next model can be trained from it without a fresh backfill? [Likely] [6]
5. Where do the offline metric and the online metric live in your diagram, and how do you show the link between them? [Reported] [1]
6. What is your fallback path if the model server or the feature store times out? Draw it. [Likely]

Diagram 1. Stays search ranking (end to end)

Reported as "end-to-end search ranking", with probes on latency, freshness and experimentation [1][3]. Also reported in the variants "rank home search results without a text query", "home page: search + availability + ranking" and "personalized search ranking" [2]. The layout is drawn from Airbnb's search papers: the deep-learning ranker [9] [10], diversity re-ranking [12][13], the multi-stage journey ranker [21], maps ranking [22], embedding-based retrieval [20] and location retrieval [29].

```
GUEST QUERY: location text / map box, dates, guests, filters, (maybe nothing)
     |
     v
[Query understanding]  geocode + location retrieval model -> search area     ~10 ms
     |                 intent: flexible dates? category? group size?
     v
[Candidate retrieval]  (a) geo index: listings inside area                   ~30 ms
     |                 (b) ANN over listing embeddings (EBR two-tower)
     |                 HARD FILTERS: availability calendar for the dates,
     |                 guest count, price filter, instant-book, policy
     |                 -> 1k-10k candidates
     v
[Feature fetch]        Chronon online KV: listing aggregates (CTR, booking   ~15 ms
     |                 rate, reviews, host response), guest history, session
     |                 embeddings (streaming), query x listing crosses
     v
[Stage-1 ranker]       light GBDT / small NN -> top ~500                      ~20 ms
     v
[Stage-2 ranker]       deep NN (pairwise / multi-task: click, wishlist,      ~40 ms
     |                 book, host-accept, no-cancel), price as a feature
     v
[Re-rank]              diversity (location, price, type), business rules,   ~10 ms
     |                 new-listing exploration slots, map vs list layout
     v
RESULTS PAGE (list + map pins)              total p99 target: write it, e.g. <200 ms
     |
     v
[Logging]  impressions with position, served features, model version,
           experiment bucket -> Kafka -> Hive
     |
     v
LABELS: click / long view / wishlist / booking request / host accept /
        completed stay (delayed days-to-weeks) --> daily retrain
EXPERIMENT: interleaving to screen rankers fast -> A/B on bookings [15]
MONITORING: NDCG on logged searches, calibration per market, feature
           null rates, availability-filter misses, latency, booking guardrails
```

Framing, labels and retrieval

7. Design Airbnb stays search ranking end to end. Draw the full pipeline before going deep on any box. [Reported] [1][3]
    - Which box would you deep-dive first if you had only 20 minutes left, and why?
    - What is your single online success metric, and which guardrails sit beside it?
8. Rank home search results when the guest gives no text query (just location and dates, or nothing at all). How does the diagram change? [Reported] [2]
    - With no query, what does query understanding output, and what does the retrieval stage key on?
9. Design the home page as one system covering search, availability and ranking. Where does the availability check sit, and what does it cost if it sits in the wrong place? [Reported] [2]
    - If the availability calendar is eventually consistent, how do you avoid ranking a listing that was booked a minute ago?
    - Would you filter on availability before or after ANN retrieval? What happens to recall in each case?
10. Design a personalized search ranking system. Which boxes in the diagram carry personalization, and which stay global? [Reported] [2]
11. Why split retrieval from ranking at all, given that a city may have only a few thousand listings? [Likely] [20]
    - When does embedding-based retrieval beat a pure geo-filter, for example for flexible-location or category searches? [Likely] [20]
12. How do you train the two-tower retrieval model, and what are its negatives? [Likely] [20]
    - How do you keep the ANN index fresh when hosts change prices, photos and calendars many times a day?
13. How would you build the query-understanding step that maps "near the beach in Portugal" to a search area? [Likely] [29]
    - What is the label for location retrieval, and how do you evaluate it offline?

Ranker, re-rank and latency

14. Would you use GBDT or a neural network as the main ranker? Defend it in the context of Airbnb's own move from GBDT to deep NN. [Likely] [9][10]
    - What would make you keep a GBDT stage in the cascade?
15. Write the latency budget on the board for each stage. Which stage do you cut first if p99 doubles? [Reported] [1]
    - What do you precompute offline, and what must run at request time?
    - How do you handle a feature-store timeout inside the latency budget?
16. How does the ranker stay fresh: real-time session features, daily retraining, or both? [Reported] [1]
    - Which features must be streaming, and which are fine as daily batch?
17. How do you combine click, wishlist, booking and host-acceptance signals: one weighted label, a multi-task model, or separate stages? [Likely] [21]
18. Where do you put diversity: in the loss, in a re-rank stage, or both? What does diversity mean for Airbnb results? [Likely] [12][13]
    - How do you measure whether diversity helped bookings rather than only changing the page?

19. How do new listings with no history get exposure without hurting booking rate? Where does exploration sit in the diagram? [Reported] [1]
20. How does ranking pins on the map differ from ranking the list, and does it need its own model? [Likely] [22]
21. Price is both a feature and something the guest filters on. How do you stop the ranker from simply learning "cheaper is better"? [Likely] [9][10]
22. You are moving the production ranker from GBDT to a neural network. Draw the migration path (shadow scoring, feature parity checks, interleaving, ramp) and say how you would stop the first NN from losing to the tuned GBDT. [Likely] [9][42]

Adjacent prompts on the same diagram

23. Design the Airbnb homepage backend "with ranking signals". Show where the ML ranker sits relative to the backend fan-out, the cache and the page-module layout. [Reported] [26]

- Which homepage sections can be precomputed per guest offline, and which must be ranked per request?

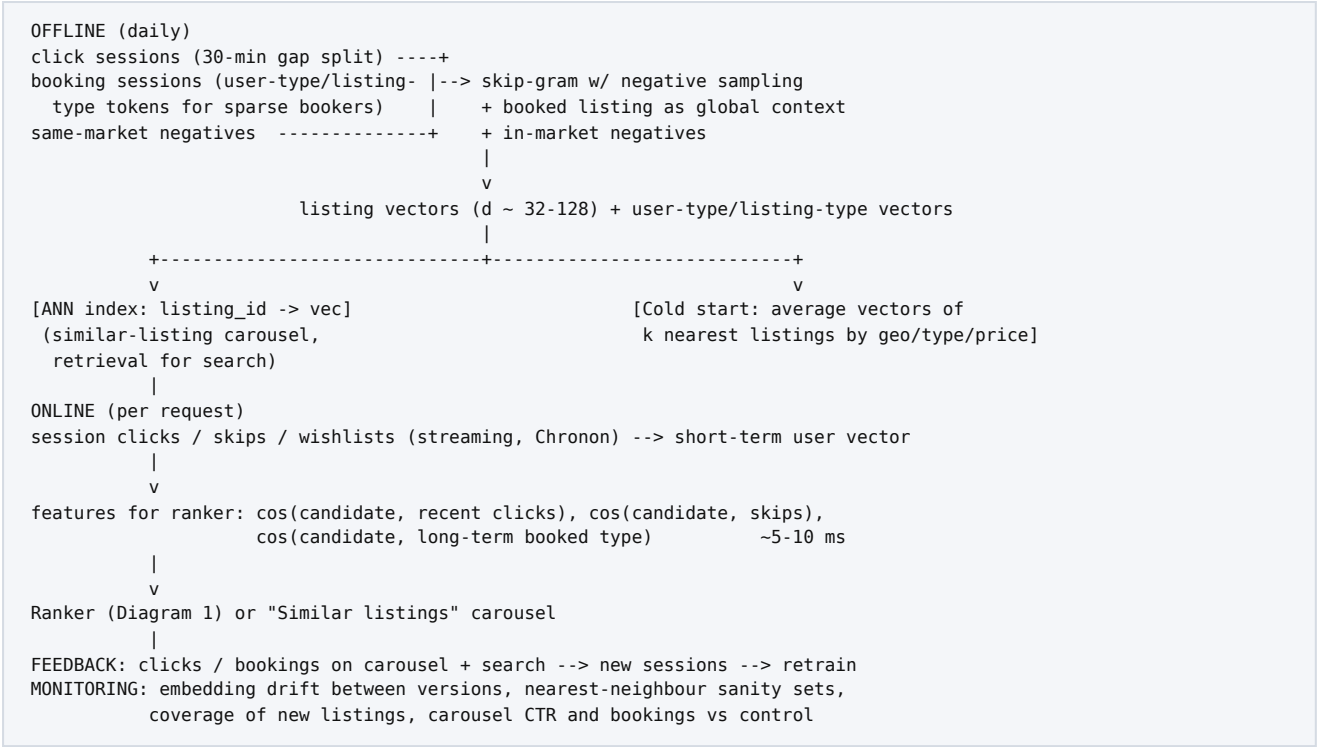
24. Design booking with availability search, pagination and indexing. Then plug the two-stage ranker into it. How do you keep page 2 consistent with page 1 when scores are recomputed between requests? [Reported] [26]
25. How would the search diagram change for Airbnb Experiences, where inventory is small, sold in time slots, and bookings are sparse? [Likely] [14]
26. Design category browsing ("Beachfront", "Treehouses"). How do categories assigned by ML and checked by humans feed candidate retrieval, and how do you rank inside a category? [Likely] [33]
27. Which home attributes (wifi, parking, a workspace) should a listing card show to a given guest? Where does that model sit in Diagram 1, and what is its label? [Likely] [38]

Logging, feedback and monitoring

28. What exactly do you log per impression so that you can correct for position bias later? [Likely] [10]
    - How would you estimate position bias from Airbnb's logs?
29. Bookings arrive days after the search. How does that delay change your training-data pipeline and your retraining cadence? [Likely] [9]
30. How do you test a new ranker: offline replay, interleaving, then A/B? What decision does each stage make? [Reported] [1][15]
    - When does interleaving disagree with the A/B result, and which do you trust?
31. What would you put on the search-ranking monitoring dashboard on day one after launch? [Reported] [1]
    - A drop in bookings hits a single market only. How do you narrow it down to data, model or product causes?

Diagram 2. Listing-embedding personalization (similar listings and real-time session signals)

Reported in the forms "design user embedding semantic search" (Apr 2026) [2] and "listing recommendation" [5]. The layout follows Airbnb's listing-embedding work [11] and the later embedding-based retrieval work [20].



Questions

32. Design a user-embedding semantic search system for Airbnb. [Reported] [2]
    - What are the user tower's inputs, and how do you represent a guest with one past booking or none?
    - How do you evaluate the embedding before any online test?
33. Design a listing recommendation system ("similar listings" or homepage recommendations). [Reported] [5]
    - Where does the candidate set come from, and how is the final list ranked?
34. Why train listing embeddings on click sessions rather than on listing attributes? What does each capture? [Likely] [11]
    - How do you split a browsing log into sessions, and why does that choice matter?
35. Bookings are far sparser than clicks. How would you learn embeddings that capture long-term booking preference? [Likely] [11]
36. How do you give a brand-new listing an embedding on day one? [Likely] [1][11] (Cold start for new listings is a graded area [1], but no source reports this embedding variant.)
    - How long until the cold-start vector is replaced, and how do you detect that it was a bad guess?
37. How do you turn in-session signals (clicks, skips, wishlists) into ranker features within the latency budget? [Likely] [11]
    - Where does the session state live, and what happens when the guest switches device?
38. Embeddings are retrained daily, and vectors from different runs are not aligned. How do you keep downstream features and the ANN index consistent? [Likely]

39. Which ANN index would you use, and what recall or latency trade-off would you accept? [Likely] [20]
    - How do you apply hard filters (dates, guests) to ANN results without losing recall?
40. How would you add text and photo signals to listing embeddings, and when is that worth the cost? [Likely] [7]
41. How would you check whether personalization is narrowing guests into a filter bubble, and what would you monitor? [Likely] [12]
42. A photo classifier tags each listing photo by room type (bedroom, kitchen, exterior). Draw it as an upstream model that feeds both cover-photo choice and listing-quality features. How do you version it so that downstream models do not break? [Likely] [35]
43. Amenities detected from photos and amenities entered by the host can disagree. How do vision-derived amenities enter the feature store and the embeddings, and which source wins when they conflict? [Likely] [36][37]

Diagram 3. Smart Pricing and demand forecasting

Reported as "design a system to help landlords set prices" [4] and "price prediction" [5]. The layout follows Airbnb's published dynamic-pricing work [19] and home-value prediction post [32].

```
DATA: listing attributes, calendar (booked / open by night), price history,
      market supply, search demand (searches for dates x area), events, seasonality
     |
     v
+------------------------------ OFFLINE (daily) ----------------------------+
| [Demand model]   P(book night d | price p, lead time, features)  (GBDT/NN) |
|       |          label: booked by check-in? (censored for future nights)   |
| [Market forecast] demand per market x date (time series + event signals)   |
|       |                                                                    |
| [Price model]    suggested price per (listing, night); custom loss that    |
|                  penalises over- vs under-pricing differently              |
|       |                                                                    |
| backtest: would suggestion have booked? revenue vs host-set price           |
+-------|--------------------------------------------------------------------+
        v
[Price store: listing x night -> suggested price, min/max host bounds]
        |                                    ^
        | read by host app + calendar        | host overrides (feedback)
        v                                    |
HOST UI: "Smart Pricing on" / tips -----> host accepts or edits
        |
        v
SEARCH sees the new price -> guest demand responds -> bookings
FEEDBACK: bookings + host overrides + price-change elasticity --> retrain
EXPERIMENT: host-level or market-level randomisation (interference!)
MONITORING: forecast error by market and lead time, occupancy, host revenue,
           override rate, price shock alerts (events, outages)
```

Questions

44. Design a system that helps hosts set nightly prices. [Reported] [4]
    - What does the host want to maximise: revenue, occupancy, or something else? How do you agree on that with the interviewer?
45. Design a price prediction model for Airbnb listings. What exactly is the target? [Reported] [5]
    - Why is "the price the host set" a weak label?

46. Draw the split between a demand (booking probability) model and a price-suggestion model. Why separate them? [Likely] [19]
47. Future nights are not yet booked. How do you build training labels for demand at different lead times? [Likely] [19]
48. Why might a custom asymmetric loss beat plain regression for price suggestions? [Likely] [19]
49. How often do you recompute suggested prices, and which inputs must be near real time (for example a newly announced concert)? [Likely]
50. Changing prices for many hosts in one market changes the demand that every other host sees. How do you design the experiment? [Likely] [24]
    - Would you randomise by host, by listing or by market? What does each cost in power?
51. How do you estimate price elasticity from observational data where hosts chose their own prices? [Likely]
52. Hosts override suggestions often. How do overrides enter the feedback loop, and what bias do they create? [Likely]
53. What would you monitor to catch a pricing model that is quietly pushing prices too low in one city? [Likely]
54. How does demand forecasting feed other systems in the diagram (search ranking, host tips, supply growth)? [Likely]
55. How does Smart Pricing suggest a price for a brand-new listing with no booking history? Draw the cold-start path, and say when it hands over to the listing's own demand model. [Likely] [19][32]
56. Hosts choose whether to turn Smart Pricing on, so the adopters are not a random sample. How would you estimate its effect on host revenue without a randomised test? Where would an ML-based counterfactual estimate sit in the diagram? [Likely] [40]
57. Build a panel dataset in SQL and run a causal regression on it, for example bookings on price at the listing-week level. Which confounders must the panel control for? [Reported] [2]
58. Design the host listings page for a host with 100+ listings so that it shows suggested prices and demand forecasts for every listing. What do you precompute, and how do you aggregate across listings fast enough? [Reported] [2][3] (The base prompt, a host listings page or dashboard for 100+ listings, was reported. The pricing layer is this chapter's addition.)

Diagram 4. Trust: payment fraud, account takeover (ATO) and off-platform payment

Fraud and ATO are not reported as prompts in the fetched sources; they are [Likely] because Airbnb's Trust and Safety MLE team works on "fraud defense, reservation screening, party-risk prediction and property-damage risk" [1], and Airbnb has published on targeted friction for fraud [34]. **Relisting detection** is the reported trust-style prompt, including geohashing and its link to online metrics [1][2][3].

```
EVENTS: login, device / IP / geo, password reset, payout-method change,
        booking + payment, messages, listing create / edit
     |
     v
[Real-time features]  Chronon streaming: velocity (logins / cards / payouts
     |                per device, IP, account in 1h / 24h), new-device flag,
     |                geo distance from last login, account age
     |                + batch: graph features (shared device / card / phone)
     v
[Risk models]   ATO model (login + sensitive actions)              ~20-50 ms
     |          payment-fraud model (checkout)                     sync path
     |          off-platform model: message text classifier
     |             ("pay me by bank transfer", emails, phone numbers)
     v
[Decision layer]  score + rules + cost matrix ->
     |   allow | targeted friction (2FA, ID check, 3DS) | hold payout | block
     |   | send to human review queue
     v
[Human review] --> labels (fraud / not fraud), appeals --> label store
     |
DELAYED LABELS: chargebacks (30-90 days), account-recovery claims, user reports
     |
     v
retrain (weekly) with label-delay-aware windows
MONITORING: precision at review capacity, friction rate on good users,
           chargeback rate, attack-pattern drift, rule-hit dashboards
RELISTING: geohash buckets -> candidate pairs (photo / text / host-graph
           similarity) -> pair classifier -> review -> enforcement
```

Questions

59. Design a system that detects relisting: a removed listing that comes back, possibly under a new account. Draw candidate generation and the pair scorer. [Reported] [1] [2][3]
    - How does geohashing generate candidate pairs, and what geohash precision would you start with? [Reported] [1][3]
    - How do the model's outputs connect to online metrics when confirmed labels arrive late and only for flagged cases? [Reported] [1]
60. Design an account-takeover detection model. Where does it run in the login and payout flows, and with what latency budget? [Likely] [1]
    - Which actions after login are the riskiest, and should they be re-scored?
61. Design payment-fraud scoring at checkout. How does the decision layer turn a score into allow, friction or block? [Likely] [34]
    - How do you set thresholds when a false positive loses a real booking and a false negative costs a chargeback?
62. Chargebacks arrive 30-90 days later. How do you build training data and evaluate a model trained before the labels mature? [Likely]
63. Design a detector for hosts or guests who try to take payment off the platform through messages. [Likely] [17]
    - How do you handle obfuscated text ("p a y p @ l"), images and multiple languages?
    - What is the action: warn, block the message, or send to review?
64. Only reviewed cases get labels, and the model chooses which cases are reviewed. How do you correct for that selection bias? [Likely]
65. How would you use a graph of shared devices, cards and phone numbers? Is it a feature source or a separate model? [Likely]
66. Fraudsters adapt to the model. What monitoring tells you an attack pattern has shifted, and how fast can you ship a fix? [Likely]

67. How would you run an A/B test on a fraud model when the "treatment" lets some fraud through? [Likely] [24]
68. How do you measure the cost of friction on good users and include it in the objective? [Likely] [34]
69. Design reservation screening: score a booking request before it is confirmed, for example for party risk. Where does it sit between checkout and host acceptance, and what actions can it take? [Likely] [1]

- The label (a party actually happened) is rare, delayed and mostly unobserved. What proxy labels would you use, and how do you check them?

70. Design a property-damage risk model at booking time. What is the label, how rare is it, and which action does the score drive (deposit, coverage, extra checks, block)? [Likely] [1]
71. Account deduplication, where accounts that share any attribute are merged, has been reported as a coding question with a follow-up that grows from two attributes to any number [2][26]. How would you turn that entity-resolution step into a feature source for the trust models in this diagram? [Likely] [2][26]
72. Chronon's own quickstart uses a fraud model that is called when checkout starts [6]. Define the GroupBys and the Join you would write for Airbnb's checkout payment-fraud model, including the windows. [Likely] [6]

Diagram 5. LLM customer-support routing and assistance

Reported: "recent senior loops added ranking and retrieval prompts around generative-AI support tools" [1], and "Generate customer-support instructions from company policies" (Sep 2026) [2]. The Community Support AI team works on "LLM fine-tuning, retrieval, evaluation and guardrails for customer-support assistants" [1]. The layout follows Airbnb's message-intent classification [17], its Intelligent Automation Platform [18][27], its text-generation models for support [30] and its LLM-friendly knowledge representation work [23].

```
GUEST / HOST CONTACT (chat, email, voice transcript) + reservation context
     |
     v
[Pre-processing]  PII redaction, language detect, attach booking state      ~20 ms
     v
[Intent + urgency classifier]  (small fine-tuned model; safety intents        ~30 ms
     |                          e.g. safety emergency -> human immediately)
     v
[Retrieval]  policy / help-centre KB chunks, past resolved tickets,          ~100 ms
     |       reservation facts (tools / APIs) -> top-k -> re-ranker
     v
[LLM]  draft answer or self-serve action (cancel, refund within policy)      ~1-3 s
     |  grounded on retrieved policy; structured output
     v
[Guardrails]  policy-compliance check, hallucination check, refund limits,
     |        confidence threshold
     v
ROUTER:  auto-resolve | suggest reply to agent | route to queue (skill,
         language, tier, priority) | escalate to safety team
     |
FEEDBACK: resolution, re-contact within 7 days, CSAT, agent edits to drafts,
          escalations --> eval sets + fine-tuning data
OFFLINE EVAL: golden ticket set, LLM-as-judge + human rubric per policy area
MONITORING: auto-resolve rate, wrong-refund rate, re-contact, latency, cost
           per ticket, drift in intent mix (e.g. after a storm or outage)
```

Questions

73. Design retrieval and ranking for a generative-AI customer-support tool. [Reported] [1]
    - What is the retrieval unit (policy chunk, past ticket, macro), and how do you label relevance?
    - How do you evaluate the retriever separately from the generator?
74. Design a system that generates customer-support instructions from company policies. [Reported] [2]
    - How do you keep generated instructions consistent with policies that change?
    - How would you evaluate instructions before agents use them?
75. Design the routing layer that decides whether a ticket is auto-resolved, drafted for an agent or sent to a specialist queue. [Likely] [18]
    - Which intents must never be auto-resolved, and how is that enforced?
76. How would you classify contact intent across many languages with limited labelled data? [Likely] [17]
77. Where do you redact PII in the diagram, and what does redaction do to model quality? [Likely]
78. What is the latency budget for chat vs email vs voice, and how does it change the model choice? [Likely] [27]
79. How do you stop the LLM from granting a refund that policy does not allow? Draw the guardrail. [Likely]
80. Which online metric shows success: deflection, resolution, re-contact rate or CSAT? Which is the guardrail? [Likely]
81. How do agent edits to LLM drafts become training data, and what bias does that carry? [Likely]
82. A storm cancels thousands of trips in one region. How does the system cope with the spike and the shift in intent mix? [Likely]
83. How would you A/B test a new support model when tickets from the same reservation can hit both arms? [Likely] [24]
84. Design a customer-support ticketing system with agent claim rules by language and location, and first-response metrics. Then show where an ML router would replace or augment the hand-written claim rules. [Reported] [2][3][26] (The ticketing system was reported in Jun 2026 [2] and as an added design round that decided the outcome [26]. The ML router is this chapter's addition.)

    - Which first-response or resolution metric would the router optimise, and which would you keep as a guardrail?

85. Would you use one generative model for intent detection, answer retrieval and reply suggestion, or separate models? Defend the choice on latency, cost and failure isolation. [Likely] [30]
86. When would you fine-tune a model for support, and when would you prompt a base model with retrieval? What evaluation result would change your mind? [Likely] [1][30]

Diagram 6. Natural-language AI search ("a quiet cabin near Lake Tahoe for 6 with a hot tub")

No fetched source reports this prompt by name. It is [Likely] because "design user embedding semantic search" is reported [2] and Airbnb has publicly moved towards AI-powered search [28].

```
NL QUERY (+ chat history)
     |
     v
[LLM query parser]  -> structured: {location: "Lake Tahoe", guests: 6,          ~300-800 ms
     |                 amenities: [hot_tub], vibe: "quiet", dates: ?}         (cache common queries)
     |                 + clarifying question if dates missing
     v
[Hybrid retrieval]  structured filters (geo, availability, guests, amenities)
     |              + semantic ANN on text / photo listing embeddings ("quiet")
     |              + lexical match on reviews / descriptions
     v
[Ranker]  Diagram-1 ranker + query-listing semantic match features          ~50 ms
     v
[Explain / summarise]  optional LLM "why this fits" per result (async, streamed)
     v
RESULTS + follow-up chat refinements
     |
FEEDBACK: reformulations, refinements, clicks, bookings, abandons
OFFLINE EVAL: parse accuracy on labelled queries, retrieval recall on
            judged sets, end-to-end LLM-judge relevance
MONITORING: parse failure rate, empty-result rate, hallucinated amenity
          rate, LLM cost and latency, bookings vs classic search
```

Questions

87. Design natural-language search for stays. Draw where the LLM sits and where it must not sit. [Likely] [2][28]
    - Would you let the LLM rank listings directly? Why or why not?
88. How do you evaluate the query parser, and what happens when it extracts a wrong constraint, such as the wrong city? [Likely]
89. Which constraints become hard filters and which become soft ranking signals ("quiet", "good for kids")? [Likely]
90. How would you keep latency acceptable when an LLM call sits in front of retrieval? [Likely]
    - What would you cache, distil or run in parallel?
91. How would you make sure explanations do not claim an amenity the listing does not have? [Likely]
92. How do you compare NL search with classic search in an experiment when the two attract different kinds of query? [Likely] [24]
93. How do you build a judged relevance set for NL queries at Airbnb's scale? [Likely] [33]
94. How would a knowledge graph of places, activities and listing attributes help the parser and the retriever handle "quiet cabin near Lake Tahoe"? [Likely] [25]
95. The parser returns one guess for an ambiguous place name ("Springfield", "the Cape"). How do you detect the ambiguity, and do you ask, show a mix, or pick the most likely? [Likely] [29]
96. Which parts of NL search can run offline, such as listing summaries and attribute extraction from descriptions and reviews? How do you keep them fresh when hosts edit listings? [Likely] [37]

Diagram 7. Experimentation loop for ML launches

The role includes experimentation and A/B testing, and "how an A/B test confirms" offline gains is graded [1]. "Design an A/B testing platform" was reported in Apr 2026 [2]. The layout follows Airbnb's posts on interleaving [15] and experiment guardrails [24].

```
new model candidate
     |
     v
offline gate: NDCG / AUC / calibration vs prod on the same logged data
     |
     v
shadow traffic: latency, error rate, score-distribution sanity
     |
     v
interleaving (ranking only): team-draft, small traffic, days   [15]
     |
     v
A/B on bookings (unit: guest or search; market-level if interference)
     |  guardrails: cancellations, CS contacts, host earnings, latency  [24]
     v
ramp 1% -> 10% -> 50% -> 100%  with auto-rollback on guardrail breach
     |
     v
holdout kept for long-term effects; post-launch monitoring
```

Questions

97. Design an A/B testing platform that ML teams at Airbnb would use. [Reported] [2]
    - How does assignment work for logged-out guests who later log in?
98. Design robust primary and guardrail metrics, for example for a search-ranking launch. [Reported] [2]
99. Design an A/B test with causal inference for a feature where randomisation is imperfect. [Reported] [2]
100. Where in the launch funnel do you place interleaving, and what can interleaving not tell you? [Likely] [15]
101. Offline NDCG went up but the A/B on bookings is flat. Walk through the diagram to find why. [Likely] [1] (The offline-to-online link is a graded area [1]. This scenario is not reported.)
102. Which unit of randomisation fits each diagram in this chapter (search, pricing, trust, support), and why? [Likely]
103. How do you measure a long-term effect, such as repeat bookings, when the A/B runs only for two weeks? [Likely] [24]
104. Design a network-aware experiment for a "Wi-Fi" badge on listings. [Reported] [2]

    - Treated and control listings compete for the same guests. Which unit and design would you choose (listing, market cluster, switchback), and what bias does each leave?

105. Design an event aggregation pipeline for clicks (reported in its ad-click form). Then make the same pipeline feed both the experiment metrics and the ranker's training labels. How do you handle late, duplicate and bot events? [Reported] [2]
106. Two teams' launches trade bookings now against host retention later. How would one shared "future value" metric settle the trade-off, and where does it enter the launch decision in this diagram? [Likely] [39]
107. Draw the assignment, exposure-logging and metric-computation services of an experiment platform that runs hundreds of concurrent search experiments. How do

you detect interaction between two experiments? [Likely] [41]

Diagram 8. The Chronon feature path (zoom-in for any diagram)

Chronon is Airbnb's open-source feature platform. It defines features once as GroupBys (keyed, windowed aggregations) and Joins, produces point-in-time-correct backfills for training, serves the latest values from a key-value store, can update features from Kafka streams, and measures online/offline consistency by replaying logged fetches [6]. Its predecessor, Zipline, is described in two Airbnb talks [16].

```
sources: Hive tables (batch) + Kafka topics (streaming events)
     |
     v
GroupBy  key=listing_id   aggs: count(bookings, 7d/30d), avg(rating), ...
GroupBy  key=guest_id     aggs: last_k(clicked_listing_ids, 10), ...
     |
     v
Join  (left = search / checkout events with timestamps)
     |-----------------------------+-----------------------------------+
     v                             v                                   v
BACKFILL (point-in-time)     BATCH UPLOAD -> online KV           STREAMING updates
-> training table            (daily snapshot)                    (Kafka -> KV, seconds)
                                   |                                   |
                                   +-------------> fetch API <---------+
                                                   |  (low ms, high fan-out)
                                                   v
                                          model server request
     |
     v
CONSISTENCY: log online fetches -> replay as backfill -> compare -> alert
```

Questions

108. Draw how a feature like "listing bookings in the last 7 days" flows from raw events to both training data and the online model. [Likely] [6]
109. What does point-in-time correctness mean for a search-ranking training example, and how would you test that your backfill has it? [Likely] [6]
110. A feature's online value differs from its backfilled value 3% of the time. How do you find the cause? [Likely] [6]
111. Which features in the search ranker need streaming updates, and what does streaming cost you in complexity? [Likely] [6]
112. How does high fan-out (hundreds of candidate listings per request) shape the feature-fetch design and the latency budget? [Likely] [6]
113. When would you log features at serving time instead of backfilling them? [Likely] [6]
114. You are asked to design a platform feature, such as "model cumulative booking metrics and first and last booking times". How would you express it in Chronon terms? [Reported] [2]
115. Design a query system over user activity filtered by time window and geography. Then reuse it to compute windowed geo features, such as searches per area in the last hour, for pricing and ranking. [Reported] [2] (The base query system was reported in Nov 2025. The feature reuse is this chapter's addition.)
116. A streaming feature in the ranker stops updating, but the online store keeps serving its last value. How do you detect stale features, and what should the model or the

serving layer do when a feature is stale? [Likely] [6]

Cross-cutting probes (any diagram)

117. Estimate the listing lifetime value. Show where the LTV model sits in a diagram, and what decision consumes it. [Reported] [1][2][3] - The interviewer moves you from regression to bucketized classification. Redraw the output layer and the evaluation. [Reported] [1]
118. Design a system that calculates customer (guest) lifetime value. [Reported] [4] - The interviewer has no time for serving. Which part of the diagram do you drop, and which must stay? [Reported] [4]
119. Choose a listing's cover photo to lift bookings. Draw the system, including cold start. [Reported] [1]
120. Design a model that identifies high-quality listings. Where does the label come from in your diagram? [Reported] [1]
121. Which of the diagrams in this chapter has the worst feedback loop (the model shaping its own training data)? How do you break it? [Likely]
122. For each diagram, name the one metric that would page you at 3 a.m. [Likely]
123. Which pieces of these six systems would you build once as shared platform (feature store, embedding service, ANN, experiment platform), and which stay product-specific? [Likely] [6][32]
124. Your scheduled ML experience round turns into an unplanned ML design question. How do you switch to this skeleton and get a diagram on the board within five minutes? [Reported] [5]
125. A phone screen ends with two short ML design prompts. Give a two-minute opener for each: the target variable, a cold-start plan and a one-line architecture. [Reported] [1]
126. Redraw one ML system you shipped in this chapter's skeleton. Mark the boxes you decided yourself, the alternatives you rejected and the metric the system moved. [Reported] [1]

References

Status: **fetched** = read on 2026-10-14; **URL confirmed** = title and URL match the applied-ML index [7] or the case-study index [31] (both fetched), but the page itself was not opened; **unverified** = carried over from chapters 03 and 09, host blocked by the proxy on 2026-10-14. No web searches could run for this chapter (search budget exhausted). Removed in review: a PracHub report and the Glassdoor MLE page (uncited and unreachable), and an Airbnb Careers job posting (unreachable; its claim is now cited to [1]).

1. "Airbnb Machine Learning Engineer interview process" (TrueInterview-derived summary of 3 candidate reports, mostly 2026 Q1-Q2). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb" question index with report dates (Rank Home Search Results Without a Text Query, Sep 2026; Generate Customer-Support Instructions from Company Policies,

Sep 2026; Model Cumulative Booking Metrics, Sep 2026; Listing Lifetime Value, May 2026; Design User Embedding Semantic Search, Apr 2026; Design an A/B Testing Platform, Apr 2026; Home Page Search + Availability + Ranking, Mar 2026; undated: personalized search ranking, relisting detection, primary and guardrail metrics, A/B test with causal inference). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md

3. "Airbnb interview process" (one or two ML system design rounds for MLEs; "Problem and label before model"). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. 1point3acres post #40, English mirror (ML design: customer lifetime value; helping hosts set prices). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
5. 1point3acres post #79, English mirror (ML design: listing recommendation; price prediction; ML experience round turned into ML design). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
6. "Chronon: A data platform for AI/ML" (GroupBy, Join, point-in-time backfills, online serving, streaming, consistency measurement). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon
7. Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production." eugeneyan/applied-ml, GitHub. Fetched. https://github.com/eugeneyan/applied-ml
8. Dirafzoon, "ML System Design" (nine-step template; lists "Rental recommendation (Airbnb)"). alirezadir/AIMLInterviews, GitHub. Fetched. https://github.com/alirezadir/AIMLInterviews/blob/main/src/MLSD/ml-system-design.md
9. Haldar et al., "Applying Deep Learning To Airbnb Search," arXiv 1810.09591, 2018 (KDD 2019). URL confirmed. https://arxiv.org/abs/1810.09591
10. Haldar et al., "Improving Deep Learning For Airbnb Search," arXiv 2002.05515, 2020. URL confirmed. https://arxiv.org/abs/2002.05515
11. Grbovic, "Listing Embeddings in Search Ranking," Airbnb Tech Blog, 2018. URL confirmed. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
12. Abdool et al., "Managing Diversity in Airbnb Search," arXiv 2004.02621, 2020. URL confirmed. https://arxiv.org/abs/2004.02621
13. "Learning To Rank Diversely," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621
14. Grbovic, "Machine Learning-Powered Search Ranking of Airbnb Experiences," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
15. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
16. Simha, "Zipline: Airbnb's Machine Learning Data Management Platform" (2018) and "Zipline - A Declarative Feature Engineering Framework" (2020), talks, YouTube. URL

confirmed. https://www.youtube.com/watch?v=Tg5VEMEsC-0 ; https://www.youtube.com/watch?v=LjcKCm0G_OY
17. "Discovering and Classifying In-app Message Intent at Airbnb," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/discovering-and-classifying-in-app-message-intent-at-airbnb-6a55f5400a0c
18. "Intelligent Automation Platform: Empowering Conversational AI and Beyond at Airbnb," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/intelligent-automation-platform-empowering-conversational-ai-and-beyond-at-airbnb-869c44833ff2
19. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing," KDD 2018. Unverified. https://dl.acm.org/doi/10.1145/3219819.3219830
20. "Embedding-Based Retrieval for Airbnb Search," Airbnb Tech Blog, and "Applying Embedding-Based Retrieval to Airbnb Search," arXiv 2601.06873, 2026. Unverified. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/ ; https://arxiv.org/abs/2601.06873
21. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning," arXiv 2305.18431, 2023 (KDD 2023). Unverified. https://arxiv.org/abs/2305.18431
22. Haldar et al., "Learning to Rank for Maps at Airbnb," arXiv 2407.00091, 2024 (KDD 2024). Unverified. https://arxiv.org/abs/2407.00091
23. "LLM-Friendly Knowledge Representation for Customer Support," arXiv 2510.10331, 2025. Unverified. https://arxiv.org/abs/2510.10331
24. "Designing Experimentation Guardrails," Airbnb Tech Blog, 2021. URL confirmed. https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669
25. "Contextualizing Airbnb by Building Knowledge Graph," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/contextualizing-airbnb-by-building-knowledge-graph-b7077e268d5a
26. "Airbnb Software Engineer interview process" (TrueInterview-derived; the standard design round prompts: homepage backend "with ranking signals", booking with availability search, pagination and indexing, a host dashboard for 100+ listings, and an added design round on a support ticketing system "with agent claim rules by language and location and first-response metrics"; account deduplication follow-up from two attributes to any number). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
27. "Automation Platform v2: hybrid LLM conversational platform" (third-party summary of a 2024 Airbnb Tech Blog post), ZenML LLMOps database. Unverified. https://www.zenml.io/mlops-database/airbnb-chronon-internal-dataai-app-platform-conversational-ai-platform-automation-platform-v2-hybrid-llm-conversational
28. "Airbnb Adds AI-Powered Search and Social Features," Let's Data Science, 2026. Unverified. https://letsdatascience.com/news/airbnb-adds-ai-powered-search-and-social-features-dd4ce7ea
29. "Transforming Location Retrieval at Airbnb," arXiv 2408.13399, 2024. Unverified. https://arxiv.org/abs/2408.13399
30. "How AI Text Generation Models Are Reshaping Customer Support at Airbnb," Airbnb Tech Blog. URL confirmed (via [31]). https://medium.com/airbnb-engineering/how-ai-text-generation-models-are-reshaping-customer-support-at-airbnb-a851db0b4fa3

31. "A Curated List of ML System Design Case Studies" (300+ case studies, including 12 Airbnb entries). Engineer1999/A-Curated-List-of-ML-System-Design-Case-Studies, GitHub. Fetched. https://github.com/Engineer1999/A-Curated-List-of-ML-System-Design-Case-Studies
32. "Using Machine Learning to Predict Value of Homes On Airbnb," Airbnb Tech Blog, 2017. URL confirmed. https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d
33. "Building Airbnb Categories with ML and Human-in-the-Loop," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb
34. "Fighting Financial Fraud with Targeted Friction," Airbnb Tech Blog, 2018. Unverified; article URL not confirmed, so only the blog index is given. https://medium.com/airbnb-engineering
35. "Categorizing Listing Photos at Airbnb," Airbnb Tech Blog, 2018. URL confirmed. https://medium.com/airbnb-engineering/categorizing-listing-photos-at-airbnb-f9483f3ab7e3
36. "Amenity Detection and Beyond: New Frontiers of Computer Vision at Airbnb," Airbnb Tech Blog, 2019. URL confirmed. https://medium.com/airbnb-engineering/amenity-detection-and-beyond-new-frontiers-of-computer-vision-at-airbnb-144a4441b72e
37. "WIDeText: A Multimodal Deep Learning Framework," Airbnb Tech Blog, 2020. URL confirmed. https://medium.com/airbnb-engineering/widetext-a-multimodal-deep-learning-framework-31ce2565880c
38. "Prioritizing Home Attributes Based on Guest Interest," Airbnb Tech Blog. URL confirmed (via [31]). https://medium.com/airbnb-engineering/prioritizing-home-attributes-based-on-guest-interest-3c49b827e51a
39. "How Airbnb Measures Future Value to Standardize Tradeoffs," Airbnb Tech Blog, 2021. URL confirmed. https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5
40. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb," Airbnb Tech Blog, 2022. URL confirmed. https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512
41. "Scaling Airbnb's Experimentation Platform," Airbnb Tech Blog, 2017. URL confirmed. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166
42. "Improving Deep Learning for Ranking Stays at Airbnb," Airbnb Tech Blog. URL confirmed (via [31]). https://medium.com/airbnb-engineering/improving-deep-learning-for-ranking-stays-at-airbnb-959097638bde

11. Timed ML Design Walkthroughs: Diagram Templates, Timing Checklist and Stage-by-Stage Questions for Search Ranking, Host Pricing, Guest LTV, Listing Recommendation, Relisting Detection and Support Retrieval

Research note (read first). Reviewed and expanded on 2026-10-14. The shared web-search budget for this run was already used up, so **no new search queries could run** during this review. The proxy could not resolve medium.com, arxiv.org, dl.acm.org, kdd.org or trueinterview.io, so the Airbnb Tech Blog posts and papers, and TrueInterview's own question pages, could not be opened. Glassdoor, Blind, LeetCode Discuss, Reddit, interviewing.io, Prepfully, IGotAnOffer, Exponent and InterviewQuery could not be searched or opened either.

What **was** fetched and read on 2026-10-14:

- the TrueInterview-derived Airbnb MLE loop guide, built from 3 candidate reports, mostly 2026 Q1-Q2 [1], read in full, including its sample ML design prompts and the interviewer steering it reports;
- the dated Airbnb question index (78 questions) [2], the general Airbnb process page [3] and the Airbnb software-engineer process page [23];
- every English 1point3acres Airbnb mirror post in the FlowSight archive that could be downloaded (about 100 posts) [4][5]. Only posts #40 and #79 contain ML design rounds; the rest are coding, standard system design or experience rounds;
- the general nine-step ML design template [6];
- two paper indexes, used only to confirm the titles and URLs of the Airbnb papers and posts [7][21];
- the README of Airbnb's open-source feature platform, Chronon [22].

The Airbnb papers and blog posts [8]-[20] and [24]-[31] were **not** opened. Before the onsite, cross-check this chapter against Glassdoor, Blind and LeetCode Discuss.

Scope, as you asked. You asked for ML system design diagrams and for details on the tag system, as a PDF. So this chapter gives you:

1. a reusable whiteboard diagram template and six annotated prompt diagrams (four core walkthroughs plus two newly reported prompts);
2. a minute-by-minute timing checklist;

3. the tagged questions and follow-up probes an interviewer is likely to ask at each stage of the round.

It does **not** contain written model answers. The diagrams are **study scaffolds**: each one shows where the boxes go and what to label. They are **not** Airbnb's production architecture, and no source says interviewers hand out diagrams. Chapter 03 lists the prompts. Chapter 10 has deeper per-system diagrams. This chapter is the timed, end-to-end practice layer that sits on top of both.

How the tags work

Every numbered question carries exactly one tag. Chapters 01-10 use the same scheme, so the whole pack reads the same way in the PDF.

- **[Reported] [n]**: a fetched source [n] says this prompt, or a close variant of it, came up in an Airbnb interview. Sources [1], [3], [4] and [5] are MLE loops. The question index [2] does not record the role for each question, so a [2]-only item may come from a non-ML loop; the question says so where it matters.
- **[Reported: criterion] [n]**: the source does not quote this exact question. It does name the topic as something interviewers **assess** or **push on** in the Airbnb ML design round, for example "label definition", "cold start" or "freshness" [1][3]. Expect to be pushed on it, but the wording is ours.
- **[Likely]**: a well-grounded inference from the Senior MLE role, the reported round structure and Airbnb's published systems. A citation next to [Likely] points to the paper, post or platform behind the topic. It does **not** mean a candidate reported the question.
- **Follow-up probes** are the indented bullets under a question. They carry no tag of their own. They are probes a senior interviewer would plausibly use, unless the bullet itself names a source.
- **Never inferred:** no question is tagged [Reported] because it is common at other companies. The general template [6] mentions "Rental recommendation (Airbnb)" only as a practice prompt, so it is not counted as a report.

Format notes for this round

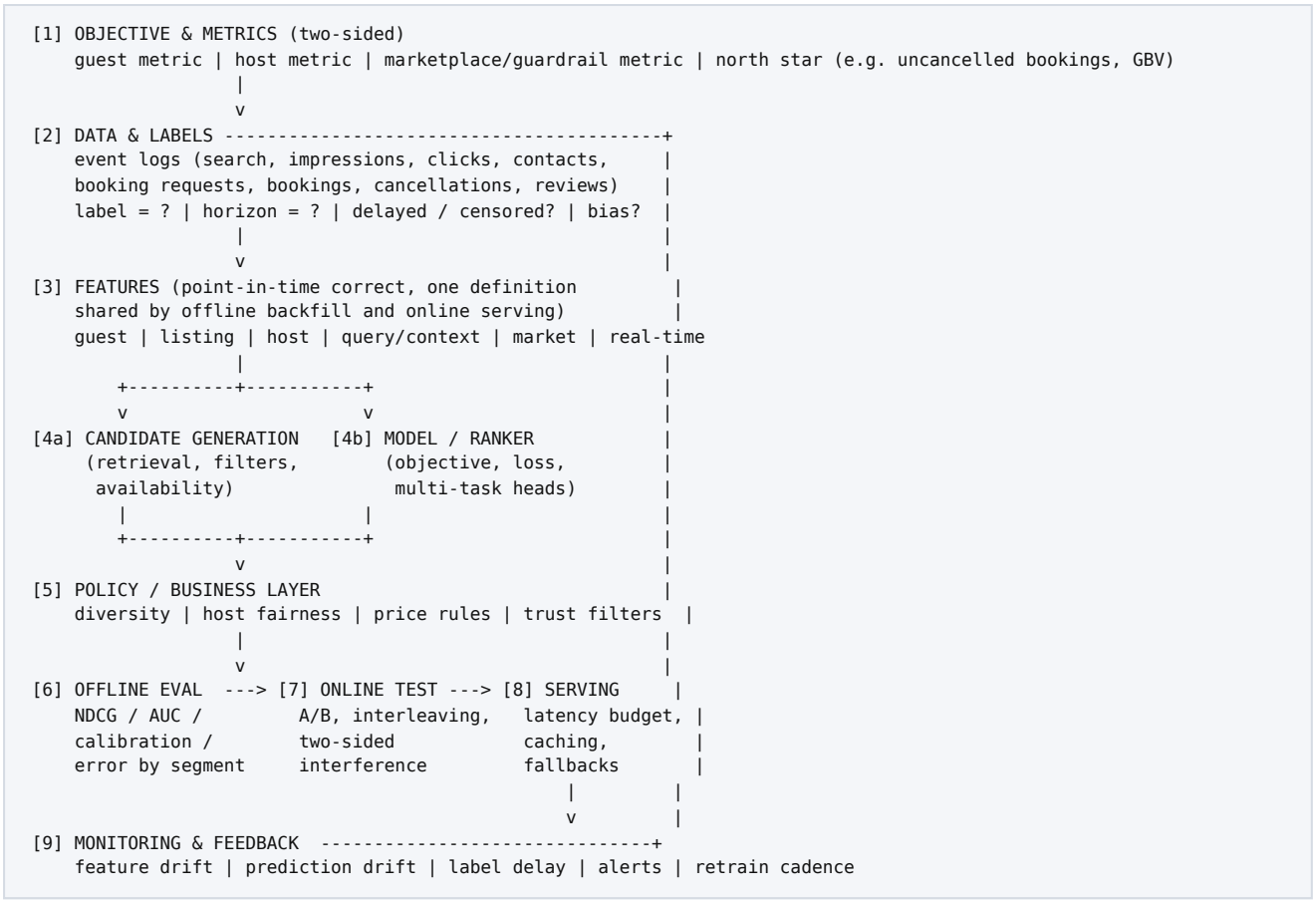
- **How many rounds, and why they matter.** MLE candidates get one or two ML system design rounds instead of a general system design round, sometimes more [1] [3]. The loop is decided mainly by design depth and past scope. In the reports, rejection feedback pointed to design depth or overall match, never coding [1].
- **Reported prompt pairs.** In one 1point3acres loop, the two design rounds were customer LTV and helping hosts set prices [4]. In another, they were listing recommendation and price prediction, and the "ML experience" round also turned into an unplanned design question [5]. A referred candidate got two ML design prompts (cover photo, high-quality listings) inside the phone screen [1]. So prepare at least four end-to-end walkthroughs, not two, and a two-minute opener for each.
- **Length and medium.** All reported onsites were over video [1]. Airbnb's standard design rounds are reported as about an hour of discussion without code [23]; the MLE

sources give no per-round length [1][3]. Plan for 45-60 minutes with a shared doc or whiteboard.

- **What the interviewer scores** [1]:
  - problem framing and the success metric, before any model is mentioned;
  - label definition, including the time window;
  - data and features (reviews, host responsiveness, booking completion, photos, location encodings);
  - the offline-to-online link through A/B testing;
  - production concerns: candidate generation, latency, freshness, cold start and monitoring.
- **Level bar.** A Senior (G9) candidate is expected to own a modelling area from framing to launch and experiments. Staff (G10) is about technical direction across several teams [1]. At G9, you should drive the agenda yourself, make trade-offs out loud and defend a launch decision.
- **Interviewer behaviour.** Interviewers steer firmly. One LTV round fixed the label first, then moved from regression to bucketized classification; a relisting round went into geohashing; search ranking rounds pushed on latency, freshness and experimentation [1]. Treat a redirection as a hint: restate the narrower scope and carry on [1]. Two failure modes are reported:
  - jumping to model ideas before the problem and label are defined ("Problem and label before model") [1][3];
  - chasing perfect detail instead of agreeing on trade-offs [3].
- **Extra round.** A borderline design signal can trigger an added design round. Treat that as a warning sign, not as a neutral step [1][3].

Reusable diagram template (draw this in the first 8-10 minutes)

The same nine boxes fit every Airbnb prompt in this chapter. Draw the skeleton first, then label each box for the prompt in front of you. Adapted from the step structure in [6] and the design skeleton and scoring criteria in [1].



Label every arrow with **what flows** (for example, "logged impressions with position") and every box with **one owner decision** (for example, "label = booking within 7 days of search"). The scoring criteria are about these decisions (label, metric, offline-to-online link), not about how many boxes you draw [1][3].

Timing checklist (45-60 minute round)

Minute (60-min / 45-min)	Stage	Must-say before moving on
0-5 / 0-4	Clarify	Who is the user: guest, host or both? What decision does the model drive? Constraints: scale, latency, cold start
5-10 / 4-8	Objective & metrics	One primary metric, one guest guardrail, one host guardrail. The offline proxy for each
10-17 / 8-13	Data & labels	Exact label, horizon, censoring, position or selection bias, leakage risk
17-24 / 13-18	Features	Feature families by entity, real-time vs batch, point-in-time correctness
24-32 / 18-25	Model & diagram	Baseline first, then the main model; draw the template boxes
32-38 / 25-30	Offline eval	Metric, split (time-based), segment cuts, the gap between offline and online
38-46 / 30-37	Online test	Randomisation unit, interference handling, duration, decision rule
46-53 / 37-42	Serving & monitoring	Latency budget, fallbacks, freshness, drift alerts, retraining
53-60 / 42-45	Trade-offs & wrap	Two senior trade-offs, what you would ship first, what you would cut

Checkpoint habits:

- At each stage boundary, ask the interviewer: "Want me to go deeper here or move on?"
- If you are behind at minute 30 (or 22), compress the features stage to entity families and protect online testing. The A/B link is an explicit scoring item [1].
- If design comes up inside a phone screen, the source says you are expected to name a target variable and a cold-start plan within minutes [1]. Use rows 1-3 of the table as a two-minute opener.

Walkthrough A: Search ranking (home search, with and without a text query)

```
Guest request (dates, guests, map viewport / destination, or NO query)
       |
       v
[Retrieval] geo + availability + hard filters --> EBR / embedding recall --> ~1-5k candidates
       |                                                                      ^
       v                                                                      | listing & guest embeddings (batch)
[First-pass ranker] light model --> top ~200                                  |
       |                                                                      |
       v                                                                      |
[Main ranker] DNN / multi-task heads: P(click), P(contact), P(book), P(no cancel)
       |   features: listing quality, price vs market, guest history, query/context, real-time session
       v
[Re-rank policy] diversity (location, price), new-listing boost, trust filters
       |
       v
Results page / map pins --> impression + position logs --> labels for next training run
```

Box labels to defend: the label (booking, contact or click, and which), position-bias correction, the "no query" context [2], feature freshness [1], and the latency budget per stage.

Clarify and objective

1. Design search ranking for Airbnb homes, end to end, from candidate generation through offline and online evaluation and deployment. [Reported] [1][3]
    - Before any model: what is the label, and why that one rather than a click? [3]
    - What is the single business metric you would make the launch decision on?
2. Design a personalized search ranking system. [Reported] [2]
    - How much of the gain do you expect from personalization rather than from listing quality?
    - What do you show a logged-out first-time guest?
3. Rank home search results when the guest gives no text query. [Reported] [1][2]
    - What replaces the query as context: dates, map viewport, past searches, location?
    - How do flexible dates or a flexible destination change retrieval?
4. Design the home page so that search, availability and ranking work together. [Reported] [2]
    - The index lists this as a hard system design question (Mar 2026) without saying whether it came from an ML loop. In an ML round, where does the availability check sit relative to the ranker, and what do you cache per map viewport?

5. Which guest-side and host-side metrics would you track, and which is the guardrail? [Likely]
    - How would you detect that a ranking gain for guests concentrates bookings on a few hosts?
6. Should the ranker optimise booking, uncancelled booking, or booking plus a good review? [Likely] [17][21]
    - How do you handle labels that arrive weeks after the search?

Data, labels and features

7. How do you correct for position bias in the training logs? [Likely] [10]
    - Would you use randomised slots, inverse propensity weighting or a position feature? What does each cost?
8. A guest searches over several sessions before booking. How do you assign credit across the journey? [Likely] [17][21]
    - Would intermediate milestones (wishlist, contact, booking request) become extra training heads?
    - How would you weight those heads, and what do you do when two heads pull in opposite directions?
9. Which listing features matter most, and which of them leak future information? [Reported: criterion] [1]
    - How do you keep review score and price features point-in-time correct?
10. How would you encode location for ranking: geohash cells, map-viewport buckets, distance to the searched point? What resolution would you choose? [Reported: criterion] [1]
11. How do you represent price: absolute, relative to the market, or relative to what this guest usually pays? [Likely] [8]
12. How would you use listing embeddings built from session co-clicks in ranking? [Likely] [11][24]
    - How do you get an embedding for a listing published yesterday?
13. How would you use the listings a guest clicked or skipped earlier in the same session to re-rank results in real time? [Likely] [24]
    - Which signals must arrive within seconds, and which can be batch?
14. How do you make sure the features the ranker trains on match what it sees at serving time, and how would you measure the gap? [Likely] [1][22]
    - Airbnb MLE postings stress training-serving skew, and Airbnb open-sourced a feature platform built for point-in-time backfills [1][22]. What would you check before blaming the model for an offline-online gap?

Model, retrieval and diagram

15. Walk me through the stages from candidate retrieval to the final page, with a latency budget for each. [Reported: criterion] [1]
    - Where does availability filtering happen, and what breaks if it happens too late?
16. Design user-embedding semantic search. [Reported] [2]
    - How are user and listing towers trained, and what are the negatives?
    - How fresh must the user embedding be within a session?

17. Why move from GBDT to a neural ranker, and what would make you stay with GBDT? [Likely] [8][9]
18. How would you add diversity across location and price without retraining the ranker? [Likely] [12][13]
    - How would you measure whether diversity helped bookings or only looked better?
19. How do new listings get impressions without hurting guest experience? [Reported: criterion] [1]
    - How long does a new-listing boost last, and how do you decide?
20. How would ranking differ for map pins compared with the list view? [Likely]
21. How would you rank Airbnb Experiences, where there is far less booking data per item than for homes? [Likely] [10]

Offline evaluation

22. Which offline metric would you use, and why might NDCG improve while bookings stay flat? [Likely]
23. How would you build a time-based holdout that mimics launch conditions? [Likely]
24. Which segments would you cut the evaluation by before trusting a win? [Likely]
    - New guests vs repeat guests, cities vs rural markets, short vs long stays?

Online test and interference

25. How would you run the A/B test for a new ranker, and what is the randomisation unit? [Reported: criterion] [1]
    - Guest, search or market: what bias does each introduce?
26. Treatment guests book the best listings, so control guests see less inventory. How do you handle this interference? [Likely] [14]
    - Would you use listing-side or market-level randomisation, or a budget-split design?
27. When would you use interleaving instead of an A/B test, and what can interleaving not tell you? [Likely] [14]
    - How do you validate that interleaving agrees with A/B results?
28. What is your launch decision rule if bookings rise but cancellations also rise? [Likely] [20]
    - Which guardrail would block the launch? (Compare the reported prompt "Design robust primary and guardrail metrics" [2].)

Serving, monitoring and trade-offs

29. What would you cache, what must be computed per request, and what is the fallback when the ranker times out? [Reported: criterion] [1]
30. Which features must be fresh within minutes (availability, price changes, in-session clicks), and how do you serve them without blowing the latency budget? [Reported: criterion] [1][22]
31. Which drift alerts would you set up, and what would trigger a retrain? [Reported: criterion] [1]
32. How do you stop the model training on its own feedback loop (ranking what it already ranked)? [Likely]

33. Which senior trade-off would you surface first: model complexity vs latency, personalization vs fairness, or short-term bookings vs long-term trust? [Likely]

Walkthrough B: Host dynamic pricing (helping hosts set prices)

```
Listing x night (date) request from Host Pricing UI / Smart Pricing
       |
       v
[Demand model]  P(book | listing, night, price, lead time)  <-- seasonality, events, market supply, search demand
       |
       v
[Price optimiser]  pick price maximising expected revenue (or host objective) within host min/max
       |                       ^
       v                       | price-elasticity estimate (needs exploration / causal design)
[Guardrails] host min/max, market sanity bounds, change-rate limits
       |
       v
Suggested nightly price --> host accepts / overrides --> booked or unbooked night --> labels
```

Box labels to defend:

- **The label.** A booked night is observed only at the price that was shown. An unbooked night is censored.
- **The objective.** Say whether you are optimising host revenue, booking probability or marketplace health.
- **Host adoption.** Hosts can override the suggested price.

Clarify and objective

34. Design a system that helps hosts set prices from market demand and other factors. [Reported] [4]
    - Is the goal to maximise host revenue, occupancy or something else, and who decides?
35. Design a price-prediction model for Airbnb listings. [Reported] [5]
    - Are you predicting the "right" price, the price that will book, or the market price of similar listings?
36. What are the host-side, guest-side and marketplace metrics for a pricing product? [Likely] [15]
    - How would you notice that suggestions raise host revenue but lower guest conversion?
37. Is this a regression problem, a classification problem or a decision problem? [Likely] [15]

Data and labels

38. You only see a booking at the price that was shown. How do you learn demand at prices you never showed? [Likely] [15]
    - What exploration would hosts accept?
39. How do you treat nights that stay unbooked: as negatives, as censored, or by lead time? [Likely]

40. Would you model the probability that a night ever books, or the probability that it books before a given lead time? How does lead time enter the model? [Likely] [15]
41. How do you separate price effects from listing-quality effects in the logs? [Likely]
42. How do host overrides of the suggested price enter the training data? [Likely]

Features and model

43. Which demand signals would you use: search volume, events, seasonality, competitor supply? [Likely] [15]
    - Which of them are available in real time and which are batch?
44. How do you price a brand-new listing with no booking history? [Reported: criterion] [1]
45. How would you build a set of comparable listings to anchor a suggested price, and how do you define "comparable"? [Likely] [19]
46. Would you use one global model or market-level models, and how would you share strength across markets? [Likely]
47. How would you estimate price elasticity, and why is a plain regression on logged prices biased? [Likely]
48. How do you turn a predicted booking probability into a recommended price? [Likely] [15]
    - How do you keep recommendations stable from one day to the next?

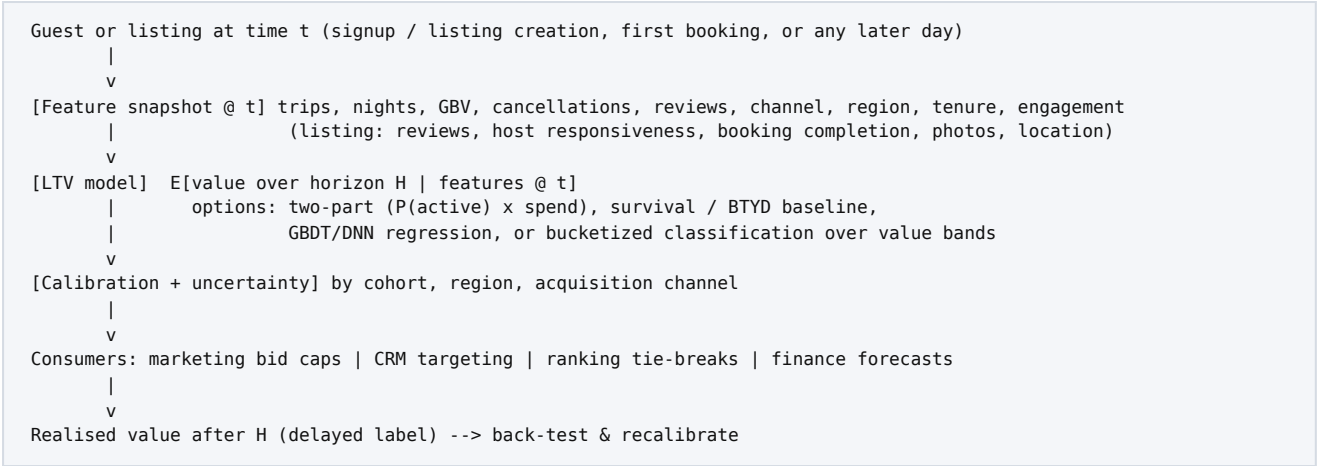
Evaluation and online test

49. Which offline metric fits a demand model, and why is price-prediction RMSE not enough? [Likely] [15]
50. Before any live test, how would you estimate the effect of a new pricing policy from logged data (off-policy or counterfactual estimation)? [Likely] [18]
51. How do you A/B test a pricing feature when treatment and control listings compete for the same guests? [Likely]
    - Would you randomise by listing, host or market, and what does each cost in power?
    - When would a market-level or switchback design be worth the loss of power?
52. How do you measure host adoption, and how do you separate the effect of the feature from selection bias among the hosts who opt in? [Likely]
53. Hosts who see suggestions may also change their calendar, minimum stay or discounts. How do you measure those second-order effects? [Likely]
54. If the experiment shows more bookings but less revenue per night, what would you ship? [Likely]

Serving, monitoring and trade-offs

55. How often do suggested prices refresh, and what triggers an out-of-cycle update (for example, an event announced)? [Likely]
56. What monitoring would catch a price spike caused by a bad feature? [Reported: criterion] [1]
57. How do you handle fairness, regulation or price-gouging concerns during emergencies? [Likely]
58. How do you explain a suggested price to a host so that they trust it? [Likely]

Walkthrough C: Guest (customer) and listing lifetime value



Box labels to defend:

- **The definition.** Is value revenue, GBV or contribution margin, and over which horizon H? Agree it before modelling [1].
- **The censored label.** A recent guest or listing has not lived through the full horizon yet.
- **The consumer.** Who will act on the score?

Clarify and objective

59. Design a system that calculates customer lifetime value. [Reported] [4]
    - Who will use the score, and how does that change the design?
    - The interviewer may not have time for deployment and serving. Which parts would you protect? [4]
60. Estimate a listing's (property's) lifetime value; start by agreeing on what lifetime value means here. [Reported] [1][2][3]
    - How does listing LTV differ from guest LTV in its label and its horizon?
61. Value, measured how: revenue, GBV, contribution margin, or incremental value? [Likely] [16]
62. What horizon would you choose, and how do you trade forecast accuracy against usefulness? [Likely] [16]
63. How would you turn an LTV estimate into a common currency that different teams can use to compare trade-offs? [Likely] [16]

Data and labels

64. How do you build a training label when recent guests have not completed the horizon? [Likely]
65. How do you stop the training snapshot from leaking behaviour that happens after time t? [Reported: criterion] [1]
66. How do you handle the heavy tail of very high-value guests and the many zeros? [Likely]
67. Should cancellations, refunds and support costs reduce the label? [Likely]

Features and model

68. Which features would you build at signup, when you have almost no behaviour? [Reported: criterion] [1]
69. For listing LTV, which marketplace signals would you use (reviews, host responsiveness, booking completion, photos, location), and which exist on the day the listing is created? [Reported: criterion] [1]
70. The interviewer moves you from regression to bucketized classification of LTV. Why might that be the better framing, and how would you choose the buckets and evaluate the model? [Reported] [1]
71. How would you estimate the value of a home before it is listed, for supply acquisition? [Likely] [19]
72. Why might a two-part model (probability of being active multiplied by spend) beat a single regression? [Likely]
73. What simple baseline (for example, a cohort average or a BTYD model) would you start with, and how would you know when to replace it? [Likely]
74. How would you add uncertainty to the estimate, and who needs it? [Likely]

Evaluation, online use and causal questions

75. Which offline metrics would you use: calibration by decile, ranking quality (Gini or AUC on top deciles), or error on aggregate value? [Likely]
76. You can't A/B test an LTV score directly. How do you test the decisions it drives? [Likely] [16]
    - For example: a marketing-spend test with LTV-based bids against rule-based bids.
77. How do you separate predicted LTV from incremental LTV caused by an action? [Likely] [18]
    - Would you use uplift models, holdouts or counterfactual estimation?
78. Design robust primary and guardrail metrics for a launch that uses the LTV score. [Reported] [2]

Serving, monitoring and trade-offs

79. How often do you rescore guests: batch daily, or on events? [Likely]
80. How do you monitor for drift after a large shift such as travel seasonality or a new market launch? [Reported: criterion] [1]
81. What would you trade off between interpretability for finance and accuracy for marketing? [Likely]
82. What privacy and fairness constraints apply when value scores affect who gets incentives? [Likely]

Walkthrough D: Listing recommendation (similar listings, homepage, email, cover photo)

```
Context: anchor listing (similar-listings carousel) OR guest home feed OR email
       |
       v
[Candidate generation]  listing-embedding kNN (session co-click / booking skip-gram)
       |                + same market + available for searched dates + guest-intent embedding
       v
[Ranker]  P(click) x P(book | click), with features: similarity, price delta, quality, distance, availability
       |
       v
[Policy]  dedupe, diversity, exclude unavailable / low-trust, new-listing exploration
       |
       v
Carousel / feed / email --> impressions, clicks, bookings --> retrain embeddings & ranker
```

Box labels to defend: the surface (each surface needs its own objective), the embedding training signal, availability at the guest's dates, and cold start for new listings.

Clarify and objective

83. Design an Airbnb listing recommendation system. [Reported] [1][3][5]
    - Which surface: similar listings, homepage or email? Does the answer change by surface?
    - What is the success metric for a recommendation carousel that is not the main search?
84. Choose a listing's cover photo to increase bookings, including cold start for new listings. [Reported] [1]
    - How do you avoid a photo that drives clicks but disappoints guests at booking?
85. Find high-quality listings to recommend or highlight. [Reported] [1]
    - What does "quality" mean, over what time window, and which label would you trust: reviews, rebookings or human ratings? [1]

Data, labels and features

86. How would you train listing embeddings, and what defines a positive pair? [Likely] [11][24]
    - Why might you use the booked listing as a global context in each session?
87. How do you handle the fact that most listings are booked rarely, so booking labels are sparse? [Likely] [11][24]
88. How do you represent a guest with no history, using only their current session? [Reported: criterion] [1]
89. Which features capture "similar" for a guest: price, location, room type, style, capacity? [Likely] [19]
90. For cover-photo selection, how would you use photo room-type classification and multimodal (image plus text) models as features? [Likely] [25][26]
91. How would you build themed listing categories (for example, treehouses or lakefront) with ML plus human review, and then recommend them? [Likely] [27]

Model, evaluation and online test

92. Why use two stages (candidate generation, then ranking) rather than one model? [Likely]
93. Which offline metrics would you use: recall@k for retrieval, NDCG for ranking, coverage and diversity? [Likely]

94. How would you A/B test a new carousel when it may take bookings from the main search page? [Likely]
    - How would you measure net incremental bookings rather than bookings moved between surfaces?
95. A cover photo appears in search, recommendations and emails at once. How would you design the online test for a cover-photo change, and what is the randomisation unit? [Likely]
96. How would you give new listings exposure without hurting click-through? [Reported: criterion] [1]

Serving, monitoring and trade-offs

97. How often do you refresh embeddings, and how do you keep old and new embedding versions compatible? [Likely]
98. What is your fallback when the nearest neighbours are unavailable on the guest's dates? [Likely]
99. How would you monitor for popularity bias, where the same listings dominate every carousel? [Likely]

Walkthrough E: Relisting detection (newly reported prompt)

The MLE guide lists relisting detection as a reported prompt and says the round went into geohashing and into how the outputs show up in online metrics [1][3]. The question index also lists "Design Airbnb Relisting Detection" [2]. None of the sources defines "relisting", so clarifying it is part of the round.

```
New or edited listing (photos, text, amenities, lat/long, host account)
       |
       v
[Candidate pairs]  same / neighbouring geohash cells  +  near-duplicate photo hash  +  host-account links
       |
       v
[Pair model]  P(same home or banned host returning | pair features)
       |        features: photo & text similarity, amenity overlap, geo distance, account / device / payment links
       v
[Decision policy]  threshold by action: auto-merge | human review queue | allow
       |
       v
Actions --> reviewer verdicts + appeals --> labels;  online metrics: guest issues, removed-listing reappearance
```

Box labels to defend: what counts as a relisting, the geohash cell size, the cost of a false positive on a legitimate host, and the online metric the system moves.

100. Design Airbnb relisting detection. [Reported] [1][2][3] - What counts as a relisting: the same physical home listed twice, a removed listing re-created, or a removed host returning under a new account?
101. How does geohashing help you generate candidate pairs, and what cell size would you use in a dense city versus a rural area? [Reported] [1][3] - What breaks at cell borders, and how do you fix it?
102. How do the outputs of a relisting system show up in online metrics, and which metric would you report? [Reported] [1]

103. Where do labels come from (trust-team investigations, reviewer verdicts, appeals), and how do you handle so few positives? [Likely]
104. Which pair features would you use: photo and text similarity, amenity overlap, geo distance, account and device links? [Likely] [25][31]
105. How would you set the threshold and size the human review queue, given the cost of wrongly blocking a legitimate host? [Likely]
106. Relisters change photos and shift the map pin to evade detection. How do you monitor for this and retrain? [Likely]

Walkthrough F: Retrieval and ranking for generative-AI support (newly reported prompt)

The MLE guide says recent senior loops added ranking and retrieval prompts around generative-AI support tools, and it lists "Generate Customer-Support Instructions from Company Policies" (hard, Sep 2026) [1][2]. Chapter 10, Diagram 5 has a deeper support-routing diagram; this is the timed version.

```
Support ticket / guest or host message
       |
       v
[Intent + routing]  intent classifier --> topic, urgency, language
       |
       v
[Retrieval]  policy & help-article index (hybrid: keyword + embedding) --> top-k passages
       |
       v
[Re-ranker]  passage relevance x policy freshness x region applicability
       |
       v
[Generator + guardrails]  instructions grounded in cited passages; refuse if no policy matches
       |
       v
Agent or guest --> resolution, escalation, CSAT, agent edits --> labels for retriever & re-ranker
```

Box labels to defend: the retrieval label (which passage was actually correct), policy versioning by region, the grounding check, and the online metric (resolution without escalation).

107. Generate customer-support instructions from company policies. [Reported] [1][2] - How would you retrieve the right policy passages for a ticket, and how would you evaluate retrieval separately from generation?
108. Design the ranking of help articles or policy passages shown to a support agent or a support assistant. [Reported] [1] - What is the label: agent click, agent use in the reply, or ticket resolved?
109. How would you classify message intent and route a ticket before retrieval? [Likely] [28][29]
110. What guardrails stop the assistant from stating a policy that does not exist or does not apply in the guest's region, and how would you measure that offline and online? [Likely]
111. How would you A/B test a support assistant: unit of randomisation, primary metric (resolution, escalation, handle time, CSAT) and guardrails? [Likely] [29]

Cross-cutting experimentation probes (asked in any walkthrough)

112. Design an A/B testing platform. [Reported] [2] - How does the platform handle assignment, logging, metric computation and guardrails? - How would you keep search and pricing experiments that run at the same time from colliding? [30]
113. Design an A/B test with causal inference. [Reported] [2] - What would you do when randomisation is impossible?
114. Design a network-aware experiment (the reported prompt was a Wi-Fi badge experiment). [Reported] [2] - How does a two-sided marketplace create spillover between treatment and control?
115. Compute control and treatment metrics from bookings and visits data, and say which denominator you would use. [Reported] [2] - The index lists this as a SQL question; in a design round, expect it as "show me how you would compute the metric".
116. Build a panel dataset and run a causal regression to estimate a treatment effect. [Reported] [2] - Which fixed effects would you include, and what assumption must hold for the estimate to be causal?
117. In a two-sided marketplace, when would you randomise by guest, by listing, by market or by time (switchback)? [Likely] [14]
118. How long would you run the test, and how do you handle novelty effects and weekly seasonality? [Likely]
119. How do you choose guardrails so that a ranking change can't quietly hurt hosts? [Likely] [20]
120. How do you reduce variance (for example, CUPED) so that a booking test reaches power sooner? [Likely]
121. How do you decide when an offline gain is large enough to justify an online test? [Reported: criterion] [1]

Senior-level trade-off probes (final 5-7 minutes)

122. What would you ship in the first month, and what would you defer? [Likely]
123. What did you trade off to stay inside the latency budget? [Reported: criterion] [1]
124. Where would you push back on the business metric you were given? [Likely]
125. Which part of your design is most likely to fail in production, and how would you know? [Reported: criterion] [1]
126. How would you split this work across two engineers and a data scientist, and what would you own personally? [Reported: criterion] [1] - At G9 you are expected to own the area from framing to launch [1]. Show that.
127. If the interviewer narrows scope halfway through, how do you restate the plan and keep going? [Reported: criterion] [1]
128. Your past-project round turns into a design question: redesign the system you built for Airbnb's marketplace. What changes? [Reported] [5]
129. What would make you decide not to use ML here at all? [Likely]

References

Status: **fetched** = read on 2026-10-14; **title confirmed** = title and URL match a fetched index ([7] or [21]), but the page itself was not opened (medium.com, arxiv.org and kdd.org could not be resolved); **unverified** = host unreachable and not found in a fetched index, listed from prior knowledge. No web searches could run during this review.

1. "Airbnb Machine Learning Engineer interview process" (TrueInterview-derived guide built from 3 candidate reports, mostly 2026 Q1-Q2; sample prompts, interviewer steering, scoring criteria, levels). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb" question index, 78 questions with report dates (Rank Home Search Results Without a Text Query, Sep 2026; Generate Customer-Support Instructions from Company Policies, Sep 2026; Listing Lifetime Value — Estimation, May 2026; Design User Embedding Semantic Search, Apr 2026; Design an A/B Testing Platform, Apr 2026; Home Page — Search + Availability + Ranking, Mar 2026; undated: personalized search ranking, relisting detection, primary and guardrail metrics, A/B test with causal inference, network-aware Wi-Fi badge experiment, C/T metrics from bookings and visits, panel and causal regression). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
3. "Airbnb interview process" ("Problem and label before model"; one or two ML design rounds for MLEs; search ranking, property LTV, recommendations, relisting detection with geohashing). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. 1point3acres post #40, English mirror (ML design: customer LTV; helping hosts set prices; deployment may be skipped for time). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
5. 1point3acres post #79, English mirror (ML design: listing recommendation; price prediction; ML experience round turned into ML design). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
6. Dirafzoon, "ML System Design" (nine-step template; lists "Rental recommendation (Airbnb)" as a sample prompt). alirezadir/AIMLInterviews, GitHub. Fetched. https://github.com/alirezadir/AIMLInterviews/blob/main/src/MLSD/ml-system-design.md
7. Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production." eugeneyan/applied-ml, GitHub. Fetched. https://github.com/eugeneyan/applied-ml
8. Haldar et al., "Applying Deep Learning To Airbnb Search," arXiv, 2018. Title confirmed [7][21]. https://arxiv.org/abs/1810.09591
9. Haldar et al., "Improving Deep Learning For Airbnb Search," arXiv, 2020. Title confirmed [7]. https://arxiv.org/abs/2002.05515
10. "Machine Learning-Powered Search Ranking of Airbnb Experiences," Airbnb Tech Blog, 2019. Title confirmed [7]. https://medium.com/airbnb-engineering/machine-learning-powered-search-ranking-of-airbnb-experiences-110b4b1a0789
11. "Listing Embeddings in Search Ranking," Airbnb Tech Blog, 2018. Title confirmed [7]. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e

12. Abdool et al., "Managing Diversity in Airbnb Search," arXiv, 2020. Title confirmed [7] [21]. https://arxiv.org/abs/2004.02621
13. "Learning To Rank Diversely," Airbnb Tech Blog, 2022. Title confirmed [7]. https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621
14. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving," Airbnb Tech Blog, 2022. Title confirmed [7]. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
15. Ye et al., "Customized Regression Model for Airbnb Dynamic Pricing," KDD 2018. Unverified. https://dl.acm.org/doi/10.1145/3219819.3219830
16. "How Airbnb Measures Future Value to Standardize Tradeoffs," Airbnb Tech Blog, 2021. Title confirmed [7]. https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5
17. Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning," arXiv, 2023. Title confirmed [21]; arXiv URL unverified. https://arxiv.org/abs/2305.18431
18. "Artificial Counterfactual Estimation: Machine Learning-Based Causal Inference at Airbnb," Airbnb Tech Blog, 2022. Title confirmed [7]. https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512
19. "Using Machine Learning to Predict Value of Homes On Airbnb," Airbnb Tech Blog, 2017. Title confirmed [7]. https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d
20. "Designing Experimentation Guardrails," Airbnb Tech Blog, 2021. Title confirmed [7]. https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669
21. Wang, "Reco-papers: classic papers and resources on recommendation" (lists the Airbnb search, embedding, diversity and multi-task search-journey papers). wzhe06/Reco-papers, GitHub. Fetched. https://github.com/wzhe06/Reco-papers
22. "Chronon: A Data Platform for AI/ML" (point-in-time accurate backfills; online/offline consistency monitoring). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon
23. "Airbnb Software Engineer interview process" (design rounds about an hour each without code; search, pricing and community-support teams). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-software-engineer.md
24. Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb," KDD 2018. Title confirmed [7][21]. https://www.kdd.org/kdd2018/accepted-papers/view/real-time-personalization-using-embeddings-for-search-ranking-at-airbnb
25. "Categorizing Listing Photos at Airbnb," Airbnb Tech Blog, 2018. Title confirmed [7]. https://medium.com/airbnb-engineering/categorizing-listing-photos-at-airbnb-f9483f3ab7e3
26. "WIDeText: A Multimodal Deep Learning Framework," Airbnb Tech Blog, 2020. Title confirmed [7]. https://medium.com/airbnb-engineering/widetext-a-multimodal-deep-learning-framework-31ce2565880c
27. "Building Airbnb Categories with ML and Human-in-the-Loop," Airbnb Tech Blog, 2022. Title confirmed [7]. https://medium.com/airbnb-engineering/building-airbnb-categories-with-ml-and-human-in-the-loop-e97988e70ebb

28. "Discovering and Classifying In-app Message Intent at Airbnb," Airbnb Tech Blog, 2019. Title confirmed [7]. https://medium.com/airbnb-engineering/discovering-and-classifying-in-app-message-intent-at-airbnb-6a55f5400a0c
29. "Intelligent Automation Platform: Empowering Conversational AI and Beyond at Airbnb," Airbnb Tech Blog, 2022. Title confirmed [7]. https://medium.com/airbnb-engineering/intelligent-automation-platform-empowering-conversational-ai-and-beyond-at-airbnb-869c44833ff2
30. "Scaling Airbnb's Experimentation Platform," Airbnb Tech Blog, 2017. Title confirmed [7]. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166
31. "Amenity Detection and Beyond — New Frontiers of Computer Vision at Airbnb," Airbnb Tech Blog, 2019. Title confirmed [7]. https://medium.com/airbnb-engineering/amenity-detection-and-beyond-new-frontiers-of-computer-vision-at-airbnb-144a4441b72e

12. ML Platform and Experimentation Pipeline Diagrams: Bighead, Zipline/Chronon, ERF/Minerva and Search Interleaving

> **Research note (read first).** Written on 2026-10-14 and revised the same day by an adversarial review. **No web search could run in either pass**: the shared search budget was used up, and both passes were refused. In the review pass, medium.com, arxiv.org, datacouncil.ai, youtube.com and other non-GitHub hosts did not resolve through the proxy. GitHub pages did. The review re-fetched and checked the interview-report pages [1][2][3], the 1point3acres mirrors [4][5] and the Chronon README and docs [6]-[11]. It also read the raw applied-ml index [12], which confirms the titles, years and URLs of the Airbnb posts [13]-[18] and [22]-[25]. It scanned about 100 Airbnb posts in the 1point3acres mirror for ML or experimentation content; only [4], [5] and [26] were relevant. The review made these changes: one [Reported] question (old Q67, which cited an "offline NDCG" wording that [1] does not contain) was rewritten and retagged [Likely]; one citation was moved from [5] to [4]; a round-length claim that [3] does not support was removed; one reference with a suspect URL (the old [22], Analytics India Magazine) was dropped; and Section E (Q101-Q130) was added. Section E maps the newly checked reported prompts onto the four diagrams. Rerun with web access (Glassdoor, Blind, LeetCode Discuss, Reddit, Prepfully) before relying on the [Reported] coverage.
>
> **What this chapter adds.** Chapters 04, 06, 09 and 10 describe these four systems in prose, and chapter 10 has one-box-per-stage sketches of the experiment funnel (Diagram 7) and the Chronon path (Diagram 8). This chapter draws each platform at whiteboard level, adds notes on how to draw and defend it in an MLE design round, and lists the questions an interviewer is likely to probe it with.
>
> **About the diagrams.** No source reports that an interviewer handed out or asked for these exact diagrams. They are **study scaffolds**. Bighead, ERF and Minerva internals are reconstructed from public talks and posts that were not reachable this session [14][19][20][21]; treat component names as "as publicly described, possibly outdated". The Chronon diagram rests on documentation fetched and read for this chapter [6]-[11]. Nothing here is Airbnb's current production architecture.

How the tags work

- **[Reported] [n]**: a fetched source [n] reports this prompt, or a very close variant, from an Airbnb interview. Source [2] does not always name the role.

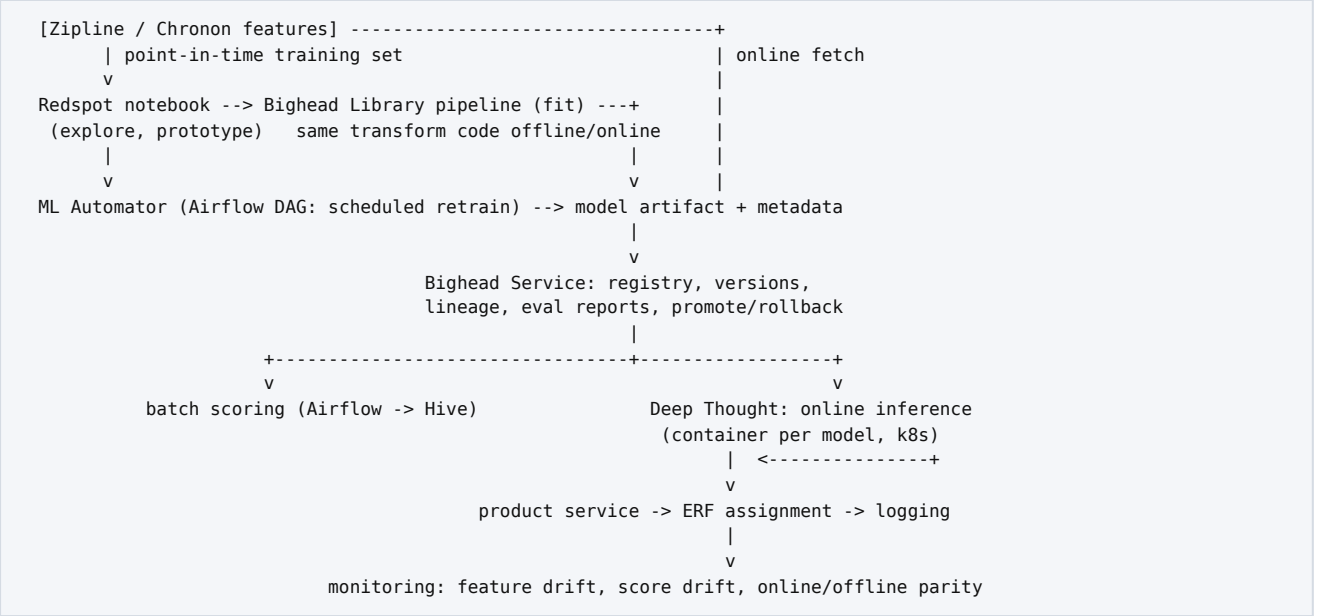
- **[Likely]**: an inference from the Senior MLE (experimentation) role, the reported round structure and Airbnb's public platforms. A citation next to [Likely] points to the system behind the topic. It does **not** mean a candidate was asked the question.
- **Follow-up probes** are the indented bullets. They inherit no tag unless a source is cited inline.
- **Reference status**: **fetched** = read on 2026-10-14; **URL confirmed** = title, year and URL confirmed by the index [12], but the page itself was not opened; **unverified** = host unreachable, carried over from earlier chapters.

Format notes

- **Where these diagrams get used.** MLE candidates get "one or two ML system design rounds" in place of the standard system design round [3]. Senior (G9) loops often have two, with depth "close to staff level" [3]. The ML design rounds are "conversations without code" [1], and interviewers want the problem and its label defined before any model is discussed [3]. The fetched sources give no round length. The MLE guide draws on three candidate reports, "primarily from 2026 Q1 through 2026 Q2" [1]. No source reports a round devoted to ML platform design. These diagrams come up as zoom-ins inside product prompts such as search ranking, LTV, pricing, relisting and cover-photo selection [1][4][5], and inside the reported "Design an A/B Testing Platform" prompt (Apr 2026) [2].
- **What is probed.** Search-ranking rounds "pushed on latency, freshness and experimentation" [1]. Candidates are told to name "how an A/B test would confirm" each model, and role postings stress "training-serving skew, A/B testing, and feature and model selection" [1]. One rejection cited "jumping straight to model ideas" [3], so draw the label and data path before the model.
- **Interviewer.** "Interviewers steer firmly"; treat a redirection as a hint about the expected answer [1]. One LTV interviewer had no time for deployment and serving [4]. Be ready to collapse a platform diagram to its two or three boxes that matter, or to expand any single box on request.
- **Senior bar [Likely].** Owning a system end to end [3] here means you can (a) draw the offline/online split, (b) point at where skew, leakage and bias enter, and (c) say how you would detect each with a metric or a check.

Diagram A. Bighead model lifecycle (Airbnb's end-to-end ML platform)

Publicly described components, from conference talks that were not reachable this session [20]: Redspot (hosted notebooks), Bighead Library (pipeline and transformer abstractions shared by training and serving), ML Automator (scheduled training on Airflow), Bighead Service and UI (model registry and versioning), Deep Thought (online inference) and Zipline (features). Treat the names as historical; Chronon later replaced Zipline [6].



How to draw and defend it. Draw the training path left to right and the serving path below it. Mark the **one shared transform library** with a double line, because that is your answer to training-serving skew. Put the registry in the middle as the single promote/rollback point. End at the experiment assignment so the interviewer sees the model is launched through an A/B test, not just deployed.

A1. Lifecycle and stages

1. Draw the lifecycle of a search-ranking model from notebook prototype to 100% traffic. Which boxes are platform-owned, and which are owned by the product team? [Likely] [20]
    - Where in your diagram does the model first see production-like data?
    - Which stage takes the most calendar time at a company without a platform, and why?
2. Airbnb built Bighead because shipping a model took weeks to months. Which single component of your diagram would you build first for a team that has none of it? [Likely] [20]
    - How would you measure that the platform actually shortened time to production?
3. Your interviewer says "you have no time for serving". Which two boxes of the lifecycle stay on the board, and what do you say about the rest in one sentence? [Reported] [4]
4. Walk through the lifecycle of the model in your own past project using these same boxes: architecture, implementation, challenges and how you solved them. Where did it differ, and what broke? [Reported] [4]
    - Which box would you redesign today?
5. How does the training pipeline get a training set whose features match what Deep Thought will see at inference time? [Likely] [6][20]
6. Compare scheduled retraining on Airflow with event-triggered retraining (on drift or data arrival). What would make you move a pricing model from one to the other? [Likely] [20]

A2. Training-serving skew

7. Role postings stress training-serving skew [1]. Name every arrow in the lifecycle diagram where skew can enter, and the check that catches each. [Likely] [1]
    - Which of those checks run before launch, and which only after?
8. Why put feature transforms in a shared library rather than re-implement them in the serving service? What does that cost you in language or runtime choice? [Likely] [20]
9. A model's offline AUC is 0.82 on a holdout but its online-logged predictions score 0.76 on the same period's labels. Draw where you would look first. [Likely] [1]
    - How do you tell skew from a genuine distribution shift?
10. How would you log features at inference time so the next model can train on exactly what was served? What does that do to your ability to add new features? [Likely] [9]

A3. Registry, versioning and reproducibility

11. What metadata must the model registry store so you can reproduce any model that served traffic in the past 90 days? [Likely] [20]
    - Is the code commit enough? What about the feature definitions and the training-set snapshot?
12. How do you link a model version to the ERF experiment that launched it, so that a later regression can be traced to a launch? [Likely] [14][20]
13. Design the promote and rollback flow. Who can promote, what gates must pass, and how fast can you roll back? [Likely]
    - What is the gate for a model whose label arrives 30 days later (cancellations, LTV)?
14. Two teams train models on the same feature set but different snapshots. How does the registry stop them from comparing offline metrics that are not comparable? [Likely]

A4. Serving and inference

15. Design the online inference tier for a search ranker that scores several hundred candidates per request inside a tight latency budget. Where do the features come from, and what happens if the feature fetch times out? [Likely] [1]
    - Do you fall back to default values, a cached score or a simpler model, and how does each choice show up in the A/B readout?
16. When would you choose batch scoring into a table over online inference, for example for listing quality, host LTV or relisting risk? [Likely] [1]
17. How do you serve a challenger model in shadow mode on the same traffic, and what do you compare before it gets real exposure? [Likely]
18. Container-per-model isolation versus a shared multi-model server: what does each buy you at Airbnb's scale of models and traffic? [Likely] [20]

A5. Monitoring and feedback loops

19. What would page you at 3 a.m. for a production ranker, and what is merely a dashboard? [Likely]
20. Draw the feedback loop by which the ranker shapes its own training data. Where in the lifecycle do you break it (exploration, position-bias correction, randomized slots)? [Likely] [1]

21. How do you detect that an upstream table changed meaning (for example, a new booking status) before it silently degrades every model that uses it? [Likely] [6]

Diagram B. Zipline/Chronon: point-in-time-correct backfill and online serving

Fetched documentation [6]-[11] describes: **GroupBy** (keyed, windowed aggregations over a source), **Join** (many GroupBys joined to a "left" table of keys and timestamps), **accuracy** either `SNAPSHOT` (midnight values, refreshed daily) or `TEMPORAL` (real time while serving, point-in-time correct in backfills) [7], **Label Join** for adding late-arriving labels to features [8], **Bootstrap** for reusing logged or precomputed features in a backfill [9], an **online/offline consistency** job that logs fetches and replays them as a backfill [10], and an optional **tiled** online architecture that stores pre-aggregated tiles instead of raw events [11]. Zipline was the earlier version, presented in 2018 and 2020 talks [13].

```
SOURCES                      DEFINITIONS (Python, compiled)          OFFLINE (Spark)
Hive event tables  ---+      GroupBy: key=listing_id                 Join backfill against LEFT
Hive snapshot tables -+--->    count(bookings, 7d/30d)  ------------> left = search impressions
Kafka topics  --------+        avg(review_rating, 365d)              (keys + ts per row)
                       |       accuracy = TEMPORAL | SNAPSHOT         -> feature value AS OF ts
                       |     GroupBy: key=guest_id                     (point-in-time correct)
                       |       last_k(viewed_listing_ids, 10)               |
                       |     Join: left + [GroupBys]                        v
                       |     LabelPart: booked_within_7d, offset       training table
                       |                                                + Label Join (labels land late)
                       |                                                + Bootstrap (reuse logged features)
                       |
                       |   ONLINE
                       +-> daily batch upload  ----> KV store <---- streaming (Spark/Flink)
                              (midnight snapshot)      ^  ^           events or 1h tiles
                                                       |  |
                       model server --- fetchJoin(keys) --+  (batch value + stream tail merged)
                             |
                             v
                        fetch log (keys, values, ts)
                             |
                             v
     CONSISTENCY: replay logged keys+ts as a Join backfill -> compare with logged values
                  -> per-feature consistency metrics -> alert / dashboard            [10]
```

How to draw and defend it. Draw one definition box feeding two outputs, offline and online, because "define once, compute twice" is the whole argument. Write the **left table with timestamps** explicitly: point-in-time correctness is computed against those timestamps. Draw the consistency loop as a closed arrow from the fetch log back into the backfill, and say it is how you *measure* skew rather than assume it away [10].

B1. Definitions and point-in-time correctness

22. Draw how "number of bookings for this listing in the last 7 days" flows from raw booking events to both a training row and an online prediction. [Likely] [6][7]
    - What timestamp do you attach to a booking event: request, acceptance or payment capture? Why does it matter for leakage?
23. What exactly does point-in-time correctness mean for a search-ranking training row whose label is "booked within 7 days of the search"? [Likely] [7][8]

    - Which features would leak the label if they were computed as of the end of the day instead of the search timestamp?
24. How would you test that a backfill is point-in-time correct? Design the unit test or data check. [Likely] [10]
25. When is `SNAPSHOT` (midnight) accuracy good enough, and when must a feature be `TEMPORAL`? Give one Airbnb feature of each kind. [Likely] [7]
    - What does choosing TEMPORAL cost you in infrastructure and in backfill compute?
26. Design the feature set for the reported "model cumulative booking metrics and first and last booking times" prompt in GroupBy and Join terms. [Reported] [2]
    - Which aggregations are reversible within a sliding window, and which (first, last, unique count) are not?
    - How do you keep first-booking time correct when a booking is later cancelled or refunded?
27. Your left table is search impressions, with billions of rows a day. How do you keep the backfill tractable (sampling, key filtering, partition pruning)? [Likely] [6][8]
28. What goes wrong when an entity source (daily snapshot of a listings table) is joined to event-level timestamps? How do you explain the resulting midnight staleness to a modeler? [Likely] [8]

B2. Labels and late-arriving outcomes

29. Bookings, cancellations and reviews arrive days or weeks after the search. Draw how Label Join attaches them to features computed at search time. [Likely] [8]
    - How do you version a label that keeps changing (a booking later cancelled), and which version trains the model?
30. For an LTV model, where is the cutoff between the feature window and the label window on your diagram, and how do you prevent overlap? [Likely] [1][4]
    - The interviewer moves you from regression to bucketized classification. Which part of the data diagram changes? [Reported] [1]
31. How do you build a training set for a model whose label needs 30 or 90 days to mature, without the newest month of data being dropped entirely? [Likely] [8]

B3. Online serving path

32. Draw the online fetch for a search request that ranks hundreds of listings. How many KV lookups is that, and how do you meet the latency budget? [Likely] [1][6]
    - Batch the keys, cache hot listings, or precompute per-listing vectors: which, and why?
33. How does the fetcher combine a midnight batch value with the streaming events since midnight? What happens if the stream is lagging? [Likely] [7]
34. A host with thousands of bookings is a hot key. Why does the raw-event read path degrade, and how does the tiled architecture change the read and write paths? [Likely] [11]
    - What do tiles cost you in freshness and in storage?
35. Which features of a search ranker actually need second-level freshness? Defend streaming against the reported "latency and freshness" pushback. [Reported] [1]

    - What is the measurable gain from making one feature real time, and how would you run that experiment?
36. The KV store is down for 3 minutes. What does the model server do, and how is that period treated in experiment readouts and in the next training set? [Likely] [9]

B4. Online/offline consistency

37. A feature's online value differs from its backfilled value 3% of the time. Using the consistency loop, how do you find the cause? [Likely] [10]
    - Which causes are expected (stream lag, midnight upload delay), and which are bugs?
38. Why does the consistency job sample *all fetches of a subset of keys* rather than a random subset of fetches? [Likely] [10]
39. Which consistency threshold would block a model launch, and who owns that decision, the platform or the model team? [Likely]
40. When would you switch from backfilling a feature to "log stitching", training on logged online values via Bootstrap? What do you lose? [Likely] [9]
    - How do you add a new feature to a model whose training data comes only from logs?

B5. Feature experimentation and governance

41. You want to A/B test a model that adds 20 new features. How do you build its training set without disturbing the production Join? [Likely] [9]
42. Two teams define "listing booking rate" differently. How does a feature platform stop duplicate, inconsistent definitions, and how does that connect to Minerva's "define once" goal for metrics? [Likely] [6][21]
43. How do you deprecate a feature that 15 models depend on? [Likely]
44. What privacy and retention constraints apply to guest-level features and fetch logs, and where in the diagram do you enforce them? [Likely] [10]

Diagram C. ERF/Minerva: assignment, exposure logging, metric computation and analysis

Public descriptions, from posts confirmed by the index but not opened this session [14][15] [17], and a Minerva talk that could not be reached [21]. ERF (Experiment Reporting Framework) assigns subjects, logs exposures and computes experiment metrics daily. Minerva is the metric layer that defines a metric once for dashboards and experiments. Guardrails flag launches that hurt key metrics [15].

```
EXPERIMENT CONFIG (owner, hypothesis, unit, allocation, layers/exclusions,
                   primary + guardrail metrics from Minerva, MDE, duration)
       |
       v
ASSIGNMENT  hash(unit_id, experiment_salt) -> bucket -> arm
  unit: visitor/device id | user id | listing | market (cluster)
       |
       v
PRODUCT / MODEL SERVICE  --> treatment applied (e.g. ranker v2)
       |                          |
       v                          v
EXPOSURE LOG (unit, arm,     EVENT LOGS (searches, bookings, cancellations,
 ts, surface) - logged only   CS contacts, host earnings)   + identity map
 when the treatment could     (logged-out visitor -> user after login)
 change what the unit saw
       |                          |
       +-----------+-------------+
                   v
METRIC COMPUTATION (daily, Airflow/Spark)
  Minerva metric definitions -> per-unit, per-day aggregates
  join exposures to events after first exposure (attribution window)
  pre-period covariates for CUPED
                   |
                   v
ANALYSIS: SRM check -> deltas, CIs (delta method / cluster-robust)
          -> variance reduction -> sequential/fixed-horizon decision
          -> guardrails [15] -> heterogeneity cuts
                   |
                   v
DECISION: ship / iterate / kill;  holdout; long-term value translation [17]
```

How to draw and defend it. Draw assignment and exposure as two separate boxes: an interviewer will ask why. Write the unit of randomization on the assignment box and make sure the analysis box uses the same unit for its variance. Put the SRM check first in analysis, because nothing downstream is trustworthy without it. Name where interference breaks the diagram (shared listing inventory) and redraw the unit as a market or cluster.

C1. Platform design (the reported prompt)

45. Design an A/B testing platform. [Reported] [2]
    - Which parts are online (latency-critical) and which are offline (batch)?
    - How many concurrent experiments must it support, and how do they avoid colliding?
46. Design an A/B testing platform that ML teams at Airbnb would use to launch ranking, pricing and trust models. What would you add for ML that a generic platform lacks? [Likely] [1][2]
    - How does the platform link a model version to an experiment arm?
47. Draw ERF's end-to-end path from a config change to a readout. Where would you spend your first quarter of engineering effort if readouts were slow? [Likely] [14]

C2. Assignment and randomization units

48. Assignment is a hash of unit id and salt. Why a hash rather than a stored random draw, and what must the salt guarantee across experiments? [Likely] [14]
49. A guest browses logged out on mobile, then logs in on desktop and books. How do assignment and exposure work, and what does the identity merge do to your estimate? [Likely]
50. Which randomization unit fits each case: a search-ranking model, Smart Pricing suggestions for hosts, a trust model that blocks bookings, a support-routing LLM?

[Likely] [1]
    - Which of these violates SUTVA through shared inventory, and how do you redraw the assignment box?
51. Design a network-aware experiment for a "Wi-Fi badge" on listings, where treatment effects spill over between listings. [Reported] [2]
    - Cluster by market, by geohash or by listing graph? How do you trade bias against power?
52. How do you run layered or mutually exclusive experiments so that two ranking teams do not test on the same traffic? [Likely] [14]
53. Design an A/B test with causal inference for a change where randomization is imperfect or impossible (for example, a policy that applies to a whole city). [Reported] [2]
    - Where does a synthetic-control or ML counterfactual (ACE) box sit relative to the ERF diagram? [Likely] [18]

C3. Exposure logging

54. Why log exposure separately from assignment? What goes wrong if every assigned user counts in the analysis? [Likely]
    - The ranker change only affects searches with more than 50 results. Where exactly do you fire the exposure event?
55. Exposure events are lost on 2% of mobile sessions, unevenly by app version. What does that do to the readout, and how would SRM catch it? [Likely]
56. How do you log exposure for a model whose effect is server-side and invisible (for example, a fraud model that blocks a few bookings)? [Likely] [1]

C4. Metric computation (Minerva layer)

57. Design robust primary and guardrail metrics for a search-ranking launch. [Reported] [2]
    - Which metric is the decision metric, and which are guardrails that can only block?
58. Why define metrics once in a metric layer (Minerva) rather than per experiment? What breaks when the experiment readout and the finance dashboard disagree on "bookings"? [Likely] [21]
59. Compute treatment/control booking metrics from bookings and visits tables. What is the join key, and what is the attribution window? [Reported] [2]
    - A booking happened 9 days after first exposure on a different device. Does it count?
60. Build a user-level panel and run a causal regression on it. Draw where the panel is built in the pipeline and which covariates come from the pre-period. [Reported] [2]
61. Find users with seven consecutive impression days. Where would such a metric be used in experiment analysis (engagement, novelty decay)? [Reported] [2]
62. Bookings are rare and heavy-tailed in value. Which metrics would you precompute per unit-day so that analysis stays fast and variance stays honest? [Likely]
63. Metrics are recomputed daily for thousands of experiments. Design the computation so a late-arriving partition does not silently change last week's readouts. [Likely] [14]
    - How do you version metric definitions so an old readout can be reproduced?

C5. Analysis and decision

64. Walk through the analysis box in order. Why must the SRM check come first? [Likely]
65. Users are randomized but the metric is bookings per search. How does the variance computation change, and what happens if you treat searches as independent? [Likely]
66. Where does CUPED or regression adjustment plug into the pipeline, and which pre-period data must already be logged? [Likely]
67. Your offline ranking metric went up but the A/B on bookings is flat. Interviewers grade "how an offline metric predicts the online result" [1]. Walk through this diagram (and Diagram B) to find why. [Likely] [1]
    - Which of the following would you check first: exposure dilution, skew, metric sensitivity or a real null?
68. Two guardrails disagree: bookings up 0.4%, cancellations up 0.3%. How should the platform present this, and who decides? [Likely] [15]
69. A two-week test cannot see repeat bookings or host churn. How do you translate a short-term delta into long-term value, and where does that model sit in the diagram? [Likely] [17]
    - How would you validate the long-term proxy against holdouts?
70. Design the decision layer to allow peeking without inflating false positives. Sequential test, fixed horizon or Bayesian, and why? [Likely]
71. How should the platform report heterogeneous effects (new versus returning guests, markets) without inviting p-hacking? [Likely]

C6. Platform operations and scale

72. The daily ERF pipeline misses its SLA during a holiday peak. What degrades gracefully, and what must never be wrong? [Likely] [14]
73. How would you add near-real-time guardrail monitoring so a harmful model launch is stopped within hours, not days? [Likely] [15]
74. How do you audit that an experiment's ship decision was made on the pre-registered metric? [Likely]

Diagram D. Interleaving pipeline for search ranking

From Airbnb's 2022 interleaving post (URL confirmed, not opened) [16] and a 2025 paper on interleaving and counterfactual evaluation for Airbnb search (unverified) [19]. Interleaving merges two rankers' results into one list per search and credits each engaged or booked listing to the ranker that contributed it. Airbnb reported it as a faster, more sensitive screen before A/B tests [16].

```
search request (guest, query, dates)
       |
       +---> interleaving assignment (hash guest_id -> interleaving slice;
       |                              rest of traffic -> normal A/B layers)
       v
candidate retrieval (shared)  ---> ranker A scores   ranker B scores
                                        \              /
                                         v            v
                                   TEAM-DRAFT MERGE
                                   (coin flip per round; each team picks its
                                    best remaining listing; record team label)
                                         |
                                         v
                         results page + map (dedupe; pagination keeps teams)
                                         |
                                         v
LOG: search_id, guest_id, position, listing_id, team (A/B/both), ts
       |
       v
ATTRIBUTION: join clicks, wishlists, booking requests, bookings (within window)
             -> credit to team of the listing; drop "both" or split
       |
       v
PER-GUEST PREFERENCE: wins_A vs wins_B -> preference score
       |
       v
STATS: sign test / bootstrap over guests -> direction, magnitude, CI
       |
       v
GATE: winner goes to full A/B (Diagram C); track interleaving-vs-A/B agreement
```

How to draw and defend it. Show retrieval as shared and only the ordering as interleaved, so the interviewer sees interleaving compares rankers, not candidate sets. Write "team label logged per position" on the merge box, because attribution is impossible without it. Draw the gate into Diagram C explicitly and name what interleaving cannot measure (price display, result-set size, set-level diversity), which is why the A/B still runs.

D1. Mechanics

75. Explain team-draft interleaving on this diagram. Why team draft rather than balanced interleaving? [Likely] [16]
76. Search ranking rounds have been reported to push on experimentation [1]. Where in your launch funnel does interleaving go, and what gates sit before and after it? [Likely] [1][16]
    - What does the offline gate check that interleaving cannot?
77. The two rankers share most of their top results. How does that affect sensitivity, and how do you handle listings both rankers placed? [Likely] [16]
78. How do you keep a guest's interleaved experience consistent across page 2, the map view and a repeated search the next day? [Likely] [16]

D2. Attribution and metrics

79. Bookings happen days after a search, often after several searches. How do you attribute a booking to a team in an interleaved list? [Likely] [16]
    - Last search, first exposure or all exposures? Defend one.
80. Clicks are plentiful but bookings are the goal. Which interleaving metric would you trust for a go/no-go, and how would you show it agrees with A/B booking results? [Likely] [16][19]

81. Why is per-impression significance wrong here, and what is the unit of analysis for the interleaving statistics? [Likely]
82. How do you combine interleaving signals with counterfactual (off-policy) evaluation from logged data? Where does each box sit in the funnel? [Likely] [19]

D3. Validity and limits

83. Name ranking changes that interleaving cannot evaluate and must go straight to A/B. Why does each break the merge? [Likely] [16]
84. A diversity-aware ranker optimizes the whole list. Why can interleaving give a misleading verdict for it? [Likely] [16]
85. Interleaving says B wins clearly; the A/B on bookings is flat. Using Diagrams C and D, list the hypotheses in the order you would test them. [Likely] [16]
86. Can interleaving and A/B tests share traffic? Draw how you would allocate slices so neither contaminates the other. [Likely] [16]
87. How would you build a historical agreement dataset of interleaving versus A/B outcomes, and what agreement rate would make you skip the A/B for small changes? [Likely] [16][19]

D4. Infrastructure and cost

88. Interleaving doubles the scoring cost for the slice. How do you stay inside the latency budget? [Likely] [1]
89. What must be logged per position for the analysis to be reproducible, and how large is that log per day? [Likely]
90. How do you run 10 candidate rankers through interleaving in a week? Pairwise tournaments or multileaving? [Likely] [16]

Cross-diagram probes (connecting A, B, C and D)

91. Draw all four systems on one board in five minutes: where do Bighead, Chronon, ERF and interleaving touch? [Likely]
    - Which arrow carries the most risk of silent error?
92. A new ranker passes offline evaluation, wins interleaving, and loses the A/B on cancellations. Trace the path backwards through the four diagrams. [Likely] [15][16]
93. Which logs must be joined to turn one search into (a) a training row, (b) a consistency check, (c) an interleaving credit and (d) an A/B metric? Can one logging design serve all four? [Likely] [9][10]
94. The experiment arm is a feature in the model (for example, "guest saw new pricing UI"). How do you stop the next model from learning the experiment instead of the user? [Likely]
95. Rank Home search results without a text query. Point to where Chronon features, the model lifecycle and the experiment pipeline enter your design. [Reported] [2]
96. Design user-embedding semantic search. How are embeddings versioned in the registry, served from the feature path, and A/B tested when the index changes? [Reported] [2]

97. Design Airbnb search ranking from candidate generation through offline and online evaluation and deployment. Which of the four diagrams would you draw in full, and which as a single box? [Reported] [1]
98. For a pricing model (reported as a design prompt [4][5]), how do the experiment unit and the feature freshness requirements differ from search ranking? [Likely] [4][5]
99. If you owned the ML platform for a year, which metric would you use to show that Bighead, Chronon and ERF together made ML teams faster? [Likely]
100. Which piece of these four systems would you buy or adopt from open source, and which must Airbnb build itself? [Likely] [6]

Section E. Reported prompts mapped onto the four diagrams (added in review)

Each prompt below comes from a fetched source: the dated index [2], the MLE guide [1] or a 1point3acres mirror [4][26]. The sources give only the prompt titles. The diagram angle added to each is an inference for this role. The follow-up probes are [Likely] unless they cite a source.

E1. Product ML prompts: which diagram to zoom into

101. Estimate listing lifetime value (reported May 2026). Draw the label window, the feature cutoff and the batch-scoring path from Diagram A. [Reported] [2] - "Agree on what lifetime value means here" first [1]. Which definition changes the Chronon Join's label offsets [8]? - The scores land in a Hive table nightly. Who consumes them, and how would you A/B test a model whose output is used by a pricing or sales team, not shown to guests?
102. Design a system to calculate customer lifetime value. Which part of the lifecycle do you collapse when the interviewer has no time for deployment and serving? [Reported] [4]
103. Design the home page: search, availability and ranking (reported Mar 2026). Where does the availability filter sit relative to retrieval and the ranker, and which features have to be fresher than the midnight batch upload? [Reported] [2][7] - Availability changes minute to minute. Is it a feature, a filter or both? What does each choice do to the training data?
104. Design a personalized search ranking system. Show where guest-level features come from (Diagram B) and how a new ranker is screened by interleaving (Diagram D) before an A/B test (Diagram C). [Reported] [1][2] - Real-time personalization from in-session clicks: which GroupBy is TEMPORAL, and what is its window? [Likely] [7][24]
105. Pick a listing's cover photo to raise its booking rate, with cold start for new listings. Which randomization unit do you use: listing, guest or search? Why does the choice change the exposure log and the analysis? [Reported] [1] - The treatment is a listing attribute every guest sees. How do you avoid hosts seeing their photo flip between arms?
106. Find high-quality listings (reported as a phone-screen design prompt). The model's output is a score used by search, by marketing and by trust teams. Draw it as a batch-

scored entity feature. How do downstream rankers consume it point-in-time correctly? [Reported] [1][8]

107. Design a relisting system, including "how its outputs show up in online metrics and how geohashing helps". Where does geohash enter the Chronon keys, and where does it enter the experiment unit if you cluster by area? [Reported] [1][2]
108. Design a system to help hosts set prices. Draw the experiment: host-level randomization, market-level spillover through guest substitution, and the guardrails you watch. [Reported] [4] - Hosts can override the suggestion. Which metric captures adoption, and which captures the effect?
109. Design listing recommendation and price prediction (two ML design rounds in one loop). Which pieces of the platform diagrams do the two designs share, and what would you say once and refer back to in the second round? [Reported] [5]
110. Generate customer-support instructions from company policies (reported Sep 2026). Draw the lifecycle for an LLM-based system: offline evaluation set, registry entry for prompt plus model version, shadow traffic and an A/B on resolution and recontact rates. [Reported] [2] - What is the analogue of training-serving skew when the policy documents change weekly?

E2. Data and infrastructure prompts that map onto the pipelines

111. Design an ad-click event aggregation system (reported Jan 2026). Redraw it as the exposure-and-event path of Diagram C: what changes when the counts feed an experiment readout instead of billing? [Reported] [2] - Exactly-once counts versus late events: which matters more for a daily experiment readout, and which for a real-time guardrail?
112. The same ad-click aggregator, but feeding online ranking features. Compare it with Chronon's streaming write path, untiled and tiled. [Likely] [2][11]
113. Design a job scheduler (reported Mar 2026; also reported in a G9 loop). Use it as the scheduler behind ML Automator retraining and the daily ERF metric pipeline. How do you express dependencies on upstream partitions landing, and how do you backfill a failed day? [Reported] [2][26] - A retrain depends on a label partition that is three days late. What does the scheduler do, and what does the registry show?
114. Design a query system for time- and geo-filtered user activity (reported Nov 2025). Which indexes would you build so an analyst can slice an experiment readout by market and date without rescanning raw logs? [Reported] [2]
115. Optimize host listings-page aggregations (reported Jan 2026). Draw the precomputed per-host aggregates as a Chronon GroupBy. Which aggregations can be updated incrementally, and which need a periodic batch correction? [Reported] [2][7]
116. Model cumulative booking metrics and first and last booking times (reported Sep 2026). Show where these tables sit: Minerva metric source, Chronon feature source, or both, and who owns the definition. [Reported] [2][21]

E3. Metric-computation prompts behind the analysis box

117. Estimate page-visit duration from the next user event (SQL, reported Sep 2026). Where in the ERF metric pipeline would you compute it, and how do you treat the last event of a session, which has no next event? [Reported] [2] - Does a treatment that changes app navigation bias this duration metric? How would you detect that?

118. Compute browsing metrics in Python from raw logs. What would make you move this from an ad hoc script into the shared metric layer? [Reported] [2][21]
119. Compute treatment and control metrics from bookings and visits tables, then add a ratio metric (bookings per visit). How does the per-unit pre-aggregation change so the delta-method variance is computable? [Reported] [2]

E4. Chronon depth probes grounded in the fetched documentation

120. Sliding windows are exact but must store every raw event; hopping windows bound memory but miss recent events. When would you accept Chronon's sawtooth window for a 30-day booking count in search ranking? [Likely] [7]
121. A GroupBy's batch upload runs weekly to save compute, but the KV store's TTL is five days. What happens at serving time, and how would the platform stop this configuration from being deployed? [Likely] [7]
122. A listings table arrives as a stream of inserts, updates and deletes. Which aggregations (average versus max) can be corrected in place when a row is deleted, and how does the online value become correct for the others? [Likely] [7]
123. You add a feature keyed by host_id, but your log-based left table only has guest_id and listing_id. Draw the key backfill needed before the Join can compute it. [Likely] [9]
124. Why does the online fetch need to carry a row_id if the training set will be built from fetch logs? What breaks in the label join without it? [Likely] [9]
125. A modeler renames the left table of a production Join to clean up naming. What happens to the historical training data, and how should the platform guard against this? [Likely] [8]
126. The consistency job reports distributions through quantile sketches, not only mismatch rates. Which skew does a distribution comparison catch that a per-row equality check misses, and the reverse? [Likely] [10]

E5. Senior-bar framing

127. One rejection cited "jumping straight to model ideas" [3]. In a design round, how long do you spend on the label and data diagram before you name a model, and what do you draw first? [Likely] [3]
128. Senior loops are described as "close to staff level" [3]. Which platform-level trade-off (shared transforms, metric layer, interleaving gate) would you raise unprompted to show staff-level judgment, and how do you keep it short? [Likely] [3]
129. Airbnb has written about building a model to predict the value of homes [22]. If you were the first product team on a new ML platform, which platform gaps would you accept and which would you block on? [Likely] [22]
130. Airbnb's search team has written about lessons from moving search ranking to deep learning [23] and about ranking for diversity [25]. Pick one lesson, then show which box in Diagram A or D it would change. [Likely] [23][25]

References

1. "Airbnb Machine Learning Engineer Interview Process" (MLE prompts; "conversations without code"; search ranking rounds "pushed on latency, freshness

and experimentation"; postings stress training-serving skew and A/B testing). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process-machine-learning-engineer.md
2. "Airbnb" question index with report dates (Design an A/B Testing Platform, Apr 21 2026; Model Cumulative Booking Metrics and First and Last Booking Times, Sep 2026; Rank Home Search Results Without a Text Query, Sep 2026; Design User Embedding Semantic Search, Apr 2026; Find Users with Seven Consecutive Impression Days, Sep 2026; undated: primary and guardrail metrics, A/B test with causal inference, network-aware Wi-Fi badge experiment, C/T metrics from bookings and visits, panel and causal regression). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb.md
3. "Airbnb interview process" (MLE candidates get one or two ML system design rounds; 45-60 minutes; senior loops "close to staff level"; rejection for "jumping straight to model ideas"). kevin-2023-code/Tech-Interview-Questions, GitHub, 2026. Fetched. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/airbnb/interview-process.md
4. 1point3acres post #40, English mirror (MLE loop: project deep dive; ML design on customer LTV, with little time for deployment and serving; host pricing system). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/40.md
5. 1point3acres post #79, English mirror (MLE loop: ML design on listing recommendation and price prediction; "ML experience" round turned into an unplanned ML design question). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/79.md
6. "Chronon: a data platform for AI/ML" (README: GroupBy, Join, point-in-time backfills, KV-store serving, fetcher, consistency measurement). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon
7. Chronon documentation, "GroupBy" (aggregations, windows, SNAPSHOT versus TEMPORAL accuracy, online upload). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/GroupBy.md
8. Chronon documentation, "Join" (left table, entity versus event sources, Label Join and label offsets). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/Join.md
9. Chronon documentation, "Bootstrap" (log stitching, external feature backfills, production versus experimental Joins, data recovery). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/Bootstrap.md
10. Chronon documentation, "Online Offline Consistency" (causes of inconsistency, query-log table, key-subset sampling, replay and compare). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon/blob/main/docs/source/test_deploy_serve/Online_Offline_Consistency.md
11. Chronon documentation, "The Tiled Architecture" (pre-aggregated tiles via Flink, hot keys, latency). airbnb/chronon, GitHub. Fetched. https://github.com/airbnb/chronon/blob/main/docs/source/Tiled_Architecture.md
12. Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production." eugeneyan/applied-ml, GitHub. Fetched

(raw README re-read in review); used to confirm [13]-[18] and [22]-[25]. https://github.com/eugeneyan/applied-ml
13. Simha, "Zipline: Airbnb's Machine Learning Data Management Platform" (2018) and "Zipline - A Declarative Feature Engineering Framework" (2020), conference talks, YouTube. URL confirmed. https://www.youtube.com/watch?v=Tg5VEMEsC-0 ; https://www.youtube.com/watch?v=LjcKCm0G_OY
14. "Scaling Airbnb's Experimentation Platform" (ERF), Airbnb Tech Blog (Medium), 2017. URL confirmed. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166
15. "Designing Experimentation Guardrails," Airbnb Tech Blog (Medium), 2021. URL confirmed. https://medium.com/airbnb-engineering/designing-experimentation-guardrails-ed6a976ec669
16. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving," Airbnb Tech Blog (Medium), 2022. URL confirmed. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
17. "How Airbnb Measures Future Value to Standardize Tradeoffs," Airbnb Tech Blog (Medium), 2021. URL confirmed. https://medium.com/airbnb-engineering/how-airbnb-measures-future-value-to-standardize-tradeoffs-3aa99a941ba5
18. "Artificial Counterfactual Estimation (ACE): Machine Learning-Based Causal Inference at Airbnb," Airbnb Tech Blog (Medium), 2022. URL confirmed. https://medium.com/airbnb-engineering/artificial-counterfactual-estimation-ace-machine-learning-based-causal-inference-at-airbnb-ee32ee4d0512
19. Zhang et al., "Harnessing the Power of Interleaving and Counterfactual Evaluation for Airbnb Search Ranking," arXiv 2508.00751, 2025. Unverified (arxiv.org unreachable in both passes; not listed in [12]). https://arxiv.org/abs/2508.00751
20. "Bighead: Airbnb's End-to-End Machine Learning Platform," Data Council talk. Unverified (component names from prior knowledge). https://datacouncil.ai/talks/bighead-airbnbs-end-to-end-machine-learning-platform
21. "Ep 1: Robert Chang on Scaling Metrics @ Airbnb" (Minerva), dbt Roundup. Unverified. https://roundup.getdbt.com/p/ep-1-robert-chang-on-scaling-metrics
22. "Using Machine Learning to Predict Value of Homes On Airbnb," Airbnb Tech Blog (Medium), 2017. URL confirmed by [12]; page not opened. https://medium.com/airbnb-engineering/using-machine-learning-to-predict-value-of-homes-on-airbnb-9272d3d4739d
23. Haldar et al., "Applying Deep Learning To Airbnb Search," arXiv 1810.09591, 2018. URL confirmed by [12]; page not opened. https://arxiv.org/abs/1810.09591
24. "Listing Embeddings in Search Ranking" (real-time personalization using embeddings), Airbnb Tech Blog (Medium), 2018. URL confirmed by [12]; page not opened. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
25. "Learning To Rank Diversely," Airbnb Tech Blog (Medium), 2022. URL confirmed by [12]; page not opened. https://medium.com/airbnb-engineering/learning-to-rank-diversely-add6b1929621
26. 1point3acres post #80, English mirror (G9 loop: system design on group chat and a job scheduler; the experience round decided the outcome). FlowSight/coding, GitHub. Fetched. https://github.com/FlowSight/coding/blob/main/company%20wise/airbnb/1p3a/80.md